

Distributed Systems

分布式系统

中英对照学习导读

Maarten van Steen

Andrew S. Tanenbaum

Contents

前言：如何使用本导读	1
1 Introduction	2
1.1 From networked systems to distributed systems	2
1.2 Design goals	3
1.3 A simple classification	6
1.4 Pitfalls	7
1.5 Summary	7
2 Architectures	8
2.1 Architectural styles	8
2.2 Middleware and distributed systems	10
2.3 Layered-system architectures	11
2.4 Symmetrically distributed system architectures	12
2.5 Hybrid system architectures	14
2.6 Summary	15
3 Processes	17
3.1 Threads	17
3.2 Virtualization	20
3.3 Clients	23
3.4 Servers	25
3.5 Code migration	31
3.6 Summary	34
4 Communication	36
4.1 Foundations	36
4.2 Remote procedure call	39
4.3 Message-oriented communication	43
4.4 Multicast communication	49
4.5 Summary	53

5	Coordination	54
5.1	Clock synchronization	54
5.2	Logical clocks	55
5.3	Mutual exclusion	57
5.4	Election algorithms	58
5.5	Gossip-based coordination	60
5.6	Distributed event matching	62
5.7	Location systems	63
5.8	Summary	64
6	Naming	65
6.1	Names, identifiers, and addresses	65
6.2	Flat naming	66
6.3	Structured naming	67
6.4	Attribute-based naming	70
6.5	Named-data networking	72
6.6	Summary	73
7	Consistency and Replication	74
7.1	Introduction	74
7.2	Data-centric consistency models	75
7.3	Client-centric consistency models	77
7.4	Replica management	78
7.5	Consistency protocols	80
7.6	Example: Caching and replication in the Web	82
7.7	Summary	82
8	Fault Tolerance	84
8.1	Introduction to fault tolerance	84
8.2	Process resilience	85
8.3	Reliable client-server communication	89
8.4	Reliable group communication	90
8.5	Distributed commit	91
8.6	Recovery	91
8.7	Summary	93

9 Security	94
9.1 Introduction to security	94
9.2 Cryptography	95
9.3 Authentication	97
9.4 Trust in distributed systems	98
9.5 Authorization	99
9.6 Monitoring	101
9.7 Summary	102

前言：如何使用本导读

本书是 Maarten van Steen 与 Andrew S. Tanenbaum 合著 *Distributed Systems*（第 4 版，2023 / 修订版 4.03）的中英对照学习导读。它并不是原书的逐段翻译，而是按原书章节结构，用中文重述每一节的核心思想、关键术语与设计权衡，并在必要处引用原书的简短原文，辅以解读与实践清单。

为什么做成“导读”而不是“全文翻译”

原书版权归作者所有，明确禁止整书翻译或复制。因此本导读采用更利于学习的形式：中文讲解 + 关键术语对照 + 少量原文短引 + 解读/行动清单。想要完整论述、图例和习题，请对照原书阅读。

原书概览

书名	Distributed Systems, 4th edition (Version 4.03, 2025)
作者	Maarten van Steen, Andrew S. Tanenbaum
出版	作者自出版 (digital version 免费发放)
篇幅	9 章，约 680 页

原书结构：九个视角

原书刻意从不同的视角切入分布式系统，每个视角对应一章：

1. **Introduction** —— 什么是分布式系统、设计目标、分类与常见陷阱
2. **Architectures** —— 架构风格、中间件、分层/对称/混合架构
3. **Processes** —— 线程、虚拟化、客户端、服务器、代码迁移
4. **Communication** —— RPC、面向消息通信、多播
5. **Coordination** —— 时钟、逻辑时钟、互斥、选举、gossip、定位
6. **Naming** —— 扁平/结构化/基于属性的命名，DNS、DHT
7. **Consistency and replication** —— 一致性模型与副本管理
8. **Fault tolerance** —— 失败模型、共识、Paxos、提交、恢复
9. **Security** —— 密码学、认证、信任、授权、监控

贯穿全书的一条主线

分布式系统的本质困难来自部分失败（**partial failure**）与缺乏全局时钟。几乎每一章都在回答同一个问题：在进程可能失败、消息可能丢失、节点可能任意来去的条件下，我们还能对系统行为作出什么样的可证保证？

Introduction

本章确立全书的基本概念：分布式系统与去中心化系统的区别、为什么“分散”本身不是目标、六大设计目标（资源共享、分布透明、开放性、可靠性、安全、可扩展性），以及三类系统（高性能计算、信息处理、普适计算）。最后给出著名的“八个错误假设”，它们是全书反复要打破的思维惯性。

1.1 From networked systems to distributed systems

从网络系统到分布式系统

计算机从 1945 年到 1985 年之间大多是彼此孤立的大型机。转折出现在 1980 年代中期，两股力量改变了局面：强大的微处理器（从 8 位到 16/32/64 位再到多核）与高速计算机网络（LAN 与 WAN）。叠加设备小型化（智能手机、树莓派这类“纳米计算机”），联网计算迅速渗入汽车、电网、桥梁等基础设施——这也让它们变得“可被攻击”。

1.1.1 Distributed versus decentralized systems

作者给出的区分，关键在“必要（necessity）”与“充分（sufficiency）”之分：

- 去中心化（**decentralized**）：被迫分散。原因往往是行政边界、信任缺失或地理约束。三个典型例子：联邦学习（数据不能出组织边界，于是把训练送到数据那里）、区块链/分布式账本（参与方互不信任，交易必须公开可验证）、地理上天然分散的监控系统（卫星、交通）。
- 分布式（**distributed**）：主动分散，以换取可扩展性与容错。典型例子：Gmail（逻辑上只有两台服务器，背后是海量机器）、CDN（Akamai，2022 年超 40 万台服务器）、家用 NAS（多用户共享文件的迷你分布式系统）。

原文精读

A *decentralized system* is a networked computer system in which processes and resources are *necessarily* spread across multiple computers.

一个去中心化系统，是进程与资源必然分散在多个计算机上的网络系统。

A *distributed system* is a networked computer system in which processes and resources are *sufficiently* spread across multiple computers.

一个分布式系统，是进程与资源充分分散在多个计算机上的网络系统。

原文定位：§1.1.1，原书 p.4

Note 1.1：中心化就一定差吗？

常见误解：中心化无法扩展、且必然带来单点故障。作者强调必须区分 逻辑设计与物理设计。DNS 逻辑上是一棵树，物理上根节点由 13 台根服务器实现、每台又是集群；它高度副本化，几乎不可能被单点打垮。中心化反而因为“只有一个故障点”而更容易管理、加固，云（cloud）就是

高度可扩展又稳健的中心化方案。

1.1.2 Why the distinction is relevant

中心化方案通常更简单。把服务拆散是一个需要慎重权衡的决定，因为分布式/去中心化系统天生困难：存在大量非预期依赖；几乎持续处于部分失败（**partial failure**）状态；节点不断来去、系统高度动态；跨网络跨域使其易受攻击。

解读 全书立场：“分散”永远不能是目的本身，原则是能少分散就少分散（追求“充分”而非“必然”）；能简单就简单，因此本书基本不讨论优化——作者认为优化带来的复杂度代价高于其性能收益。

原文精读

Throughout this book, we take the standpoint that decentralization can never be a goal in itself.

贯穿本书的立场是：去中心化永远不能成为目的本身。

In principle, the less spreading, the better.

原则上，分散得越少越好。

原文定位：§1.1.2，原书 p.8

1.1.3 Studying distributed systems

由于依赖关系无处不在，本书选择从一组有限的、互不相同的视角切入，每章一个视角：架构、进程、通信、协调、命名、一致性与副本、容错、安全。

原文精读

For example, there is no such thing as a separate communication module, or a separate security module.

例如，并不存在一个独立的“通信模块”，也不存在一个独立的“安全模块”。

Each perspective is considered in a separate chapter.

每一个视角都在单独的一章中讨论。

原文定位：§1.1.3，原书 p.8-10

1.2 Design goals

设计目标

1.2.1 资源共享（Resource sharing）

目标：让用户与应用方便地访问和共享远程资源（外设、存储、数据、文件、服务、网络）。动机包括经济性（共享一套高端存储比人人各买一套便宜）、协作（协同编辑/视频会议、跨国软件外包）、P2P 文件共享（BitTorrent）。共享已高度“无感”：第三方维护的共享文件夹，用起来几乎和本机文件夹无异。

原文定位：§1.2.1，原书 p.10-11

1.2.2 分布透明 (Distribution transparency)

目标：隐藏进程与资源物理上分散在多地这一事实。实现方式是 中间件 (middleware)，它为应用提供处处一致的接口，把“在哪、怎么访问”藏起来。原书（引 ISO 1995）列出七种透明性：访问、位置、重定位、迁移、复制、并发、失败。其中失败透明的核心困难是：无法区分“死进程”与“只是很慢的进程”。

原文精读

An important goal of a distributed system is to hide the fact that its processes and resources are physically distributed across multiple computers.

分布式系统的一个重要目标，是隐藏“其进程与资源在物理上分散于多台计算机”这一事实。

The price for achieving full transparency may be surprisingly high.

实现完全透明所付出的代价，可能高得惊人。

原文定位：§1.2.2，原书 p.11-15（透明性表见图 1.3）

关键权衡：透明到什么程度？

透明并非越多越好。跨时区的“早晨报纸”不该假装还按你本地作息；旧金山到阿姆斯特丹的往返延迟有物理下限（光速约 35ms 起，实测几百毫秒）；“跨大洲永远一致”的复制会让单次更新耗时数秒，藏不住。作者主张：透明应和性能、可理解性一起权衡，有时显式暴露分布反而更好（如基于位置的服务）。Waldo 等（1997）指出，用 RPC 掩盖分布会导致难以理解的语义，因为过程调用一旦跨失败链路，性质就变了。

1.2.3 开放性 (Openness)

开放的分布式系统：其组件能被别的系统方便地使用/集成，自身也常由“来自别处”的组件构成。接口通常用 IDL 描述，能精确刻画语法，但语义往往只能用自然语言非正式描述。理想规范应完整且中立，从而带来互操作性 (interoperability)、可移植性 (portability)、可扩展性 (extensibility)。实现灵活性的关键是策略 (policy) 与机制 (mechanism) 分离。

原文精读

To be open means that components should adhere to standard rules that describe the syntax and semantics of what those components have to offer.

所谓开放，意味着组件应当遵循统一的规则，这些规则描述了组件所提供的语法与语义。

What we need is a separation between policy and mechanism.

我们需要的，是把策略与机制分离开来。

原文定位：§1.2.3，原书 p.15-18（Note 1.4 讨论分离过度的代价）

1.2.4 可靠性 (Dependability)

可靠性指系统被信赖按预期运行的程度。分布式系统中由于部分失败，应假设任意时刻总有部分失败在发生，核心目标是掩盖失败及其恢复，即容错 (fault tolerance)。要求包括可用性、可靠性、安全性 (safety)、可维护性。故障三层次：fault（成因）→ error（可能引发失败的状态）→ failure（无法履约）。故障按时序分 瞬时/间歇/永久。

原文精读

Availability is defined as the property that a system is ready to be used immediately.

可用性被定义为：系统随时处于可供立即使用的状态。

Reliability refers to the property that a system can run continuously without failure.

可靠性指的是：系统能够持续无故障地运行。

原文定位：§1.2.4，原书 p.18-21

可用性 ≠ 可靠性

每小时随机宕机 1 毫秒的系统，可用性超过 99.9999%，却不可靠；从不崩溃、但每年 8 月停机两周的系统，可靠性高，可用性只有 96%。常用度量：MTTF（平均无故障时间）、MTTR（平均修复时间）、MTBF = MTTF + MTTR。

1.2.5 安全 (Security)

“不安全的分布式系统就不可靠。”核心是认证 (**authentication**，验证所声称的身份) 与信任 (**trust**)。本节只引入最基本密码学工具（详见第 9 章）：对称密码（加解密同一密钥）、非对称/公钥密码（公钥加密、私钥解密；反向使用即数字签名）、哈希函数（定长、改一点就变、不可反推）。

原文精读

In a symmetric cryptosystem, encryption and decryption takes place with a single key.

在对称密码系统中，加密与解密使用同一把密钥。

In an asymmetric cryptosystem, the keys used for encryption and decryption are different.

在非对称密码系统中，用于加密和解密的密钥是不同的。

原文定位：§1.2.5，原书 p.21-23（记号表见图 1.4）

1.2.6 可扩展性 (Scalability)

按三个维度衡量 (Neuman 1994)：规模 (**size**)、地理 (**geographical**)、行政 (**administrative**)。规模瓶颈的根因是 CPU、存储与 I/O、网络；地理的难点是同步通信延迟、可靠性、带宽与缺乏多点通信；行政最棘手，涉及政策冲突。向外扩展 (scale out) 只有三种手段：隐藏通信延迟、分区与分布、复制。复制（含缓存，区别在于缓存由使用者而非所有者决定）的代价是一致性。

原文精读

A system can be scalable regarding its size, meaning that we can easily add more users and resources to the system without any noticeable loss of performance.

系统在规模上可扩展，意味着我们可以轻松地的系统加入更多用户与资源，而性能没有可察觉的下降。

There are basically only three techniques we can apply: hiding communication latencies, distribution of work, and replication.

我们基本上只能采用三种技术：隐藏通信延迟、分摊工作、以及复制。

原文定位：§1.2.6，原书 p.24-32（Note 1.5 用排队论分析规模可扩展性）

1.3 A simple classification

分布式系统的简单分类

1.3.1 高性能分布式计算

集群计算 (cluster): 硬件同构, 一批相似节点经高速互连, 用于并行编程; 现流行多内核 (**multikernel**)——满配 OS 与轻量内核并存。**网格计算 (grid)**: 异构, 把不同组织的资源聚成虚拟组织 (**virtual organization**)。Foster 等的四层架构: fabric / connectivity / resource / collective, 上接 application; 中间三层即“网格中间件”。

原文定位: §1.3.1, 原书 p.32–37 (网格四层见图 1.10)

1.3.2 分布式信息系统

分布式事务处理: 事务满足 **ACID** (原子、一致、隔离、持久)。分布式事务常拆成嵌套事务: 即使某子事务已提交、其父事务后来中止, 该子事务结果也必须回滚——所以“持久性”只对顶层事务成立。核心组件是 **TP monitor**, 按分布式提交协议协调子事务提交。企业应用集成 (**EAI**) 的常见方式: **RPC**、**RMI**、**MOM** (发布-订阅)。

原文精读

The characteristic feature of a transaction is either all of these operations are executed or none are executed.

事务的特征是: 这些操作要么全部执行, 要么全部不执行。

A transaction-processing monitor's main task is to allow an application to access multiple server/databases by offering it a transactional programming model.

事务处理监视器 (TP monitor) 的主要任务, 是通过提供一个事务式编程模型, 让应用得以访问多个服务器/数据库。

原文定位: §1.3.2, 原书 p.37–43 (Note 1.7 讨论四种集成方式)

1.3.3 普适系统 (Pervasive systems)

因移动/嵌入式设备而不同于前两类: 用户与组件的边界模糊, 没有单一屏/键接口, 而是大量传感器与执行器。三类 (有重叠):

- **泛在计算 (ubiquitous)**: 系统持续存在, 交互多为“隐式输入”。Poslad (2009) 五项要求: 分布、交互、情境感知、自治、智能。
- **移动计算 (mobile)**: 设备多样且位置随时间变化; MANET 未流行, 移动计算回归“移动设备连到服务器”, 由此引出 **MEC** (移动边缘计算)。
- **传感器网络 (sensor networks)**: 数十至上千小节点, 资源与能耗受限; 关键是把网络当作一个数据库 (TinyDB 声明式接口), 做网内处理 (**in-network processing**)——沿树上传并在汇合处聚合。

由 IoT 与云连接, 形成云-边-物层级 (图 1.17): 越高层越可靠、容量越大、性能越强; 越低层越易实现位置相关能力与低延迟, 设备数量也越多。

原文定位: §1.3.3, 原书 p.43–52 (云-边-物层级见图 1.17)

1.4 Pitfalls

陷阱：八个错误假设

Peter Deutsch（当年在 Sun）总结了人们初次开发分布式应用时的八个错误假设：

1. The network is reliable（网络可靠）
2. The network is secure（网络安全）
3. The network is homogeneous（网络同构）
4. The topology does not change（拓扑不变）
5. Latency is zero（延迟为零）
6. Bandwidth is infinite（带宽无限）
7. Transport cost is zero（传输零成本）
8. There is one administrator（只有一个管理员）

为什么这八条是全书索引

它们正好对应分布式系统独有的性质：可靠性、安全性、异构性、拓扑；延迟与带宽；传输成本；行政域。本书几乎每个原理，都是在反驳其中某条假设所产生的问题——例如“可靠网络不存在”导致失败透明不可能；“网络本就不安全”单独成章；复制是为解决延迟与带宽问题；管理问题穿插在各处。

原文定位：§1.4，原书 p.52-53

1.5 Summary

小结

- 分布式系统 = 进程与资源充分分散；去中心化系统 = 必然分散。分散本身不是目标，多数分布式选择是为提升可靠性、可扩展性、效率；但中心化系统往往更好管理，做决定前要三思——有时又别无选择。
- 设计目标：共享资源、保证开放、日益重要的安全，以及隐藏分布细节。透明有性能代价且永远无法完全实现；它与可扩展性常相互冲突。
- 系统分类：面向计算（集群 → 网格 → 云）、面向信息处理、面向普适（移动、传感器、泛在）。
- 反复出现的陷阱：开发者对底层网络作根本错误的假设，事后很难掩盖由此产生的不良行为。

原文定位：§1.5，原书 p.53-54

实践清单

- 读原书第 1 章对应小节，重点看图 1.1（三种组织）、图 1.3（透明性表）、图 1.10（网格四层）、图 1.17（云-边-物层级）。
- 用“必然 vs 充分”重新审视一个你熟悉的系统（如微信/网盘）：哪些分散是被迫的？哪些是主动的？
- 把你正在做或想做的项目，逐条对照八个错误假设，写下你默认了哪几条。
- 记住三个“反直觉结论”：中心化不必差、透明不是越多越好、可用性 ≠ 可靠性。

Architectures

本章分三层讲“怎么组织”：软件架构（逻辑上组件如何组织与交互，即架构风格）、中间件（把应用与平台隔开、提供分布透明，以及如何让它可改）、系统架构（组件实际放在哪些机器上：分层、对称/对等、混合）。

2.1 Architectural styles

架构风格

软件架构关注逻辑组织。所谓架构风格（**architectural style**），是用组件（**component**）、组件的连接方式、交换的数据，以及它们如何组合成系统来刻画的。组件是可替换的模块，替换的前提是接口不变（系统通常不能停机维护）。连接器（**connector**）则中介组件间的通信/协调/协作，如过程调用、消息传递、流数据。三种主要风格：分层、面向服务、发布-订阅；现实中多为组合，尤其“分层”几乎无处不在。

2.1.1 分层架构（Layered architectures）

组件按层组织：高层 L_j 向低层 L_i ($i < j$) 发 **downcall** 并期待应答；**upcall**（低层调用高层）是例外。三种形态：纯分层（只向下一层调用，如网络通信）、混合分层（多个高层共享同一低层，如应用与数学库共用 OS 库）、带 upcall（如 OS 通过回调句柄通知应用事件）。

原文精读

A component is a modular unit with well-defined required and provided interfaces that is replaceable within its environment.

组件是一个模块化单元，具有明确定义的所需接口与所提供接口，并可在其环境中被替换。

A connector is generally described as a mechanism that mediates communication, coordination, or cooperation among components.

连接器通常被描述为一种机制，用于在组件之间中介通信、协调或协作。

分层通信协议是经典例子。关键要区分三件事：一层提供什么服务、服务通过什么接口暴露、以及为实现通信所遵循的协议。以可靠、面向连接的服务为例：通信前须先建立连接，保证消息不丢且按序到达——Internet 中由 **TCP** 实现。协议规定握手/拆链的消息、保序与纠错的做法；服务以简单的编程接口（连接、收发、关闭）提供；而该协议有多种实现与接口（如 Python 的 `socket`）。

原文精读

It is important to understand the difference between a service offered by a layer, the interface by which that service is made available, and the protocol that a

重要的是理解三者的区别：一层所提供的服务、暴露该服务所用的接口、以及该层为实现通信所遵循的协议。

layer implements to establish communication.

原文定位：§2.1.1，原书 p.57–62（图 2.1 三种分层、图 2.2 服务/接口/协议、图 2.3 socket 示例）

应用分层：大量分布式应用服务于“访问数据库”，可划为三个逻辑层次——应用接口层、处理层、数据层。以房产搜索引擎为例：接口层收集城市/价格/房型等条件，处理层把它们转成数据库查询、取出结果并排序生成 HTML；本书官网的“生成个性化 PDF”也类似，处理层（每 5 分钟生成一次、把邮箱嵌进图里）的计算量远超数据层与接口层。

2.1.2 面向服务的架构 (Service-oriented architectures)

分层的主要缺点是层间强依赖。反面教材：一个仅用于“补零/补空格”的小组件被从 NPM 撤下，导致成千上万程序受影响 (Kucharski 2020)。由此转向把系统组织成一批松散、独立的实体，每个实体封装一个服务（无论叫服务、对象还是微服务，都以独立进程/线程运行）。

- **基于对象 (object-based)**：对象即组件，通过过程调用机制连接，调用可跨网络。接口与实现分离，于是接口在一台机器、对象在另一台机器，即远程对象/分布式对象。客户端绑定后会把一个 **proxy**（代理）载入其地址空间（相当于 RPC 的 client stub），只负责把方法调用打包成消息、把应答解包返回；服务端的 **skeleton**（骨架）则把请求解包、在对象接口上发起调用并打包返回值。要点：分布式对象的状态不分散，仍驻留单机，只有接口在别处可用。
- **微服务 (microservices)**：每个微服务作为独立的（网络）进程运行，受 Unix“许多小工具可组合”启发。核心是真正的独立与模块化；至于“多大算微”众说纷纭。因为独立成进程，就产生了“放在哪里”的编排问题。
- **粗粒度服务组合**：与微服务的（非严格）区别在于，服务未必同属一个组织（如业务应用使用云厂商的存储服务）。例：卖电子书的网站，下单、发货、支付分属不同服务，支付甚至由另一组织提供、把顾客重定向过去。一般服务往往较大，把“微服务集合”实现成一个服务也很常见。难点回归到“服务组合”，与第 1 章的企业应用集成同构。
- **基于资源 (REST)**：把系统看成一大批由组件分别管理的资源。RESTful 架构四要点：单一命名方案标识资源、所有服务同一接口（最多四个操作）、消息完全自描述、执行后服务忘记调用者（无状态执行）。四个操作：**PUT**（修改，传新状态）、**POST**（创建）、**GET**（读取）、**DELETE**（删除）。例：Amazon S3，对象/桶，<https://s3.amazonaws.com/BucketName/ObjectName>，用 HTTP 的 PUT/GET 操作。REST 的简单性也可能限制复杂通信（如需要分布式事务），优点是接口通用、易容纳变化；缺点是弱化了编译期检查与语义表达（参数空间被塞进字符串）。

原文精读

A characteristic, but somewhat counterintuitive, feature of most distributed objects is that their state is not distributed: it resides at a single machine.

大多数分布式对象的一个特征——也稍显反直觉——是：它们的状态并不分散，而是驻留在单台机器上。

Resources are identified through a single naming scheme.

资源通过单一命名方案来标识。

原文定位：§2.1.2，原书 p.62–68（图 2.6 远程对象、图 2.7 REST 四操作）

2.1.3 发布-订阅架构 (Publish-subscribe architectures)

目标是让进程间依赖尽量松，把处理与协调强分开。用两个维度刻画协调模型 (Cabri 等 2000)：时间耦合/解耦与引用耦合/解耦：

- 直接协调（时间耦合 + 引用耦合）：通信双方须同时在线、且知道对方名字（如手机通话）。
- 邮箱协调（时间解耦 + 引用耦合）：把消息放进（可能共享的）邮箱，无需同时在线，但必须显式指定邮箱。
- 事件驱动（时间耦合 + 引用解耦）：进程互不知道，只发布通知描述事件，其他进程订阅某类通知；理想情况下通知精确送达订阅者，但订阅者一般须在发布时在线。
- 共享数据空间（时间解耦 + 引用解耦）：进程完全通过元组（**tuple**，类似数据库的一行）通信，用搜索模式匹配元组（关联式检索），匹配到的元组被取出。

关键：进程之间没有显式引用，靠事件总线匹配发布者与订阅者（图 2.10）。经典实现是 Linda 的 **tuple space**，三操作：**out(t)** 放入、**rd(t)** 读取（阻塞）、**in(t)** 取出（阻塞）。订阅描述分 **topic-based**（属性-值对）与 **content-based**（属性-范围对，甚至类 SQL 谓词）——表达力越强，匹配越难。实现的关键难点是高效可扩展地把订阅匹配到通知；作者提醒：一旦涉及安全与隐私，实现很容易成为瓶颈。

原文精读

In an ideal event-based coordination model, a published notification will be delivered exactly to those processes that have subscribed to it.

在理想的事件驱动协调模型中，一条已发布的通知将恰好投递给订阅了它的那些进程。

The key idea is that processes communicate entirely through tuples.

其核心思想是：进程完全通过元组进行通信。

原文定位：§2.1.3，原书 p.68–73（图 2.9 协调模型、图 2.10 事件总线/共享数据空间、Linda 示例图 2.11）

2.2 Middleware and distributed systems

中间件与分布式系统

中间件是逻辑上介于应用与各机器的 **OS** 之间的软件层（跨多台机器、对每个应用提供同一接口）。它之于分布式系统，犹如 OS 之于单机：资源管理器 + 一批通用服务（应用间通信、安全、计费、失败掩盖与恢复）。这些服务多数对很多应用都有用，故中间件也可看作“通用组件的容器”。（Note 2.4: middleware 一词最早见于 1968 年 NATO 软件工程会议报告。）

2.2.1 中间件的组织

两种关键设计模式，目标都是开放性：

- 包装器（**wrapper/adaptor**）：把组件原生接口转换成客户可接受的接口。例：对象适配器让应用调用“其实是关系数据库上的库函数”的远程对象；S3 的 REST 接口本质就是一个适配器。但逐对开发包装器不可扩展—— N 个应用理论上需 $O(N^2)$ 个包装器。用 **broker**（消息代理）集中转发可降到 $O(N)$ 。
- 拦截器（**interceptor**）：打断常规控制流、插入应用特定代码，是适配中间件的主要手段。三步式远程对象调用：本地同接口调用 → 转成通用对象调用 → 转成传输层消息。在“通用调用”这层插入请求级拦截器，就能处理“对象 B 被复制、每次调用要对所有副本发起”，而对象 A 与对象中间件都无需知道复制；在消息层插入消息级拦截器，可对大参数做分片等。

原文精读

Interceptors are a primary means for adapting

拦截器是把中间件适配到某个应用特定需求的主

middleware to the specific needs of an application. 要手段。

2.2.2 可修改的中间件 (Modifiable middleware)

运行环境持续变化（移动性、网络服务质量波动、硬件故障、电量耗尽……），把“应对变化”的责任从应用移到中间件，且系统大到不能停机，必须在线修改。于是有了可修改中间件（Parlavantzas & Coulson 2007）：不仅要自适应，还要能 有目的地改而不停服。主流做法是按组件动态组装中间件，需要迟绑定；难点在于“替换某组件会如何影响其他组件”往往不清楚——组件并没有想象中那么独立。底线：至少要在运行时能装卸组件，并显式声明每个组件提供/需要的接口；若组件间还保持状态，更需特殊处理。

原文定位：§2.2，原书 p.73–78（图 2.13 中间件层、图 2.14 broker 降包装器数、图 2.15 拦截器）

2.3 Layered-system architectures

分层系统架构

系统架构 = 组件放在哪些机器上。先从最容易理解的客户端-服务器开始。

2.3.1 简单客户端-服务器架构

服务器实现某服务；客户端发请求并等待应答（request-reply）。网络较可靠时可用无连接协议（高效，但难抗偶尔的传输失败）：客户端超时重发时，无法区分“请求丢了”还是“应答丢了”，若是后者会重复执行。可安全重复执行的操作叫 幂等（**idempotent**）（如“查余额”），不可重复的（如“转账”）则宁可报错。另一种是可靠、面向连接的协议（几乎所有 Internet 应用协议基于 TCP/IP），但建立/拆除连接有开销。客户/服务器的界限常常模糊——数据库服务器可能同时是文件服务器的客户端。

原文精读

When an operation can be repeated multiple times without harm, it is said to be idempotent.

当一个操作可以被重复执行多次而不造成损害时，就说它是幂等的。

2.3.2 多层架构 (Multitiered)

把三层（用户界面、处理、数据）铺到机器上：最简是两层——客户端只有界面、服务器包办处理与数据（瘦客户端，如“哑终端”）。(a)–(e) 展示界面/应用/数据库在客户端与服务器间的不同切分：(a) 仅终端相关部分在客户端；(b) 整个界面在客户端；(c) 把部分应用移到前端（如表单校验、字处理器的基础编辑在本地、拼写语法检查在服务器）；(d)(e) 应用主体在客户端、文件/数据库操作去服务器（e 还在本地磁盘缓存数据，如浏览器缓存）。

Note 2.5: 存在“最佳组织”吗？

趋势是从 (d)(e) 这种“胖客户端”转向把处理与存储放到服务器端。原因：客户端机器虽强，但难管理（要应对各种终端用户行为、依赖其平台、需维护多版本）。不过“胖客户端”不会消失——办公套件、多媒体、离线场景仍需要它；而且现代 Web 技术让动态部署客户端脚本变得容易、平台依赖减弱，“管理复杂”这一反对理由已常不成立（虚拟桌面环境，见第 3 章）。另外，放弃胖客户

端并不等于不需要分布式系统：服务器端本身正变得越来越分布式（云计算即例）。

当服务器也需要充当客户端（如应用服务器 AS 向数据库服务器 DS 请求，图 2.18），就得到三层架构。典型场景：事务处理（TP monitor 协调跨数据服务器的事务）、网站组织（Web 服务器 → 应用服务器 → 数据库服务器）。

原文精读

In this organization everything is handled by the server while the client is essentially no more than a dumb terminal.

在这种组织中，一切都由服务器处理，而客户端本质上不过是一个哑终端。

2.3.3 例：网络文件系统 NFS

每个 NFS 服务器对本地文件系统提供标准化视图，使异构的进程（不同 OS/机器）共享同一文件系统。两种模型：远程访问模型（**remote access**）——客户端透明访问服务器上的文件，实际的写/读操作都在服务器执行；上传/下载模型——客户端把文件下载到本地操作、完成后再传回（如 FTP）。NFS 采用远程访问模型，实践中两大优点位于 **VFS**（虚拟文件系统）：本地 Unix 文件接口被换成 VFS 接口，操作要么交给本地文件系统、要么交给 **NFS client**（用 **RPC** 与服务器通信）；服务端 **NFS server** 把入站请求转成 VFS 操作。好处是 NFS 基本独立于本地文件系统（Unix/Windows/MS-DOS 皆可），只要符合 NFS 的模型（MS-DOS 的短文件名无法完全透明地实现 NFS 服务器）。

原文精读

The basic idea behind NFS is that each file server provides a standardized view of its local file system.

NFS 的基本思想是：每个文件服务器对其本地文件系统提供一个标准化的视图。

2.3.4 例：Web

Web 的架构并不特殊，但能看出从“支持分布式文档”到“支持服务”的演化。简单 Web 系统仍是经典客户端-服务器：URL 内嵌服务器 DNS 名与文件名、并指明应用层协议；浏览器与服务器遵循 **HTTP**。文档从纯文本、到 **HTML** 标记、再到嵌入 **JavaScript** 脚本（增强交互）。当服务器不止“取文档”：**CGI** 让服务器按用户数据执行程序（把“取文档”委派给外部程序）；服务端脚本（如 **PHP**）则在本地取到文档后先执行脚本、把结果并入文档再发给客户端（脚本本身不发送）。文档因此变得完全动态。

原文定位：§2.3，原书 p.78-88（图 2.17 两层组织、图 2.18 三层、图 2.19 两种文件模型、图 2.20 NFS、图 2.22 CGI）

2.4 Symmetrically distributed system architectures

对称分布式系统架构

垂直分布 (vertical)：把逻辑上不同的组件放到不同机器（多层架构即此，类比数据库的“按列切分”。
水平分布 (horizontal)：把客户端/服务器拆成逻辑等价的多份，各处理一部分数据以均衡负载——这正是对等（**peer-to-peer**）系统。P2P 中进程都“平等”，同时充当客户端与服务器（即 **servant**）。核心问题是如何组织覆盖网络（**overlay network**）：节点是进程、边是可能的通信信道（常为 TCP 连接）。分结构化与非结构化两类。

2.4.1 结构化 P2P

节点按确定的拓扑组织（环、二叉树、网格等）以高效查找。基础是无语义索引（**semantic-free index**）：每个数据项唯一关联一个 key， $\text{key} = \text{hash}(\text{value})$ ，系统整体负责存 $(\text{key}, \text{value})$ 对，每个节点负责一部分 key，于是系统实现了一个 **分布式哈希表 (DHT)**。查找归结为 $\text{existing node} = \text{lookup}(\text{key})$ ，靠拓扑高效路由。

原文精读

In a structured peer-to-peer system the nodes are organized in an overlay that adheres to a specific, deterministic topology.

在结构化对等系统中，节点被组织在一个遵循特定、确定性拓扑的覆盖网络中。

In essence, the system is thus seen to implement a distributed hash table.

本质上，这样的系统实现了（就构成了）一个分布式哈希表。

例：超立方体路由（把 key 哈希到 16 个节点之一，逐位靠近目标）。更现实的 **Chord** (Stoica 等 2003)：节点排成环，key k 映射到标识符 $\geq k$ 的最小节点 $\text{succ}(k)$ ；每个节点维护若干快捷边，使任意两点最短路由为 $O(\log N)$ 。查找时“尽量往远跳但不越过目标”。加入：联系任一节点查出 $\text{succ}(u)$ 、把自己插到其前驱与它之间，并迁移相应数据；离开同样简单。

2.4.2 非结构化 P2P

每个节点维护临时（**ad hoc**）邻居表，覆盖网络近似随机图。无法按预定路线查找，只能搜索。两种极端：

- 泛洪（**flooding**）：把请求转给所有邻居，已见过的请求忽略；用 **TTL** 限制跳数（太小到不了、太大太贵），或从 $\text{TTL}=1$ 逐轮增大。
- 随机游走（**random walk**）：每次只问一个随机邻居，网络流量小、但可能很久才找到；同时启动 n 条可把等待时间约降到 $1/n$ （ $n = 16$ 或 64 即有效）。

Note 2.7: 为什么随机游走可行

数据通常是被复制的。设 N 个节点、每项复制到 r 个随机节点，则平均搜索规模 $S \approx N/r$ 。当 $r/N = 0.1\%$ 时 $S \approx 1000$ ；而泛洪到 $d = 10$ 个邻居需至少 4 步、触及约 7290 个节点。只有 d 很大（如 33）时泛洪才与之相当。代价是随机游走回答更慢。

介于两者之间是基于策略的方法（如优先记住常给出好结果的邻居）。

2.4.3 分层组织的 P2P

非结构化 P2P 随规模增长难以定位数据（只能搜）。办法是引入维护索引/充当 broker 的 **超级节点 (super peer)**，形成分层组织：普通弱节点（**weak peer**）作为客户端连到某个超级节点，往来通信都经它。绑定通常固定（弱节点加入时选一个、离开前不变），故期望超级节点长寿高可用，可用“每个超级节点配一个备份、弱节点同时连两个”补偿。更灵活的做法是让弱节点连到“维护其感兴趣文件索引”的超级节点，并允许随发现更好的超级节点而改绑。新问题：如何选出有资格的超级节点——与领导者选举密切相关（见 §5.4）。

2.4.4 例：BitTorrent

(大体) 非结构化的文件共享系统。用户下载一个文件的分块直到拼成完整文件。设计目标是促进协作，防止只下载不上传的 **free riding**：只有在下载者同时向他人提供内容时才允许其下载（若 P 发现 Q 下载多于上传，可降低发给 Q 的速率）。流程：用户访问全局目录（少数知名网站）拿到 **torrent** 文件，其中含指向 **tracker**（记录拥有该文件分块的活跃节点）的链接。**tracker** 易成瓶颈，改进版让节点加入一个 **DHT** 来分担跟踪功能（用 **magnet link** 在 DHT 中查找初始 **tracker**）。

原文精读

An important design goal was to ensure collaboration.

一个重要的设计目标是确保（参与者之间的）协作。

原文定位：§2.4，原书 p.88-98（图 2.23 超立方体、图 2.24 Chord、图 2.25 超级节点、图 2.26 BitTorrent）

2.5 Hybrid system architectures

混合系统架构

现实系统多是集中式 + P2P + 分层的混合，且常跨组织边界。

2.5.1 云计算（Cloud computing）

源于效用计算（**utility computing**）（把任务上传到数据中心、按资源计费）。Vaquero 等（2008）：云计算以易于使用且可访问的虚拟化资源池为特征，可动态配置，从而提供可扩展的基础；按 **pay-per-use** 模型，用定制的 **SLA** 提供保证。云分四层：**Hardware**（处理器/路由/供电/制冷）、**Infrastructure**（用虚拟化提供虚拟存储与计算资源）、**Platform**（给开发者类似 OS 的能力与厂商 API，如 S3、**exec** 家族）、**Application**（真正运行的应用）。据此有三种服务：**IaaS**、**PaaS**、**SaaS**。从系统架构看，云可视为高度先进的客户端-服务器架构，但服务器实现对客户端完全隐藏（位置不明、常高度分布式）；**FaaS** 更进一步——客户端连服务器都不用操心就能跑代码。

2.5.2 边缘-云架构（Edge-cloud）

IoT 兴起后需要“云之外的东西”。边缘计算把服务放到“网络边缘”（如企业网与 Internet 的边界、校园网）。其存在理由要批判地看：

- 延迟与带宽：带宽持续增长，用“带宽不足”当理由越来越站不住脚，但视频等服务质量场景确实需要靠近终端；延迟是更硬的理由（到达云可能要 100ms，半自动驾驶等实时应用无法接受；车与车可在路口附近通过本地边缘设施互相“显形”）。
- 可靠性：有人认为云连接不够可靠，但很多场景连接其实很好；医院/工厂等关键场景本就有其它措施，边缘是否带来新东西未必清楚。
- 安全与隐私：“云不安全则边缘也未必安全”；边缘确有“数据在组织边界内”的保护优势，但挡不住内部攻击。更硬的理由是法规——某些数据（如病历）根本不允许上云，只能自建边缘设施。

引入边缘层会打开“潘多拉魔盒”：客户组织必须决定什么放哪里，而边缘相比云资源更少、异构更强、负载更动态，对编排（**orchestration**）的需求更高。编排要点：资源分配、服务放置（如视频会议编码放在边缘）、边缘选择（不一定最近，还要看与云的连通性等）。

原文精读

Overcoming latency is one of the most compelling reasons for developing edge infrastructures.

克服延迟，是发展边缘基础设施最具说服力的理由之一。

2.5.3 区块链架构 (Blockchain architectures)

区块链使交易的登记成为可能，故也称分布式账本。银行是可信第三方；区块链的设计前提是参与方原则上不可信，因而排除可信第三方。参与者共同在公开账本中登记交易，任何人都能看到并验证（如检查一枚币是否已被花掉）。流程：Alice 广播转账意图 (Step 1) → 验证者把若干交易打包成块以提高效率 (Step 2) → 验证并不可变地附加到链上、把块广播给所有参与者 (Step 3)。逻辑上只有单一链，每个块不可变（改动必被发现），因而适合海量复制（每人本地存一份）。

系统的关键差异在于谁有资格验证，这需要（分布式）共识，有三种组织：

1. 集中式：可信第三方验证（不符合区块链目标）；
2. 分布式（许可链 **permissioned**）：一小组被许可的节点验证，假设至多 $k \leq (n-1)/3$ 个节点失败/作恶；缺点是 n 通常只有几十。
3. 完全去中心化（无许可链 **permissionless**）：所有愿验证的节点通过领导者选举参与，选出的领导者追加块（常有奖励）；领导者选举本身开销大。

原文精读

An important design assumption for blockchains is that participating parties can, in principle, not be trusted.

区块链的一个重要设计前提是：参与方原则上不可被信任。

Each block is immutable, in the sense that if an adversary decides to modify any transaction from any block in that chain, the modification can never go unnoticed.

每个块都是不可变的——只要攻击者改动了链上任一块中的任一交易，这种改动就绝不可能不被发现。

原文定位：§2.5，原书 p.98–108（图 2.27 云的四层、图 2.28 边缘-云、图 2.29 区块链流程、图 2.30 三种组织）

2.6 Summary

小结

- 区分软件架构（逻辑组织：组件如何交互、如何独立化）与系统架构（组件实际放在哪些机器上）。
- 架构风格反映组织组件交互的基本原则，重要者有分层、面向服务、以及以事件处理为特征的发布-订阅。
- 客户端-服务器是经典且常高度集中的组织：客户端发请求、服务器返结果。
- 去中心化架构中进程角色平等，即 P2P；进程组织成覆盖网络——结构化者可用确定性的路由方案，非结构化者需搜索算法。
- 混合架构组合集中与去中心化：云计算（逻辑上是客户端-服务器、服务器实则高度分布式）、边缘计算（终端与云之间的若干层，编排要求高）、以及越来越流行的区块链。

实践清单

- 读原书第 2 章，重点看图 2.1（分层三形态）、图 2.10（事件总线）、图 2.17（两层切分）、图 2.19（NFS 两模型）、图 2.27（云四层）、图 2.30（区块链三组织）。
- 拿一个你熟悉的系统（如电商下单），画出它的组件与连接器，判断属于哪种架构风格。
- 对比 REST 与“服务专用接口”：何时 REST 的简单性会成为限制？（提示：需要分布式事务时。）
- 用“时间耦合/引用耦合”给 Slack、消息队列、Redis 各定位一个坐标。
- 判断你所在场景是否真需要边缘：延迟、可靠性、法规三条理由分别成立吗？

Processes

本章聚焦“进程”这一分布式系统的基本构件。先讲线程 (*threads*)：为什么比进程更细的粒度能让分布式应用更好写、更快；再讲虚拟化 (*virtualization*)：虚拟机与容器如何隔离并复用资源；接着分别剖析客户端 (*clients*) 与服务器 (*servers*) 的解剖结构，并专门考察 *Apache* 与服务器集群；最后讨论代码迁移 (*code migration*)：为什么要迁移、有哪些模型、以及在异构系统里如何落地。

3.1 Threads

线程

进程是分布式系统的构件，但它的粒度太粗：为每个并发流开一个进程，代价高且难组织。于是操作系统提供更细的粒度——每进程多个控制线程 (**threads of control**)。本节的核心问题是：进程与线程是什么关系，线程为什么既是性能手段也是结构化手段。

3.1.1 线程介绍 (Introduction to threads)

操作系统为每个要执行的程序创建虚拟处理器，并用进程表记录其 进程上下文 (**process context**)：CPU 寄存器值、内存映射、打开的文件、记账信息、权限等。进程 (**process**) 常定义为“执行中的程序”。操作系统要保证独立进程之间不能互相破坏正确性，这种并发透明通常需要硬件支持，代价是：创建进程要建立完整独立地址空间，切换进程要改 Memory Management Unit (MMU) 寄存器、令 Translation Lookaside Buffer (TLB) 失效，必要时还要在主存与磁盘间交换进程。

线程与进程一样独立执行各自的代码，但不再强求高并发透明：线程系统只维护让 CPU 被多线程共享所需的最少信息，线程上下文通常只含处理器上下文加上少量线程管理信息（如“某线程正阻塞在 mutex 上”）。因此，同一进程内线程间的不当访问，防护责任完全落在应用开发者身上。于是三者层层包含：处理器上下文 $\subset$ 线程上下文 $\subset$ 进程上下文。这带来两个推论：多线程应用性能几乎不会比单线程差，往往还更好；但多线程开发需要额外的智力投入，因为线程之间没有进程那样的自动保护。

Note 3.1：上下文切换的代价

测性能从来不容易。时钟中断处理程序每 T 毫秒被激活一次（常见 T 为 0.5–20 ms，即 2000 Hz 到 50 Hz）。开销分两部分：直接开销是真正切换上下文的耗时（Intel 处理器上约 0.5–1 μ s）加处理程序自身工作耗时（约 0.5–7 μ s，强依赖实现）；间接开销则主要来自缓存扰动。对简单的时钟中断，测得间接开销约 80%：一次中断就可能让块 D 被逐出缓存，回访 D 又把 C 逐出……引起较持久的缓存重组，从而拖慢应用整体性能。

线程有哪些好处？首要一条：单线程进程中一旦执行阻塞系统调用，整个进程都被阻塞。以电子表格为例——它要维护单元格间（常跨表）的函数依赖，改一个值可能触发一连串计算；若只有一个控制流，等待输入时就不能算，计算依赖时又难接收输入。自然解法是至少两个线程：一个管交互、一个管更新，再加第三个后台备份。第二个好处是可在多处理器/多核上利用 并行：每个线程分配到不同 CPU 或核，共享数据放共享主存，设计得当则并行对单处理器系统也是透明的（只是更慢）。第三，大

应用常拆成协作程序、用操作系统提供的 进程间通信 (IPC) (命名管道、消息队列、共享内存段) 协作；IPC 的致命缺点是通信需要大量上下文切换 (图 3.1 的 S1 用户态 → 内核态、S2 进程上下文切换、S3 内核态 → 用户态)，而线程间改用共享数据通信，切换有时可完全在用户态完成，性能提升显著。最后还有一个纯粹的软件工程理由：很多应用用一组协作线程来组织就是更自然。

线程实现有两条路线。用户级线程库：创建/销毁便宜 (主要是分配与释放栈内存)，线程上下文切换往往只需存取几个 CPU 寄存器，无需改内存映射、清 TLB、做 CPU 记账。但用户级线程采用 **many-to-one** 模型——多个线程映射到单一可调度实体，一旦调用阻塞系统调用，整个进程连同其所有线程都被阻塞，对“阻塞 I/O 不该阻止其他部分执行”的场景几乎没用。内核级线程采用 **one-to-one** 模型，每个线程都是可调度实体，但每次线程操作都要系统调用，切换开销可能和进程一样大；不过既然上下文切换开销主要由缓存使用决定、而非 many-to-one 与 one-to-one 之分，许多操作系统出于简单就选了 one-to-one。

Note 3.2: many-to-many 线程模型

折中方案是把用户级与内核级线程混合，即 **many-to-many**：内核线程在单进程上下文中运行、可有多条；运行库同时提供用户级线程包 (创建/销毁/同步)，且完全在用户态实现，操作线程无需内核介入。多个内核线程共享同一线程包，各自可运行自己的用户线程：内核线程通过系统调用获得自己的栈后，执行调度例程去找可运行的用户线程；线程表由多个内核线程共享，用用户态 mutex 保护。用户线程阻塞或切换时，内核线程无需知情——切换纯在用户态完成，对内核线程看起来就是普通程序代码。好处：创建/销毁/同步便宜、阻塞调用不冻结整个进程、应用只见到用户线程、内核线程可轻易用于多处理器。Go 语言与 libfibre 是这一模型的实现，Arachne 稍作变体 (每核一个内核线程，但不支持阻塞 I/O)。

原文精读

A process is often defined as a program in execution, that is, a program that is currently being executed on one of the operating system's virtual processors.

Like a process, a thread executes its own piece of code, independently of other threads.

The most important benefit comes from the fact that in a single-threaded process, whenever a blocking system call is executed, the process as a whole is blocked.

A major drawback of user-level threads comes from deploying the many-to-one threading model: multiple threads are mapped to a single schedulable entity.

进程常被定义为“执行中的程序”，即当前正在操作系统的某个虚拟处理器上执行的程序。

与进程一样，线程独立于其他线程、执行自己的一段代码。

最重要的好处来自这一点：在单线程进程中，只要执行一次阻塞系统调用，整个进程就会被阻塞。

用户级线程的一大缺点源于采用 many-to-one 线程模型：多个线程被映射到单一可调度实体上。

原文定位：§3.1.1，原书 p.113–122 (图 3.1 上下文切换、图 3.2 缓存扰动、图 3.3–3.4 Python 进程/线程示例、图 3.5 内核级与用户级线程的组合)

3.1.2 分布式系统中的线程 (Threads in distributed systems)

线程让“阻塞系统调用不阻塞整个进程”成为可能 (前提是非 many-to-one 模型)，因此在分布式系统中尤其有吸引力：同时维护多条逻辑连接变得容易。分客户端与服务器两侧看。

多线程客户端：广域网中往返延迟常达数百毫秒甚至数秒，为做到高分布透明，需要“把通信延迟藏起来”。常用办法是发起通信后立刻去做别的事。典型例子是 Web 浏览器：一个 Web 文档常由 HTML 加一堆图片/图标组成，取每个元素都要建立 TCP/IP 连接、读数据、交给显示组件，而建连接和读数据都是阻塞操作。浏览器通常先取 HTML 并开始显示，一边让用户滚动一边继续取图片，图片到了就显示——用户不必等整页取完。把浏览器写成多线程客户端能显著简化实现：主 HTML 到手后，为其余部分各起线程，每个线程建立自己的连接、用标准阻塞调用拉数据，代码彼此相同且简单。更进一步，若 Web 服务器被复制到多台机器（同一站点、同一名字，请求按 round-robin 或其他负载均衡策略转发），多线程客户端可对不同副本建立连接、并行传输，使整页显示时间远短于单副本；这要求客户端能处理真正并行的数据流，而线程正适合此事。

Note 3.3: 客户端线程与性能

Blake 等 (2010) 用线程级并行度 (**thread-level parallelism, TLP**) 衡量多核利用：设 c_i 为恰好有 i 个线程同时执行的时间比例，则 $TLP = \frac{\sum_{i=1}^N i \cdot c_i}{1 - c_0}$ ， N 为可同时执行的最大线程数。当时典型 Web 浏览器 TLP 仅 1.5–2.5，即客户机具备 2–3 个核/处理器就够用。而现代浏览器会创建数百个线程、同时有数十个活跃（活跃未必在运行，可能阻塞等 I/O）。可见多线程更多用于组织应用结构，而非直接换来硬件层面的巨大加速；要真正利用并行，还需改造算法 (Meyerovich & Bodik 2010)。

多线程服务器：这才是分布式中多线程的主要用武之地——它既简化服务器代码，又便于利用并行（现代多核下并行更是显而易见）。以文件服务器为例：流行组织是 **dispatcher/worker** (图 3.6)，一个 **dispatcher** 线程读入客户端发到知名端点的请求，检查后交给一个空闲（阻塞中）的 **worker** 线程；worker 对本地文件系统做阻塞读，被挂起时另一个线程被调度（dispatcher 再取活，或其他就绪 worker 运行）。对比单线程服务器：主循环取一个请求、处理完再取下一个，等磁盘时服务器空闲、无法处理其他客户端请求，若跑在专用机器上 CPU 也白白闲置，单位时间处理的请求数少得多。第三种设计是单线程有限状态机：不用阻塞磁盘操作，而是发起异步（非阻塞）磁盘操作并稍后由操作系统中断唤醒，期间记录请求状态、继续看有没有别的请求；代价是失去了“顺序进程”模型，每次阻塞操作都要精确记录自己处理到哪、存下额外状态，等于用困难的方式模拟多线程及其栈。三种方式可总结为 (图 3.7)：多线程 = 有并行 + 阻塞调用；单线程进程 = 无并行 + 阻塞调用；有限状态机 = 有并行 + 非阻塞调用。线程的价值由此清晰：既保住“顺序进程 + 阻塞系统调用”的易写性，又获得并行；阻塞调用写起来就像普通过程调用。同样，也可以用多个进程来组织服务器（多进程服务器），操作系统能提供更强的共享数据保护，但进程间通信量大时性能会比线程差。

原文精读

An important property of threads is that they can provide a convenient means of allowing blocking system calls without blocking the entire process in which the thread is running

线程的一个重要特性是：它们能方便地允许阻塞系统调用，而不阻塞该线程所在的那个进程。

The usual way to hide communication latencies is to initiate communication and immediately proceed with something else.

隐藏通信延迟的常用办法是：发起通信后立即去做别的事情。

They make it possible to retain the idea of sequential processes that make blocking system calls and still achieve parallelism.

线程使我们可以保留“顺序进程 + 阻塞系统调用”的观念，同时仍然获得并行。

原文定位：§3.1.2，原书 p.122–126（图 3.6 dispatcher/worker 多线程服务器、图 3.7 构造服务器的三种方式）

3.2 Virtualization

虚拟化

若说线程/进程是“同时做更多事”，其本质是在单 CPU 上快速切换制造并行假象。这种“一个资源装成多个”的思路可推广到其他资源，即资源虚拟化（**resource virtualization**）。它已用了几十年，近年随（分布式）系统愈发普遍与复杂而重新受到重视——因为应用程序的寿命往往长于其所依赖的系统软件与硬件。

3.2.1 虚拟化原理（Principle of virtualization）

每个（分布式）计算机系统都向上层软件提供编程接口（从 CPU 的指令集到中间件的大量 API）。虚拟化的本质就是扩展或替换一个已有接口，以模仿另一个系统的行为（图 3.8）。它最早（1970 年代）的动机是让遗留软件（含其操作系统）跑在昂贵的大型机上，如 IBM 370 提供的虚拟机。硬件变便宜、OS 种类减少后虚拟化一度降温，但 1990 年代末又变了：高层次软件（中间件、应用）比底层软硬件稳定得多，虚拟化可把遗留接口移植到新平台；网络无处不在后，把每个应用连同其库与操作系统放进各自虚拟机、跑在共同平台上，能大幅降低平台与机器的异构性。可移植性（**portability**）因此成为虚拟化在分布式系统中如此关键的最重要原因之一。另一个重要动机是隔离代码（尤见于云计算），但它也带来新的安全威胁。

计算机系统一般在三个层次提供四类接口：硬件与软件之间的指令集架构（**instruction set architecture, ISA**）（分特权指令与一般指令）、操作系统提供的系统调用、以及通常构成 API 的库调用（图 3.9）。虚拟化即模仿这些接口，有两种方式：其一，构建运行环境提供抽象的指令集来执行应用——可解释（如 Java 运行环境）也可模拟（如在 Unix 上跑 Windows 应用，此时还得模仿系统调用，远非易事），得到进程虚拟机（**process virtual machine**）（图 3.10a），只针对单个进程；其二，做一层屏蔽原始硬件、却对外提供同一（或他者）完整指令集的系统，即原生虚拟机监视器（**native virtual machine monitor, VMM**）（图 3.10b），它需实现设备驱动。更省事的做法是宿主式 VMM（**hosted VMM**）（图 3.10c），运行在可信的宿主操作系统之上、复用其设施（但通常需特殊权限而非普通用户级应用），这在数据中心与云中极为流行。VMM 可同时向不同程序提供接口，于是多个不同的 guest OS 能独立并发跑在同一平台上。Rosenblum & Garfinkel（2005）强调：虚拟机让完整应用及其环境隔离，一次错误或安全攻击不再影响整台机器；同时进一步解耦硬件与软件，提升可移植性，便于整体搬迁（见 §3.5）。

Note 3.5: 虚拟机为何不慢

VMM、guest OS 与应用的大部分代码都原生跑在底层硬件上，一般（非特权）指令由底层机器直接执行。Popek 与 Goldberg（1974）给出了高效执行虚拟机的形式化条件，核心结果：“对任何常规计算机，只要该机器的敏感指令集合是特权指令集合的子集，就可以构造出虚拟机监视器。”只要敏感指令在用户态执行时能被捕获，其余指令即可原生执行。麻烦在于并非所有指令集都满足该条件，最流行的 Intel x86 就有 17 条非特权的敏感指令。两个解法：全虚拟化（如 VMWare：扫描可执行文件，在非特权敏感指令周围插入代码，把控制转到 VMM 处模拟，guest OS 与应用都无需改动）；或半虚拟化（**paravirtualization**）（要求修改 guest OS，如 Xen），让这些指令的副作用被消解。

原文精读

In its essence, virtualization deals with extending or replacing an existing interface to mimic the behavior

本质上，虚拟化所做的是扩展或替换一个已有接口，以模仿另一个系统的行为。

of another system

portability is perhaps the most important reason why virtualization plays such a key role in many distributed systems.

可移植性，或许正是虚拟化在许多分布式系统中扮演如此关键角色的最重要原因。

原文定位：§3.2.1，原书 p.127–133（图 3.8 接口与虚拟化、图 3.9 四类接口、图 3.10 进程 VM / 原生 VMM / 宿主式 VMM、图 3.11 分层执行）

3.2.2 容器 (Containers)

虚拟机要求应用依赖整套指令集与操作系统，往往需要不小代价来保证可移植性与性能。但现实中应用通常只依赖特定的库与支撑软件：我们真正想要的是让不同应用并行运行、各自使用自己的环境、互不察觉。这就是容器 (**container**)：可看作一组二进制 (镜像)，共同构成运行应用的软件环境——类似登录 Unix 系统时看到的那套标准目录 (可执行程序、库、文档等)。朴素实现是把整套环境复制成根文件系统下的子目录，用 **chroot** 把用户改道到该子目录，应用只看到自己需要的库与依赖，其它容器各有各的视图：在此意义上，容器虚拟化了的软件环境。但朴素实现不够：不同容器中的应用与进程需要互相隔离，整份复制效率低 (很多库其实相同)，宿主还需对自身资源有控制权。Linux 用三种机制解决：**namespaces** (给容器内一组进程独立的标识符视图，如 PID namespace 让每个容器都有自己的 init 进程，凭 `unshare --pid --fork --mount-proc bash` 即可观察)、**union file system** (把多个文件系统按层组合，只有最高层可写；通过多次 mount 只读叠加，可让 PHP 应用改用旧版本而无需整份复制)、以及 **control groups (cgroups)** (限制一组进程可用的主存、CPU 优先级等，防止单个容器耗尽资源、妨碍其他容器)。总之，容器是“一份文件归档 + 特定的共享只读层栈”，容器内进程通过 namespaces 以为自己的上下文是唯一的，通过 cgroups 获得受限的资源能力 (图 3.12)。

Note 3.6: PlanetLab

PlanetLab 是云计算流行之前的协作式广域集群：各组织捐出计算机，合计数百节点，构成 1 层服务器集群，每节点的访问、处理、存储都在本地完成，管理几乎完全分布式 (项目 2020 年关闭)。每个节点有两个关键组件：**VMM** (增强型 Linux，支持容器) 与 (Linux) **Vserver** (本质是运行中的容器)。Vserver 之间严格隔离：不同 Vserver 中进程可有相同 user ID 却不意味着同一用户，这极大方便了不同组织用 PlanetLab 做试验床。PlanetLab 用 **slice** (分布在不同节点上的一组 Vserver，可看作经广域网连接的虚拟服务器集群) 支持实验；**node manager** 只负责在本节点创建 Vserver 与控制资源分配，**slice creation service (SCS)** 向其申请，而 slice authority 才有资格请求创建 slice。资源由 **rspec** 描述 (含时间区间与各类资源)，以全局唯一 128 位 **rcap** 标识。容器式虚拟化的一大好处是资源分配简单 (可超订与动态分配)，这在主存只有几 GB、需同时跑数十个容器的节点上至关重要 (内存 hogging 的 Vserver 会被 reset)。类似系统如今以 EdgeNet 形式运行，容器技术已换成 Docker + Kubernetes。

原文精读

A container can be thought of a collection of binaries (also called images) that jointly constitute the software environment for running applications.

容器可以看作一组二进制文件 (也称镜像)，它们共同构成运行应用所需的软件环境。

In this sense, a container effectively virtualizes the software environment for an application.

在这个意义上，容器实际上为应用虚拟化了其软件环境。

原文定位：§3.2.2，原书 p.133–138（图 3.12 容器在宿主环境中的组织、图 3.13 PlanetLab 节点、图 3.14 PlanetLab slice）

3.2.3 虚拟机与容器之比较 (Comparing virtual machines and containers)

自 Docker 让容器流行以来,“虚拟机还是容器更好”的争论不断,且常因术语误用(“轻量容器 vs 重量虚拟机”暗示重即慢)而失焦。本节只看一个具体而重要的方面:性能;可移植性留到代码迁移处再谈。衡量性能需多维度:CPU、内存、各类 I/O (磁盘、网络),还要问用什么负载/基准来比较。Sharma 等 (2016) 系统比较了 Linux 容器 (LXC) 与 Linux 虚拟机 (KVM): 基线情形下容器略优,但差距不大,唯 I/O 例外——虚拟机明显不如容器,这不奇怪,尤其传统 I/O 中操作系统发挥关键作用(要执行大量特权指令)。van Rijn 与 Rellermeier (2021) 进一步表明,两者的差异常近乎可忽略,连磁盘与网络 I/O 也是;原因之一是宿主 OS 把结果缓存在主存里,很多后续 I/O 其实作用于内存数据(这既让基准测试更难,也更贴近真实应用行为)。运行 mysql 等应用级基准时差异可能存在但可以很小。总体结论仍是:虚拟机相对容器确实引入更多开销。再看多应用并排的场景,问题变成“一个应用对另一个应用的性能影响有多大”,即调度资源的能力:总体趋势是容器化更难隔离独立应用,而虚拟机在 CPU 调度与磁盘性能上处理得更好。近年改进很多,虚拟化技术已没有理由显著慢于直接在宿主 OS 上运行应用。

原文精读

Nevertheless, depending on the actual I/O behavior, the overall conclusion is that virtual machines do impose more overhead in comparison to containers.

the general trend is that containerization has more difficulty isolating independent applications and that scheduling for CPU usage as well as for disk performance is handled better through virtual machines.

不过,取决于实际 I/O 行为,总体结论是:虚拟机相对于容器确实施加了更多开销。

总体趋势是:容器化更难隔离相互独立的应用,而在 CPU 使用与磁盘性能的调度上,虚拟机处理得更好。

原文定位: §3.2.3, 原书 p.138-139

3.2.4 虚拟机在分布式系统中的应用 (Application of virtual machines to distributed systems)

从分布式系统看,虚拟化最重要的应用是云计算。云厂商大致提供三类服务: **IaaS** (基础设施)、**PaaS** (系统级服务)、**SaaS** (实际应用)。虚拟化在 **IaaS** 中起关键作用: 厂商出租的不是物理机,而是虚拟机(监视器),可能与其他客户共享物理机;巧妙之处在于客户之间几乎完全隔离,会以为自己租到了一台专用物理机。但隔离绝非彻底——物理资源毕竟共享,会带来可观察的性能下降。以 Amazon EC2 为例: 可创建由多台联网虚拟服务器组成的环境,构成分布式系统的基础;提供大量预配置 **Amazon Machine Image (AMI)** (操作系统内核加若干服务,如 LAMP 镜像 = Linux 内核 + Apache + MySQL + PHP),本质类似启动盘(但有若干重要差异)。客户选好/改好 AMI 后启动,得到 **EC2 instance**: 托管应用的实际虚拟机。客户几乎永远不知道实例跑在哪台物理机上,只能选 Amazon 给的少数区域 (US、南美、欧洲、亚洲)。每个实例有两个 IP: 私有地址供实例间内部通信,公有地址让任意 Internet 客户端可达,二者用标准 **NAT** 映射;用 SSH 管理实例,密钥由 Amazon 生成。EC2 环境可配置 CPU (核数与类型,含 GPU)、内存、存储、平台 (32/64 位)、网络带宽等。实例自带本地存储是瞬态的: 实例停止即丢失;要防丢失需显式存到 **S3**,或挂载映射到 **EBS** 的存储设备(像挂一块额外硬盘,数据持久,且可重新挂到其他实例)。可见 EC2 之类的 IaaS 让客户能配置由联网虚拟服务器构成的完整分布式系统、运行自备的分布式应用,且无需维护任何物理机——完全可以认为虚拟化是现代云计算的核心。

原文精读

Virtualization plays a key role in IaaS.

虚拟化在 IaaS 中扮演关键角色。

One can indeed argue that virtualization lies at the core of modern cloud computing.

确实可以说，虚拟化位于现代云计算的核心。

原文定位：§3.2.4，原书 p.139–140

3.3 Clients

客户端

前面讨论过客户端-服务器模型与二者交互。本节先解剖客户端，下一节解剖服务器。

3.3.1 网络化用户界面 (Networked user interfaces)

客户端机器的首要任务是让用户与远程服务器交互。方式大致两种。其一：为每个远程服务在客户端配一个能通过网络联系该服务的对等件，由应用层协议处理同步（图 3.15a），如智能手机上要与远程共享日历同步的日历应用。其二：只提供便利的用户界面、直接访问远程服务，客户端机器仅作终端、无需本地存储，得到应用无关的方案（图 3.15b）——即网络化用户界面：一切都在服务器处理与存储。随着 Internet 连接与移动设备普及，这种瘦客户端（**thin-client**）方式备受关注，也因便于系统管理而流行。

最古老且仍在广泛使用的网络化用户界面之一是 **X Window System**（简称 X），用于控制位图终端（显示器、键盘、指点设备），也支持平板/手机的触摸屏，可视为操作系统中控制终端的那部分。其核心是 **X kernel**，含全部终端相关设备驱动，因而高度依赖硬件；它提供控制屏幕、采集键盘鼠标事件等较低层接口，经名为 **Xlib** 的库开放给应用（图 3.16，Xlib 很少被直接使用，应用通常用其上的 toolkit）。X 的有趣之处在于 X kernel 与 X 应用不必同机：X 提供 **X protocol**，让 Xlib 实例与 X kernel 交换数据与事件（可请求创建/杀死窗口、设颜色、定义光标等；X kernel 则把本地事件以事件包回送 Xlib）。多个应用可同时与 X kernel 通信，其中 **window manager** 有特殊权限，决定显示的“观感”（窗口如何加按钮装饰、如何摆放等），其他应用须遵守，故应用与 X 终端间的大量交互被经 window manager 重定向。有趣的是，X kernel 接收（可能远程的）应用操纵显示的请求，因此它在语义上充当服务器、应用是客户端——术语正确但易混淆。

瘦客户端网络计算：X 应用原则上应把应用逻辑与用户界面命令分离，但正如 Lai 与 Nieh（2002）报告，实践中二者常紧密耦合，导致应用发出大量请求、且要等响应才能继续，在长延迟广域网上会严重影响性能。解决办法之一是重做 X protocol 实现，如 **NX**（Pinzari 2003）：把消息分为作标识符的固定部分与可变部分，同标识符的消息常含相似数据，于是只发相同标识符消息间的差异，收发双方维护标识符即可在接收端解码，报告带宽下降最多达 1000 倍，使 X 能在 9600 kbps 的低带宽链路上运行。另一方向是让应用完全控制远程显示直至像素级（控制远程桌面），把位图变化发到显示端写入本地帧缓冲：著名例子是 **VNC**（Richardson 等 1998）。显然这需要精妙的编码技术，否则带宽成问题——例如 320×240、每像素 24 位、30 帧/秒的视频，无高效编码约需 53 Mbps；实践中用多种编码，最佳选择通常依应用而定。

原文精读

A major task of client machines is to provide the means for users to interact with remote servers.

客户端机器的一项主要任务，是提供让用户与远程服务器交互的手段。

In the case of networked user interfaces, everything is processed and stored at the server.

在网络化用户界面的情形下，一切都在服务器端处理与存储。

原文定位：§3.3.1，原书 p.141–144（图 3.15 两种交互方式、图 3.16 X Window System 组织）

3.3.2 虚拟桌面环境 (Virtual desktop environment)

随云计算成熟、云应用增多，把云真正变成终端用户的虚拟桌面环境变得合宜，只差客户端软件。最早的提供者之一是 Google 的 **Chrome OS**：让浏览器充当本地桌面界面，旁边可运行多个独立应用，各自原则上最终与云端对等件协作——这与现代智能手机颇为相似。随应用日益转为浏览器扩展，浏览器正在接替操作系统用户界面的角色。

Chrome 浏览器解剖：Chrome 有超 2500 万行代码（与 Linux 内核相当），其工作概览见图 3.17。核心是 **resource loader**，负责从 Web 服务器取内容：一般先取 HTML，再并行为页面引用的其他素材建立连接，这通常由独立线程完成；它需部分解析 HTML 才能发现还要取什么，而大部分解析由另一组件完成，构造 **Document Object Model (DOM)**——HTML 的树表示。样式 (**styling**) 信息来自单独文档（如每个段落的颜色与字体），解析后并入 DOM，得到 **render tree**，其中含下一步所需的全部信息：确定各元素将显示在何处（计算几何区域、断行位置，还要考虑语言特有的左右/右左书写，如希伯来语、阿拉伯语；断行需精确计算、考虑字体特性）——遇浮动图片或多栏时布局会相当复杂。布局确定后，**painting** 组件据布局指令构造一组绘制操作（如 `drawRectangle(x,y,height,width)`），随后由 **rasterization** 填充每个像素的细节（尤其颜色），并据 painting 提供的信息正确显示嵌入图片。图 3.17 未明示的是 DOM 实际被分解为若干层、每层分别光栅化，最后合成 (**compositing**) 成可显示的最终图像，合成器还可与正在滚动的用户交互。最后，每个浏览器都有处理脚本的独立组件：客户端常用 JavaScript，也日益出现 **WebAssembly**（结合真正编译时，性能可接近原生代码），生成代码始终受浏览器权限约束，因而（相对）安全——能在浏览器里跑代码正是“让用户以为自己在本地桌面上运行应用”的关键。显然整个渲染高度复杂，其中很多环节（如逐层光栅化）可并行、甚至可交给 GPU，现代浏览器正是如此：生成线程与进程，既为组织这庞杂软件，也为让渲染整体高效。事实上，浏览器还用进程互相保护代码：Chrome 把整个渲染放在独立进程（HTML 解析、样式、布局、painting、合成合在一起），光栅化单独拆出以利用可能存在的 GPU，它与主进程通过消息传递通信；每个浏览器标签有自己的 **renderer** 进程，且每个 renderer 跑在独立的 **sandbox** 中——一种阻止其直接与底层操作系统通信的保护机制（讨论移动代码时会再谈）。

浏览器与应用：有浏览器往往足以提供虚拟桌面环境，关键是浏览器能执行脚本作为远程应用在客户端的那部分，从而大体营造“在本地机器上工作”的错觉——这正是 Chrome OS 提供的。但当本地资源确实必要时（首先是相机、麦克风、扬声器等媒体资源），事情会复杂些，许多 Web 应用会请求用户授权。也可以在本机原生运行应用，现代智能手机就是如此：原生应用（也叫 mobile app）像其他本地应用一样运行，其主要优势是原则上可离线工作，但看看有多少移动 app 在无网络时仍有用，就知离线并非默认假设。一个逐渐兴起的做法是 **Progressive Web App (PWA)**：以浏览器为宿主环境、却表现得像普通移动 app，本质是把不依赖（高质量）网络的服务器端内容移到客户端并缓存，从而跑得更快、可与原生 app 相当，且因需与服务器通信的量大减，网络质量差时也能很好工作。总体看，虚拟桌面环境正从极瘦客户端走向承载更多功能、且把 Internet 通信高度优化的方向；相比主要本地安装应用的胖客户端，其最大优势是管理更容易：服务器可按需向客户端上传新部件。

原文精读

With an increasing trend to transform applications to browser extensions, we can see that the browser is

随着应用日益被改造为浏览器扩展，我们可以看到浏览器正在接替操作系统用户界面的角色。

taking over the role of an operating system's user interface.

Moreover, every browser tab gets its own renderer process. To improve security, each renderer is running in a separate sandbox: a protection mechanism that precludes direct communication with the underlying operating system.

此外，每个浏览器标签都有自己的 renderer 进程。为提升安全性，每个 renderer 都运行在独立的 sandbox 中——一种阻止其与底层操作系统直接通信的保护机制。

原文定位：§3.3.2，原书 p.144–148（图 3.17 Chrome 渲染页面概览）

3.3.3 面向分布透明的客户端软件（Client-side software for distribution transparency）

客户端软件不止用户界面：客户端-服务器应用的处理层与数据层也常有部分在客户端执行。特殊一类是嵌入式客户端软件（ATM、收银机、条码阅读器、电视机顶盒等），此时用户界面只占很小一部分，本地处理与通信设施才是大头。除界面与应用相关软件外，客户端软件还含实现 分布透明的组件。理想情况下，客户端不应察觉自己在与远程进程通信；而服务器出于性能与正确性考虑，分布往往对其不那么透明。

- 访问透明（**access transparency**）：通常由接口定义生成 **client stub** 来处理。stub 提供与服务器相同的接口，但隐藏机器架构差异与实际通信，把本地调用转成发给服务器的消息，并把服务器消息转回调用者期望的返回值。
- 位置/迁移/重定位透明：关键是方便的命名系统，客户端软件与命名系统协作很重要。例如客户端已绑定某服务器时，可被直接告知服务器换了位置，中间件即可对用户隐藏服务器当前网络位置、必要时透明地重新绑定，最坏也只是客户端应用注意到临时性能下降。
- 复制透明（**replication transparency**）：许多系统用客户端方案实现。若有复制服务器，可把请求转发给每个副本（图 3.18），由客户端软件透明地收集全部响应、向应用只交一个响应。
- 失败透明（**failure transparency**）：掩盖与服务器的通信失败通常由客户端中间件完成，例如反复尝试连接、或数次失败后改试另一服务器；甚至有如 Web 浏览器连接失败时返回上次会话缓存数据的情形。
- 并发透明：可由特殊的中介服务器（尤其事务监视器）处理，对客户端软件的依赖较少。

原文精读

Ideally, a client should not be aware that it is communicating with remote processes.

理想情况下，客户端不应察觉到自己在与远程进程通信。

Access transparency is generally handled through the generation of a client stub from an interface definition of what the server has to offer.

访问透明通常通过从“服务器所提供的接口定义”生成 client stub 来处理。

原文定位：§3.3.3，原书 p.147–148（图 3.18 用客户端方案实现服务器复制的透明）

3.4 Servers

服务器

服务器是代表一组客户端实现特定服务的进程。本质上，每个服务器的组织方式相同：等待客户端到来的请求，确保请求被处理，然后等待下一个请求。

3.4.1 通用设计问题 (General design issues)

迭代式 vs 并发式：迭代服务器 (**iterative server**) 自己处理请求、必要时返回响应；并发服务器 (**concurrent server**) 不自己处理，而是交给独立线程或另一进程，随即去等下一个请求（多线程服务器即一例；也可为每个请求 **fork** 新进程，许多 Unix 系统如此）。处理请求的线程/进程负责向客户端返回响应。

客户端在哪里联系服务器：端点。客户端总把请求发到服务器所在机器的端点 (**end point**，也叫端口)，每个服务器监听一个特定端点。客户端如何知道端点？其一，为知名服务全局分配端点（如 FTP 用 TCP 21、HTTP 用 TCP 80），由 IANA 分配并记录，此时客户端只需找到服务器的网络地址，可用命名服务。其二，很多服务不需预分配端点（如时间服务由本地 OS 动态分配），客户端须先查端点：一种做法是在每台运行服务器的机器上放一个 **daemon**，它掌握同机各服务的当前端点、自身监听一个知名端点，客户端先问 **daemon** 再联系具体服务器（图 3.19a）。把一个端点关联到一个具体服务很常见，但为每个服务各实现一个服务器可能浪费资源——Unix 上常有很多服务器在被动等待。更高效的是让单个 **superserver** 监听每个服务对应的端点（图 3.19b）：如 Unix 的 **inetd** 监听一批知名端口，请求到达时 **fork** 一个进程处理，处理完即退出。

中断服务器：用户刚上传大文件却突然发现传错了，想中断以取消后续传输。做法之一（在今天的 Internet 上“太灵了”，有时是唯一选择）是粗暴退出客户端应用（自动断开连接）、立即重启、装作什么都没发生，服务器最终会拆掉旧连接、以为客户端崩了。更好的做法是让客户端与服务器支持带外数据 (**out-of-band data**)——需先于该客户端其他数据被处理。一种解法是让服务器额外监听一个控制端点接收带外数据，同时以较低优先级监听正常数据端点；另一种是在同一连接上发送带外数据，如 TCP 可传紧急数据 (**urgent data**)，服务器收到时被中断（如 Unix 中的信号），进而检视并相应处理。

无状态 vs 有状态服务器。无状态服务器 (**stateless server**) 不保存客户端状态信息，且可改变自身状态而无需通知任何客户端。Web 服务器即无状态：只响应到来的 HTTP 请求（上传或（多数时候）取文件），处理完就彻底忘记客户端；它管理的文件集合也可变更而无需告知客户端。注意：很多无状态设计其实也维护客户端信息，但关键在于这些信息若丢失不会导致服务中断（如 Web 服务器记录全部请求日志，丢了顶多性能次优）。一种特殊形式是 **soft state**：服务器承诺代表客户端维护状态，但只限于有限时间，过期即回落到默认行为、丢弃该状态（如承诺在一段时间内通知客户端更新，之后客户端须轮询），源于计算机网络协议设计。相反，有状态服务器 (**stateful server**) 一般维护客户端的持久信息，须由服务器显式删除：典型是允许客户端保留文件本地副本（甚至用于更新）的文件服务器，它维护 (client, file) 表，从而掌握谁对哪个文件有更新权限、可能还有该文件最新版本。这能改善客户端感知的读写性能，但缺点明显：服务器崩溃后必须恢复整张表、乃至崩溃前的整个状态，否则无法保证处理了最新更新；恢复带来相当复杂度（见第 8 章）。而无状态设计下，崩溃的服务器无需任何特殊措施，重启再等服务即可。Ling 等（2004）主张区分（临时的）**session state** 与 **permanent state**：前者关联单个用户的一系列操作、应维持一段时间但非无限，常见于三层架构（应用服务器需对数据库服务器做一连串查询才能响应客户端）；只要客户端能重发原请求，丢失 **session state** 就无实质危害，故可用更简单、可靠性较低的方式存储。**permanent state** 则是数据库中维护的信息（客户信息、已购软件密钥等）。但对多数分布式系统，维护 **session state** 已意味着有状态设计，需在失败时采取特殊措施、并对状态持久性作出显式假设。设计时，无状态/有状态的选择不应影响服务器提供的服务：如文件须先打开才能读写，无状态服务器就得设法模仿——常见做法是收到读写请求时先打开文件、执行操作、立即关闭。有时服务器想记录客户端行为以更好地响应（如 Web 服务器把客户端直接引到其常访页面），无状态时常解法是让客户端随请求附带此前访问信息；Web 里这由浏览器透明地存在 **cookie** 中（一小段客户端特定数据），**cookie** 从不被执行、只被存储并在下次访问时随请求发出：首次访问时服务器随所请求页面回送 **cookie**，浏览器收好，此后每次访问该服务器都带上它。

原文精读

A stateless server does not keep information on the state of its clients, and can change its own state without having to inform any client

无状态服务器不保存其客户端的状态信息，并且可以改变自身状态而无需通知任何客户端。

In contrast, a stateful server generally maintains persistent information on its clients.

相反，有状态服务器一般会维护关于其客户端的持久信息。

In the case of an iterative server, the server itself handles the request and, if necessary, returns a response to the requesting client.

就迭代服务器而言，服务器自己处理请求，并在必要时向发出请求的客户端返回响应。

原文定位：§3.4.1，原书 p.148-154（图 3.19 daemon 与 superserver 两种客户端-服务器绑定）

3.4.2 对象服务器 (Object servers)

对象服务器与更传统的服务器的重要差别在于：对象服务器本身不提供特定服务，具体服务由驻留在服务器中的对象实现；服务器只提供“按远程客户端请求调用本地对象”的手段，因此通过增删对象即可轻易改变服务。对象服务器是对象“居住”之处；对象由两部分组成：表示其状态的数据与执行其方法的代码，二者是否分离、方法实现是否被多对象共享取决于服务器；调用对象的方式也有差异（多线程服务器中可为每个对象分配独立线程，或为每个调用请求用一个线程）。要让对象被调用，服务器需知道执行哪段代码、操作哪些数据、是否另起线程等。简单做法假定所有对象都一样、调用方式唯一，但通常不灵活、束缚开发者；更好的做法是支持多种策略。例如 **transient object**：只在其服务器存在期间存在、甚至更短，如文件的内存只读副本、计算器；合理策略是首次调用请求时创建、一旦无客户端绑定即销毁——好处是只在真正需要时占用资源，缺点是首次调用需先创建以致耗时，故另一种策略是服务器初始化时创建全部 **transient object**，代价是即便无人用也占资源。类似地，可让每个对象独占自己的内存段（不共享代码与数据），当实现不分离代码数据、或出于安全需隔离时有必要，此时服务器需特殊措施或依赖 OS 保证段边界不被侵犯；反之可让对象至少共享代码：如同一 **class** 的对象放在一个数据库里，**class** 实现只需加载一次，请求到达时取该对象状态、执行所请求方法即可。线程策略亦多：最简单是单线程；或每个对象一个线程，请求交给负责该对象的线程，忙则暂时排队——好处是对象自动免于并发访问（调用都经该对象的单线程串行化），整洁简单；也可为每个调用请求单开线程，此时对象须已自行防并发。与“每对象一线程/每方法一线程”无关的选择，是线程按需创建还是维护线程池。通常没有唯一最优策略，取决于线程是否可得、性能有多重要等。

关于“如何调用对象”的决定通常称为 **activation policy**，强调对象常需先被带入服务器地址空间（即 **activated**）才能真正被调用。于是需要按策略对对象分组的机制，有时称为 **object adapter**（或 **object wrapper**），可理解为实现某种 **activation policy** 的软件。关键在于 **object adapter** 是通用组件、只需针对具体策略配置。一个 **adapter** 可管控一个或多个对象；由于服务器须能同时支持需不同 **activation policy** 的对象，同一服务器中可有多 **adapter**。调用请求到达服务器后，先被派发到合适的 **adapter**（图 3.20）。重要观察：**adapter** 对其所管控对象的具体接口一无所知，否则就无法通用；它只需能从调用请求中提取对象引用、进而按特定 **policy** 把请求派发到被引用对象。如图 3.20，**adapter** 不把请求直接交给对象，而是交给该对象的服务器端 **stub (skeleton)**；**skeleton** 通常由对象接口定义生成，负责解包请求并调用相应方法。**adapter** 可在运行时配置以支持不同策略：如 **CORBA** 系统中可指定对象在其 **adapter** 停止后是否继续存在，可配置 **adapter** 生成对象标识符或由应用提供，也可配置为单线程或多线程模式。需强调：对象的实现可能（间接）访问数据库或调用特殊库例程，这些细节对 **adapter** 隐藏——**adapter** 只与 **skeleton** 通信；因此实际实现可能与语言级（编译期）对象毫无关系。为此通常采用另一术语：**servant** 是“构成对象实现的一段代码”的通用称呼。

例：**Ice** 运行时系统。**Ice** 部分是为应对商业对象式分布式系统的繁复而生（Henning 2004）。**Ice** 中的

对象服务器不过是普通进程，启动时初始化 Ice 运行时；其基础是 **communicator**，管理若干基本资源，最重要的是一个线程池，还关联动态分配的内存、并提供配置手段（最大消息长度、最大调用重试次数等）。通常一个对象服务器只有一个 **communicator**；当不同应用需完全分离与相互保护时，可在同一进程内建多个 **communicator**（可能配置不同），至少可分离线程池，使一个应用耗尽线程不影响另一个。用 **communicator** 创建 **object adapter**（图 3.21）：示例中创建并初始化运行时得到 **communicator**，用其创建 **object adapter**（令其监听 TCP 端口 11000），再创建 **object1**、**object2** 并以不同名字加入 **adapter**；**adapter** 在单一端口上监听，却用对象名来调用正确的对象，随后 **activate** 才真正启动监听线程。客户端为每个服务器端对象建立对应 **printer** 对象，先启服务器再启客户端即可得到预期输出；若把 **printer2** 关联到 **base1**，**printer2** 就会被关联到 **object1** 而非 **object2**。策略可通过修改 **adapter** 属性改变，例如指定始终只有一个线程、从而串行化对该 **adapter** 中所有对象的访问。上面的例子把对象在应用创建后加入 **adapter**，意味着 **adapter** 可能需同时承载很多对象、存在可扩展性问题；替代方案是按需动态加载对象，Ice 用特殊的 **locator** 支持：可视为特殊类型的 **servant**，当 **adapter** 收到未显式加入的对象的请求时转发给它处理。比如 **locator** 知道某对象状态存在关系数据库中：对象标识符可能对应记录的主键，**locator** 查该键、取出状态即可继续处理请求。一个 **adapter** 可加多个 **locator**（由它跟踪哪些对象标识符属于哪个 **locator**），从而用单个 **adapter** 支持大量对象，代价是对象（其实是其状态）需在运行时加载，但这种动态行为可能让服务器本身相对简单。

原文精读

The important difference between a general object server and other (more traditional) servers is that an object server by itself does not provide a specific service.

通用对象服务器与其他（更传统的）服务器的重要差别在于：对象服务器本身并不提供某项特定服务。

A servant is the general term for a piece of code that forms the implementation of an object.

servant 是“构成对象实现的一段代码”的通用称呼。

原文定位：§3.4.2，原书 p.154–159（图 3.20 支持不同 **activation policy** 的对象服务器、图 3.21 Ice 对象服务器与客户端）

3.4.3 例：Apache Web 服务器 (Example: The Apache Web server)

Apache 是策略与机制分离的极好例子，也是极流行的服务器，估计托管约 25% 的网站。它复杂，且文档类型繁多，故须高度可配置、可扩展，同时尽可能独立于平台。平台无关靠提供自己的基础运行时环境实现：**Apache Portable Runtime (APR)** 是一套库，为文件处理、网络、加锁、线程等提供平台无关接口；扩展 Apache 时只要调用 APR 而不调用平台特定库，可移植性就大体有保障。从某种角度看，Apache 可视为“完全通用的服务器”、为对到来的请求产生响应而定制；当然它有许多隐含依赖与假设，使之主要适合处理 Web 文档请求。例如浏览器与服务器用 HTTP 通信，HTTP 几乎总建于 TCP 之上，故 Apache 核心假定所有请求遵循基于 TCP 的面向连接方式；UDP 请求不改核心就无法处理。

除此之外，Apache 核心对“请求该如何处理”假设很少，其组织（图 3.22）的基础是模块 (**module**)——由一或多个为正确处理请求而应被调用的函数组成。随之有三个问题：如何保证函数被调用？如何保证在正确时刻被调用？如何防止函数被调用于其本不该处理的请求？第三问最简单：每个函数都会随请求被调用，但若该请求非其所管，函数返回 **DECLINED** 即可。让函数被调用靠 **hook**——一种函数的占位符；每个函数对应一个 **hook**，每个模块向核心提供其函数的 **hook** 列表，通过静态或动态链接模块到核心，即建立 **hook** 与函数的关联。例如有把 URL 转成本地文件名的 **hook**（处理请求时几乎必然要用到），还有写日志、检查客户端身份、检查访问权限、检查请求相关 MIME 类型等 **hook**；如图 3.22，**hook** 按特定顺序处理，这正是 Apache 强制特定请求处理控制流之处。函数间常可独立处理（彼此完全隔离地作用于请求），但未必总是如此，故 Apache 区分若干阶段 (**phases**)：挂接函数时须指

定其应在整个请求处理流的开始、中间还是结尾被调用；若需细粒度控制，还可指定在某模块之前或之后调用。容易看出，开发彼此隔离（实为无状态）的函数有助于模块化设计。

原文精读

Fundamental to this organization are modules.

这一组织方式的基础是模块。

Getting a function to be called at all is handled through a hook, which is a placeholder for a function.

让一个函数能够被调用，是靠 hook 来处理的，hook 是函数的占位符。

原文定位：§3.4.3，原书 p.159–161（图 3.22 Apache Web 服务器的一般组织）

3.4.4 服务器集群（Server clusters）

服务器集群不过是通过网络连接、每台机器运行一或多个服务器的一组机器；本节先看局域网中的常见集群，再看广域集群。

通用组织：集群常逻辑上分三层（图 3.23）。第一层是（逻辑）交换机，客户端请求经其路由；交换机形态差异很大，如传输层交换机接受入站 TCP 连接请求并转给集群中某台服务器，完全不同的例子是接受 HTTP 请求的 Web 服务器，把部分请求转给应用服务器进一步处理、稍后收集结果再返回 HTTP 响应。如同任何多层客户端-服务器架构，集群也常含专做应用处理的服务器（常为高性能硬件，专供算力）；而企业中所需的算力未必是瓶颈、访问存储才是，故应用也可能只需跑在低端机器上。第三层是数据处理服务器，尤其文件与数据库服务器（可能用专为高速磁盘访问配置、带大容量服务端缓存的专用机器）。并非所有集群都严格分层：常见每台机器自带本地存储、把应用与数据处理合于一个服务器，得到两层架构；如流媒体集群常部署两层架构，每台机器充当专用媒体服务器。当集群提供多种服务时，不同机器可运行不同应用服务器，于是交换机必须能区分服务、否则无法把请求转发给正确机器——这可能导致某些机器暂时空闲、另一些过载；有用的是把服务临时迁移到空闲机器，方案是用虚拟机以便较容易地迁移服务。

请求派发：第一层（交换机，亦称 **front end**）的一个关键设计目标是隐藏存在多台服务器这一事实，即远程机器上的客户端应用无需了解集群内部组织；这种访问透明总通过单一接入点提供，继而由某种硬件交换机（如专用机器）实现。交换机是集群入口，提供单一网络地址；为可扩展与可用，集群可有多接入点（各由一台专用机器实现），此处只考虑单接入点。实践中两类交换机：传输层交换机接受入站 TCP 连接请求并交给某台服务器，客户端建立 TCP 连接使请求与响应都经交换机，交换机再与所选服务器建立 TCP 连接、转递请求、接受响应并回传——实际上它坐在客户端与所选服务器的 TCP 连接中间，转发 TCP 分段时改写源/目的地址，这是网络地址转换（NAT）的一种形式。另一类是应用层交换机：顾名思义，它检视请求内容而非仅看 TCP 信息，如对 Web 服务器可检视实际 URL，从而可让专用机器处理视频/媒体、其他机器处理需特定数据库的请求等。一般而言，交换机越了解请求内容，就越能决定由哪台服务器处理；明显缺点是可能比传输层交换机慢，但用合适的软硬件成本可接受——nginx 即是例证，设计上可处理数千并发连接，被许多站点用作反向代理；Apache 也可配置成应用层交换机。

Note 3.7: TCP handoff

在请求派发还较贵的年代，一种简单优雅的做法是 **TCP handoff** (Hunt 等 1997; Pai 等 1998)：交换机把连接移交给所选服务器，使所有响应直接发给客户端、不再经交换机（图 3.24）。交换机收到 TCP 连接请求后，先挑出处理该请求的最佳服务器并转发请求包；服务器回送确认，但把交换机的 IP 地址写入承载该 TCP 段的 IP 包首部源字段——这一改写是必要的，因为客户端在继续执

行 TCP 协议、期待来自交换机的应答，而非来自某个它从未听说过的服务器。显然 TCP handoff 需操作系统级修改；当响应远大于请求（如 Web 服务器）时尤为有效。交换机由此也在负载分配中扮演重要角色：它决定往哪转发，也决定了哪台服务器处理该请求。最简单的负载均衡策略是 **round robin**。但交换机须跟踪把某 TCP 连接交给了哪台服务器、至少持续到连接拆除，维护该状态并转递同一连接后续 TCP 分段，反而可能拖慢交换机。

广域集群：PlanetLab (Note 3.6) 即广域集群——参与者贡献（相对简单的）机器托管容器，并按需“切片”到任意多台机器，此后客户自负其责。今天有更复杂、商业部署的广域集群：一是 Amazon、Google 等云厂商管理的多个数据中心，让终端用户构建由散布 Internet 的（可能大量）联网虚拟机组成的广域分布式系统，重要动机是提供本地性（**locality**）——把数据与服务放在靠近客户端处（如流媒体：视频服务器离客户端越近，越易提供高质量流），这催生了内容分发网络（**CDN**）。另一种是单一组织把服务器铺到整个 Internet、与本地 ISP 协商使用其设施，如 Akamai CDN（2022 年约 40 万台服务器，遍布 1350 个 ISP、135 个以上国家）。

CDN 的一般组织：以简化的 Akamai 为例（图 3.25）。基本思想是有一个 **origin server** 托管某网站及其全部文档，每个文档由含 origin 域名的 URL 引用（如 `www.example.com`）；访问文档需把域名解析为网络地址（由 DNS 完成，第 6 章详述）。要让 origin 的内容由 CDN 托管，Akamai 先确保 `www.example.com` 解析为 `www.example.com.akamai.net`，后者指向 Akamai CDN 中的某台 **edge server**；同时把 origin 域名改为 `org-www.example.com`。客户端先查常规域名，被重定向到 `akamai.net` 解析器（第 1 步），解析器查出服务该客户端的最佳 edge server 并返回其网络地址（第 2 步），客户端联系该 edge server（第 3 步），它知道 origin 的新名字；若所请求内容不在其缓存中，就从 origin 取文档（第 4 步）、缓存并把被请求的文档返回客户端。

请求派发：此例已显示客户端请求重定向的重要性。原则上，通过恰当重定向，CDN 能掌控客户端感知的性能，也能通过避免把请求发给重载服务器来优化全局性能；这类自适应重定向策略可在把系统当前行为信息提供给决策进程后实施。关键问题是重定向对客户端是否透明——本质上只有三种技术：TCP handoff（仅适用服务器集群、不能扩展到广域网）、**DNS 重定向**与 **HTTP 重定向**。DNS 重定向是透明机制，客户端完全不知文档在哪，Akamai 的两级重定向即一例；也可直接用 DNS 返回多个地址之一。但它只能作用于整个站点（更准确说只在域名层面，单个文档名放不进 DNS 命名空间）。DNS 重定向不完美，原因有二：其一，客户端通常经本地 DNS 服务器（充当其代理）联系 DNS，于是用于定位客户端的不是其自身 IP、而是本地 DNS 服务器的 IP；Mao 等（2002）表明这可能带来巨大额外通信代价，因为本地 DNS 服务器常常并不“本地”，且即便返回多个 edge server 选项，简单地取列表首个（通常如此）也未必最佳。其二，取决于解析方案，可能根本不用本地 DNS 服务器的地址；甚至做决定的 DNS 服务器会被“请求者其实又是一个居间 DNS 服务器”这一事实所误导，此时本地性意识彻底丧失。尽管如此，DNS 重定向仍被广泛部署——至少它实现相对容易且对客户端透明，还无需依赖有位置感知的客户端软件。**HTTP 重定向**则是非透明机制：客户端请求某文档时，可能在 HTTP 响应中收到一个替代 URL 并被重定向过去；重要的是该 URL 对客户端浏览器可见，用户甚至可能把该 referral URL 收藏，从而让重定向策略失效。

原文精读

An important design goal for server clusters is to hide the fact that there are multiple servers.

In practice, we see two types of switches. In the case of transport-layer switches, the switch accepts incoming TCP connection requests, and hands off

服务器集群的一个重要设计目标，是隐藏“存在多台服务器”这一事实。

实践中我们见到两类交换机。就传输层交换机而言，它接受到来的 TCP 连接请求，并把这样的连接转交给其中一台服务器。

such connections to one of the servers.

原文定位：§3.4.4，原书 p.161–166（图 3.23 三层服务器集群、图 3.24 TCP handoff、图 3.25 Akamai CDN）

3.5 Code migration

代码迁移

此前我们主要关心只传数据的分布式系统。但有些场合下传递程序（有时甚至在执行中）能简化设计，本节细看代码迁移到底是什么。

3.5.1 迁移代码的理由（Reasons for migrating code）

传统上分布式系统的代码迁移采取进程迁移（**process migration**）形式——把整个进程从一节点移到另一节点（Milojicic 等 2000）。把运行中的进程搬到另一台机器既昂贵又棘手，得有充分理由。

性能：迄今代码迁移最重要的理由一直是性能。基本想法是：把进程从重载机器移到轻载机器可提升系统整体性能；负载常以 CPU 队列长度或利用率等表示。但 Milojicic 等在综述中已得出结论：进程迁移早已不再是提升分布式系统的可行选项。如今反而是把代码移过去以确保机器充分加载——尤其在数据中心优化能耗时，常迁移整台带应用套件的虚拟机到轻载机器以最小化使用节点数；有趣的是，迁移虚拟机虽可能需更多资源，任务本身却远比迁移进程简单（见后文）。一般地，由“关于一组机器的任务分配与再分配”做决策的负载分配算法在计算密集型系统中很重要。但在许多现代分布式系统中，优化算力不如（例如）尽量减少通信来得要紧；加之底层平台与计算机网络异构，靠代码迁移提升性能常基于定性推理而非数学模型。举例：客户端-服务器系统中服务器管理巨大数据库，若客户端应用需做大量涉及海量数据的数据库操作，把客户端应用的一部分送到服务器、只把结果传回网络可能更好，否则网络会被服务器到客户端的数据传输淹没——此时代码迁移假定“在数据所在处处理数据”通常是合理的。同样理由可用于把服务器的一部分迁到客户端：如许多交互式数据库应用中，客户端需填表、表随后被转成一连串数据库操作，在客户端处理表单、只把填好的表单发给服务器，有时能免去大量小消息穿网，客户端感知到更好性能、服务器也少花时间在表单处理与通信上。对智能手机，把代码移到手持设备而非服务器执行，可能是同时获得客户端与服务器可接受性能的唯一可行方案（Kumar 等 2013 综述了计算卸载）。代码迁移还能通过利用并行提升性能，却避开并行编程常见的繁复：典型如 Web 信息搜索——把查询实现成小型的**移动代理（mobile agent）**、从站点移到站点相当简单，做若干副本各去不同站点，可能获得相对单实例的线性加速。不过 Carzaniga 等（2007）指出，移动代理从未成功，因为它们相对其他技术并未真正显现明显优势。

隐私与安全：把代码移到数据所在处的另一理由关乎安全。当下最典型的例子是**联邦学习（federated learning）**——出于训练机器学习模型（尤其各种人工神经网络变体）的需要。简言之，神经网络由若干层节点组成、层间是带权链接；训练阶段通过喂入“已知应有输出”的数据逐渐算出权重，系统地调整权重以最小化网络输出与应有输出之差，最终得到模型，再用于未知数据。传统做法是把训练数据集中一处、常用专用高性能计算机建模；但这可能意味着敏感数据（如个人照片）须交给自己未必信任的组织。此时更好的做法是把（部分训练好的）模型送到数据所在处、用本地数据继续训练，得到更新后的模型（权重因新数据而进一步调整）；若多方参与本地训练，服务器可收集不同模型、聚合结果、把更新模型返回本地参与者继续训练。这一迭代过程（图 3.26）在聚合模型被认为训练充分、可用于实际时停止。必须说，从代码迁移角度看，联邦学习比（例如）迁移进程更容易：神经网络涉及的代码相对直白，因而更易被本地参与者接纳。这是个高层视角，入门可读 Zhang 等（2021a）；如 Lyu 等（2020）所述，仅做本地训练从隐私与安全角度可能还不够。

灵活性：支持代码迁移还有个重要理由——灵活性。构建分布式应用的传统做法是把应用切成不同部分、并事先决定各部分在哪执行（这导致了第 2 章讨论的各种多层客户端-服务器应用）。但若代码能在机器间移动，就可动态配置分布式系统。例如服务器对文件系统实现标准接口、用专有协议让远程客户端访问；通常客户端那侧基于该协议的文件系统接口实现需与客户端应用链接，这就要求该软件在客户端应用开发时就已备好。替代方案是让服务器在真正必要时（即客户端绑定到服务器时）才提供客户端的实现：客户端动态下载实现、完成必要初始化、随后调用服务器（图 3.27；代码仓库一般属服务器一部分）。这种从远程站点动态移动代码的模型要求下载与初始化的协议是标准的，且下载的代码能在客户端机器上执行——通常靠在（例如）Web 浏览器内嵌的虚拟机中运行的脚本即可。可以说，这种代码迁移形式是动态 Web 成功的关键。动态下载客户端软件的重要优势是：客户端无需预装全部软件就能与服务器对话，软件可按需移入、不用时同样丢弃；另一优势是只要接口标准化，客户端-服务器协议及其实现可随意更改，而不会影响依赖服务器的既有客户端应用。当然也有缺点，最严重的一个（第 9 章讨论）关乎安全：盲目相信下载的代码只实现所宣称的接口、却在你未受保护的硬盘上访问、还把最“肥”的部分发到不知道谁手里，未必是好主意；所幸保护客户端免受恶意下载代码伤害的方法已被充分理解。

原文精读

By far the most important reason for code migration has always been performance.

迄今为止，代码迁移最重要的理由一直都是性能。

In this case, code migration is assuming that it generally makes sense to process data close to where those data reside.

在这种情况下，代码迁移假定：在数据所在之处附近处理数据通常是合理的。

In such cases, a better approach is to bring the (partially) trained model to where the data is, and continue training with that local data.

在这类情形下，更好的做法是把（部分）训练好的模型带到数据所在之处，并用本地数据继续训练。

原文定位：§3.5.1，原书 p.167–171（图 3.26 联邦学习原理、图 3.27 动态配置客户端与服务器通信）

3.5.2 代码迁移的模型 (Models for code migration)

虽名为代码迁移，其涵盖范围远比“只移动代码”丰富。传统上分布式系统的通信关注进程间交换数据，而最广义的代码迁移处理的是在机器间移动程序、意在让其在目标处执行；某些情形（如进程迁移）还须移动程序的执行状态、待处理信号等环境部分。为理解不同模型，用 Fuggetta 等（1998）的框架：一个进程由三段组成——**code segment**（组成被执行程序的指令集合）、**resource segment**（进程所需外部资源的引用，如文件、打印机、设备、其他进程）、**execution segment**（进程当前执行状态，含私有数据、栈、程序计数器）。

还可区分 **sender-initiated** 与 **receiver-initiated** 迁移。前者在代码当前所在/执行的机器上发起，典型如向计算服务器上传程序、或向远程数据库服务器发送查询（或一批查询）；后者由目标机器发起，Java applet 即一例。**receiver-initiated** 比 **sender-initiated** 更简单：迁移常在客户端与服务器间发生、由客户端发起；而安全地上传代码到服务器（**sender-initiated** 所为）常要求客户端事先在该服务器注册并认证——即服务器须认识其所有客户端，因为客户端多半想访问服务器的资源（如磁盘），保护这些资源至关重要；相反，下载代码（**receiver-initiated**）常可匿名进行，且服务器一般不关心客户端资源，向客户端迁移代码只为改善客户端性能，故只需保护少数资源（内存、网络连接）。由此得到代码移动的四种范式（图 3.28）：**client-server**、**remote evaluation**、**code-on-demand**、**mobile agents**。**client-server** 下代码、执行状态与资源段都在服务器，执行后一般只改服务器执行状态（用星号标记）；**sender-initiated** 的 **remote evaluation** 中客户端把代码迁到服务器执行，改动服务器执行状态；

code-on-demand 是 receiver-initiated，客户端从服务器取得代码，其执行改动客户端执行状态并操作客户端资源；mobile agents 通常沿 sender-initiated 路线，把代码与执行状态一起从客户端移到服务器，操作双方资源，运行一般导致执行状态被改动。

代码迁移的最低要求是只提供 **weak mobility**：只传 code segment，可能附一些初始化数据。其特征是被迁移的程序总是重新开始（如 Java applet 从同一初始状态启动）——中间件不维护其在原处中断的历史，若需保留历史须由移动应用自身编码。弱移动的好处是简单，只要求目标机器能执行该 code segment，本质即让代码可移植（异构系统处再谈）。相反，支持 **strong mobility** 的系统可连同 **execution segment** 一起迁移：其特征是运行中的进程可被停止、移到另一台机器、并从离开处精确恢复执行。强移动远比弱移动通用，却也难实现得多：迁移进程时 execution segment 常含高度依赖特定 OS 实现的数据（如依赖 OS 进程表信息），故即便迁到同族 OS 也可能麻烦不断。对弱移动而言，迁移代码由目标进程执行还是另起进程也有差别：如 Java applet 由浏览器下载、在浏览器地址空间执行，好处是无需另起进程、避免目标机器的进程间通信，主要缺点显然是要保护目标进程免受恶意或不当代码执行——这本身就可能是把迁移代码隔离到独立进程的充分理由。除移动运行进程（进程迁移）外，强移动也可由远程克隆（**remote cloning**）支持：与进程迁移不同，克隆产生原进程的精确副本、运行在另一台机器上，且与原进程并行执行。Unix 中远程克隆靠 **fork** 出子进程、让子进程在远程机器上继续。克隆的好处是模型与许多应用已在用的模型很像，唯一差别是克隆进程在另一台机器执行，故以克隆实现的迁移是提升分布透明的一种简单办法。

原文精读

The bare minimum for code migration is to provide only weak mobility.

代码迁移的最低要求，是只提供弱移动性（weak mobility）。

The characteristic feature of strong mobility is that a running process can be stopped, subsequently moved to another machine, and then resume execution exactly where it left off.

强移动性的标志性特征是：运行中的进程可以被停止、随后移到另一台机器，然后从它离开之处精确地恢复执行。

In contrast to process migration, cloning yields an exact copy of the original process, but now running on a different machine.

与进程迁移相反，克隆产生原进程的精确副本，只不过现在运行在另一台机器上。

原文定位：§3.5.2，原书 p.171–174（图 3.28 代码移动的四种范式）

3.5.3 异构系统中的迁移（Migration in heterogeneous systems）

此前我们默认为迁移代码能在目标机器轻易执行——这适用于同构系统；但分布式系统通常建立在异构平台集合上，各有自己的操作系统与机器架构。异构带来的问题在许多方面与可移植性问题相同，解法也相似：如 1970 年代末，缓解 Pascal 向不同机器移植困难的一个简单办法是为抽象虚拟机生成机器无关的中间代码（Barron 1981）——该虚拟机需在众多平台实现，但此后 Pascal 程序便可到处运行；尽管此想法被用了些年，却从未真正成为其他语言（尤其 C）可移植性问题的通用解法。约 25 年后，异构系统中的代码迁移靠脚本语言与高度可移植的语言（如 Java、Python）应对，本质上采用与移植 Pascal 相同的办法，共同点是依赖一个（进程）虚拟机——要么直接解释源代码（脚本语言），要么解释编译器生成的中间代码（Java）。“在正确的时间出现在正确的地点”对语言开发者也很重要。后续发展削弱了对编程语言的依赖：已提出不仅迁移进程、还迁移整个计算环境的方案，基本思路是把整体环境分割（**compartmentalize**），让同一部分中的进程对自己所处的计算环境有各自的视图，这种分割以“运行操作系统与应用套件的虚拟机监视器”形式实现。借助虚拟机迁移，可把计算环境与底层系统解耦并真正迁到另一台机器（Medina & Garcia 2014；Zhang 等 2018 综述了虚拟机迁移机制）。其主要

优势是进程可对迁移本身一无所知：既不必被打断执行，也不应遇到所用资源的问题——后者要么随进程一起迁移，要么进程访问资源的方式保持不变（至少对该进程而言）。例如 Clark 等（2005）聚焦虚拟化操作系统的实时迁移，这在靠单一共享局域网紧密耦合的服务器集群中很方便；此情形下迁移有两大问题：迁移整个内存镜像与迁移对本地资源的绑定。对前者，原则上可三种方式（可组合）：其一，把内存页推到新机器、并重发迁移过程中被修改的页；其二，停止当前虚拟机、迁移内存、在新机器启动新虚拟机；其三，让新虚拟机按需拉取新页，即让进程立即在新虚拟机上开始、按需复制内存页。第二种若迁移的是提供连续服务的 **live service**，可能导致无法接受的停机；而第三种纯按需方式可能大幅延长迁移期，也可能因进程工作集迟迟未移到新机器而导致性能差。作为替代，Clark 等（2005）提出 **pre-copy** 方式，结合第一种与第二种的短暂 **stop-and-copy** 阶段——事实证明这种组合可实现非常低的服务停机时间。关于本地资源，在只处理集群服务器时会简化：因是单一网络，只需宣告新的“网络到 MAC 地址”绑定，客户端即可在正确的网络接口联系被迁移进程；若还能假定存储作为独立一层提供（如图 3.23），则迁移对文件的绑定同样简单，实际只是重建立网络连接。把虚拟机迁到另一个数据中心则更棘手：内存传输大体如前，但确实须把文件传到目标数据中心；网络连通性也稍麻烦——无论如何，客户端须能在虚拟机迁移期间与之后继续联系它。Zhang 等（2018）提及多种解法，本质多归为两类：用隧道等技术把本地网络延伸到目标，或采用支持透明重新分配网络地址的技术（即客户端用动态重绑定到实际网络地址）。

原文精读

With virtual machine migration, it becomes possible to decouple a computing environment from the underlying system and actually migrate it to another machine

借助虚拟机迁移，可以把计算环境与底层系统解耦，并真正将其迁移到另一台机器上。

As an alternative, Clark et al. [2005] propose to use a pre-copy approach which combines the first option, along with a brief stop-and-copy phase as represented by the second option.

作为替代，Clark 等人（2005）提出采用 pre-copy 方式，它把第一种选项与第二种选项所代表的短暂 stop-and-copy 阶段结合起来。

原文定位：§3.5.3，原书 p.174–176（图 3.29 迁移虚拟机时服务的响应时间变化）

3.6 Summary

小结

- 进程是分布式系统通信的基础；关键在于进程内部如何组织，尤其是否支持多线程。
- 线程在分布式系统中特别有用，因为它能在执行阻塞 I/O 时继续使用 CPU，从而构建高度高效、并行运行多个线程的服务器；性能攸关时，线程优于进程。
- 虚拟化让用户在自己的操作系统上运行一套应用、并在云中配置完整的虚拟分布式系统；性能接近宿主 OS，除非与其他虚拟机共享或虚拟机受 I/O 限制。容器化是虚拟化的一种特殊形式。
- 客户端进程实现用户界面并力求分布透明（隐藏通信、服务器位置与复制），还部分负责掩盖失败与恢复；虚拟桌面环境日益流行。
- 服务器比客户端更繁复，但设计问题相对少：迭代/并发、无状态/有状态、如何寻址服务与中断服务器等。
- 把服务器组织成集群时要特别小心：常见目标是向外界隐藏集群内部，故多用单一入口点；把它透明地替换为完全分布式方案是挑战。
- 代码迁移的两大理由是提升性能与灵活性；近来隐私与安全（如联邦学习）也成为理由。处理异构性的最佳方案是用虚拟机（进程虚拟机或 VMM）。

原文精读

In general, threads are preferred over the use of processes when performance is at stake.

一般而言，当性能攸关时，人们更偏好使用线程而非进程。

Virtualization has since long been an important field in computer science, but in the advent of cloud computing has regained tremendous attention.

虚拟化长期以来一直是计算机科学的重要领域，而在云计算到来之际又重新受到极大关注。

原文定位：§3.6，原书 p.177-179

实践清单

- 读原书第3章，重点看图3.1（IPC三次上下文切换）、图3.6（dispatcher/worker）、图3.10（进程VM/原生VMM/宿主式VMM）、图3.12（容器组织）、图3.23（三层集群）、图3.28（代码移动四范式）。
- 在本机跑一遍书中的Python示例（图3.3/3.4），观察进程与线程在共享变量上的差异；把sleep换成忙等待，验证线程被串行化、进程被分到不同核。
- 用 `unshare --pid --fork --mount-proc bash` 进入新PID namespace，用 `ps -ef` 观察“自己的init进程”，理解容器隔离。
- 对比你所用过的容器与虚拟机：分别测一次磁盘/网络I/O基准，验证“容器I/O更优、虚拟机调度更好”的趋势是否在你的场景成立。
- 为一个你熟悉的Web服务画出去程：客户端一层、交换机/反向代理一层、应用服务器一层、数据库一层，标记哪些是迭代/并发、无状态/有状态。
- 选一个你写的脚本或小应用，判断它属于四种代码移动范式中的哪一种，并说明为什么。

Communication

进程间通信是一切分布式系统的核心。本章先回到基础 (*foundations*): 分层协议、OSI 参考模型与通信分类 (持久/瞬态、同步/异步); 再看两种主流高层模型——远程过程调用 (*RPC*) 与面向消息的通信 (*message-oriented communication*, 含 *socket*、*ZeroMQ*、*MPI*、消息队列与 *AMQP*); 最后讨论把数据发给多个接收者的组播通信 (*multicast*): 基于树的确定性方案、泛洪、以及概率式的 *gossip* 传播。

4.1 Foundations

基础

在深入讨论之前先回顾一些根本问题: 先简要讨论网络通信协议 (一切分布式系统的基础), 再换一个角度对分布式系统中常见的通信类型进行分类。

4.1.1 分层协议 (Layered Protocols)

由于没有共享内存, 分布式系统中的一切通信都建立在发送和接收 (低层) 消息之上: 进程 P 要在自己的地址空间构造消息, 再执行系统调用让操作系统把消息经网络发给 Q; 但要让 P 与 Q 不发生混乱, 两者必须就所发比特的含义达成一致。为理清众多层次与问题, ISO 制定了开放系统互连参考模型 (ISO OSI, 常简称 OSI 模型)。要强调: 作为 OSI 一部分开发的协议从未被广泛使用、基本已死, 但模型本身对理解计算机网络相当有用。OSI 模型为让开放系统互联: 开放系统即愿意用标准规则 (规定消息的格式、内容与含义) 与任何其他开放系统通信的系统, 这些规则即通信协议; 一台机器要通信, 大家须就所用协议达成一致。协议提供通信服务, 有两种: 面向连接服务在交换数据前须显式建立连接、可能协商参数, 完毕再释放连接 (电话是典型); 无连接服务无需事先建立, 发送方准备好就直接发第一条消息 (投信进邮筒即一例)。计算机中两者都很常见。

OSI 模型把通信分为七层 (图 4.1), 每层向上一层提供一种或多种服务, 从而把“把消息从 A 送到 B”分解为可独立解决的可管理片段; 每层向上提供接口, 接口由一组操作构成、共同定义该层愿意提供的服务。七层自下而上为: 物理层 (标准化两台计算机如何连接、0 和 1 如何表示)、数据链路层 (检测并尽量纠正传输错误、让收发方保持同步)、网络层 (路由消息、处理拥塞)、传输层 (直接支持应用, 如建立可靠通信、支持实时流)、会话层 (支持应用间会话)、表示层 (规定与主机无关的数据表示方式)、应用层 (其余一切: 邮件、Web 访问、文件传输协议等)。当 P 要与远程 Q 通信, 它构造消息、经接口交给应用层; 应用层在前面加首部, 经 6/7 接口交给表示层, 表示层再加自己的首部往下传……有的层还在末尾加尾部; 到最底层由物理层把消息 (此时形如图 4.2) 放到物理传输介质上。消息到达 Q 所在机器后向上传递, 每层剥掉并检查自己的首部, 最后到达进程 Q, Q 可沿反向路径应答; 第 n 层首部中的信息供第 n 层协议使用。某系统所用协议的集合称为协议族 (**protocol suite**) 或协议栈。重要的是区分参考模型与实际协议: 如前所述 OSI 协议从不受欢迎, 而 Internet 开发的 TCP、IP 等协议则不然。

Note 4.1: OSI 各层的 Internet 对应

低层协议：最下面三层实现计算机网络的基本功能。物理层关心 0/1 的传输（电压、速率、能否双向同时、连接器形状与引脚含义），其协议标准化电气、光学、机械与信号接口，使一台机器发出的 0 比特确实被收成 0 而非 1（如串行通信的 USB 标准）。数据链路层把比特分组为帧（**frame**）、确保每帧正确接收：在帧首尾放特殊比特模式作标记，并以某种方式累加帧内所有字节算出校验和附于帧后；接收方重算并比对，一致则接受，不一致则请发送方重传；帧在首部带序号以区分彼此。局域网通常无需发送方定位接收方，广域网则像一张标有城市与道路的大地图，消息可能要多跳、每跳选择出线，如何选最佳路径即路由、是网络层的主要任务；问题还在于最短路径未必最佳，真正要紧的是延迟（与流量、排队消息数相关、随时间变化），故有的路由算法适应变化负载、有的只基于长期均值。目前最广用的网络协议是无连接的 **IP**，**IP 包**（网络层消息的术语）无需任何建立即可发送，每个包独立路由、不选定也不记忆内部路径。传输协议：传输层补足网络层接口未提供、但构建网络应用通常需要的服务，把底层网络变成应用开发者可用的东西。包可能丢失，有的应用自行恢复、有的偏好可靠连接，传输层负责提供此服务：把消息切成足够小的片段、各赋序号后发送，其首部讨论哪些包已发/已收、接收方还能接受多少、哪些需重传等。可靠传输连接（按定义是面向连接的）可建于面向连接或无连接的网络服务之上：前者若包到达则顺序正确，后者则可能后发先至，须由传输层软件重整，以维持“传输连接像一根大管子——放进去的消息无损地、按同样顺序出来”的假象，提供这种端到端行为是传输层的重要方面。Internet 传输协议是 **TCP**，**TCP/IP** 现已成为网络通信事实标准；协议族还支持无连接传输协议 **UDP**（本质是 IP 加些小改进），不需要面向连接的用户程序通常用 UDP。此外还有支持实时传输的 **RTP**（一种框架协议，规定实时数据的包格式但不提供保证投递的机制，并规定监控与控制 RTP 包传输的协议），以及作为 TCP 替代的 **SCTP**（主要差别在于 SCTP 把数据分组成消息，而 TCP 只在进程间搬运字节）。高层协议：传输层之上 OSI 还区分三层，实践中只用到应用层；在 Internet 协议族中，传输层之上的一切被归在一起。会话层本质是传输层的增强版，提供对话控制（跟踪当前谁在说话）与同步设施（允许在长传输中插入检查点，崩溃后只需回到最近检查点而非开头），实践中很少受支持、Internet 协议族中甚至没有，但在开发中间件方案时“会话”概念相当相关。表示层不同于低层（负责把比特可靠高效地从发送方送到接收方），它关心比特的含义：可定义包含姓名、地址、金额等字段的记录，并让发送方通知接收方“某消息以某种格式包含某记录”，从而使内部表示不同的机器更易通信。OSI 应用层原意是容纳一批标准网络应用（电子邮件、文件传输、终端仿真），如今已成为“无法归入下面各层的所有应用与协议”的容器；从 OSI 视角看，几乎所有分布式系统都只是应用。该模型欠缺的是对应用、应用特定协议与通用协议的清晰区分：如 FTP 定义客户端与服务器间传文件的协议，不应与 **ftp** 程序（实现该协议的终端用户应用）混淆；HTTP 的典型应用是 Web 浏览器与服务器，但现在也用于并非固有绑定 Web 的系统（如 Java 的对象调用机制可用 HTTP 请求受防火墙保护的远程对象）。还有许多对众多应用有用、却不能算传输协议的通用协议，常归为中间件协议。

中间件协议：中间件在逻辑上（大体）处于 OSI 应用层，但含许多值得拥有独立层次的通用协议。例如 **DNS** 是查找“名字对应网络地址”的分布式服务：按 OSI 参考模型它是个应用、逻辑上位于应用层，但它显然提供通用、与应用无关的服务，可以说属于中间件。再如认证协议（提供所声称身份的证据）并不紧密绑定某个具体应用，可作为通用服务集成进中间件系统；授权协议（只给已认证用户/进程访问其被授权资源）同样具有通用、与应用无关的性质。分布式提交协议确保一组（可能散布多台机器的）进程中要么都执行某操作、要么都不执行（即原子性，广泛用于事务），它们可提供与应用无关的接口，从而构成通用事务服务，通常属于中间件而非 OSI 应用层。最后，分布式锁协议可保护资源免受跨多台机器的多个进程同时访问，也不难设计成与应用无关、经简单接口访问，故一般属中间件。这些例子并不直接关乎通信，但也有许多中间件通信协议：如远程过程调用让进程能在本地调用“实际在远程机器上实现”的过程，属最古老的中间件服务之一、用于实现访问透明；类似还有为实时数据传输设置与同步流的高层通信服务；有些中间件系统还提供可扩展到广域网数千接收者的可靠组播服

务。如此分层便得到图 4.3 的 简化参考模型：会话层与表示层被单个中间件层（含与应用无关的协议）取代，网络与传输服务被归为操作系统通常提供的通信服务，由 OS 管理用于建立通信的最底层硬件。

原文精读

Due to the absence of shared memory, all communication in distributed systems is based on sending and receiving (low level) messages.

由于没有共享内存，分布式系统中的一切通信都建立在发送和接收（低层）消息之上。

In the case of a connection-oriented service, before exchanging data the sender and receiver first explicitly establish a connection, and possibly negotiate specific parameters of the protocol they will use.

就面向连接的服务而言，在交换数据之前，发送方与接收方首先显式建立连接，并可能协商它们将使用的协议的特定参数。

It is important to distinguish a reference model from its actual protocols.

重要的是把参考模型与其实际协议区分开来。

原文定位：§4.1.1，原书 p.183-190（图 4.1 OSI 分层/接口/协议、图 4.2 网络上的典型消息、图 4.3 适配后的参考模型）

4.1.2 通信的类型 (Types of Communication)

本节余下聚焦高层中间件通信服务。可把中间件看作客户端-服务器计算中的一项附加服务（图 4.4）。以电子邮件系统为例：投递系统核心可视为中间件通信服务，每台主机跑一个用户代理让用户撰写、发送、接收邮件；发送方代理把邮件交给投递系统并期待最终送达，接收方代理连接投递系统查看是否有邮件，有则转给用户代理供阅读。电子邮件是通信持久（**persistent**）的典型：持久通信中，已提交待传的消息由通信中间件存储到送达为止；中间件会把它存在图 4.4 所示的一处或多处存储设施中。于是发送应用提交后不必继续执行，接收应用在消息提交时也不必在执行。相反，瞬态通信（**transient communication**）中，消息只在收发应用执行期间被存储：若因传输中断或接收方当前未活动而无法投递，就直接丢弃。典型的传输层通信服务只提供瞬态通信，此时通信系统由传统存储转发路由器构成，路由器若无法把消息交给下一跳或目的主机，就直接丢弃。

除持久/瞬态外，通信还分同步/异步。异步通信的特征是发送方提交消息后立即继续，这意味着消息在提交时被中间件（暂时）存储；同步通信中发送方被阻塞、直到其请求被确认接受。同步可在三处发生：其一，发送方阻塞到中间件通知将接手传输请求；其二，阻塞到请求已投递给预定接收方；其三，阻塞到请求被完全处理，即到接收方返回响应之时。实践中出现各种持久性与同步性的组合：常见的“持久 + 请求提交时同步”是许多消息队列系统的常用方案（后文讨论）；“瞬态 + 请求被完全处理后同步”也被广泛使用，正对应远程过程调用（下节讨论）。

原文精读

With persistent communication, a message that has been submitted for transmission is stored by the communication middleware as long as it takes to deliver it to the receiver.

在持久通信中，已提交待传的消息会由通信中间件一直存储，直到将其投递给接收方为止。

In contrast, with transient communication, a message is stored by the communication system only as long as the sending and receiving application are executing.

相反，在瞬态通信中，消息只在发送方与接收方应用都处于执行状态期间被通信系统存储。

The characteristic feature of asynchronous communication is that a sender continues immediately after it has submitted its message for transmission.

异步通信的标志性特征是：发送方在提交待传消息后立即继续执行。

原文定位：§4.1.2，原书 p.190–192（图 4.4 中间件作为应用级通信中的中介服务）

4.2 Remote procedure call

远程过程调用

许多分布式系统基于进程间显式消息交换，但 `send` 与 `receive` 完全不掩盖通信，而对获得访问透明而言这种掩盖很重要。这个问题久已存在，直到 1980 年代研究者（Birrell & Nelson 1984）引入一种全新做法才有了转机——想法本身简单，但含义常很微妙。一言以蔽之：允许程序调用位于其他机器上的过程。当机器 A 上的进程调用机器 B 上的过程时，A 上的调用进程被挂起，被调过程在 B 上执行；信息可经参数从调用者传到被调者、再经过程结果返回。对程序员完全看不到任何消息传递。此法即 **远程过程调用（remote procedure call, RPC）**。它虽简单优雅，却存在微妙问题：调用与被调过程运行在不同机器、处于不同地址空间，带来复杂性；参数与结果须传递，若两台机器不同则更麻烦；且任一方或双方都可能崩溃，各种失败各带来不同问题。

4.2.1 RPC 的基本操作（Basic RPC operation）

RPC 的目标是让远程调用尽量像本地调用，即追求透明：调用过程不应察觉被调过程在另一台机器上执行，反之亦然。设某程序可访问某数据库、能向存储列表追加数据并返回修改后列表的引用，该操作由例程 `append` 提供：`newlist = append(data, dbList)`。传统单机系统中 `append` 由链接器从库中取出并插入目标程序，可能只是一段用几次文件操作访问数据库的短过程；即便它最终只做几次基本文件操作，调用方式仍是通常的“把参数压栈”，程序员不知道也不该知道其实现细节。RPC 以类似方式实现透明：当 `append` 实为远程过程时，提供给调用客户端的是另一个版本的 `append`，称为 **client stub**；它同样以正常调用序列被调用，但不执行追加操作，而是把参数打包成消息、请求发给服务器（图 4.6），随后调用 `receive` 阻塞自己直到应答返回。消息到达服务器后，服务器操作系统将其交给 **server stub**——client stub 在服务器端的对应物，是一段把网络到来的请求转成本地过程调用的代码；server stub 通常已调用 `receive`、阻塞等待入站消息。它从消息解包参数、再以通常方式调用服务器过程；从服务器视角看，就像被客户端直接调用——参数与返回地址都在栈上，一切正常。服务器完成工作后照常把结果返回调用者（此处即 server stub）。server stub 拿回控制权后，把结果打包成消息、调用 `send` 回送客户端，随后通常再调用 `receive` 等待下一个请求。结果消息到达客户端机器后，操作系统经先前调用的 `receive` 把它交给 client stub，客户端进程随之解除阻塞；client stub 检视消息、解包结果、复制给调用者并照常返回。当调用者在 `append` 调用后拿回控制时，它只知道往列表追加了数据，完全不知道工作是在另一台机器上远程完成的。这种让客户端“幸福地无知”正是整个方案的妙处：对它而言，远程服务通过普通（本地）过程调用来访问，而非 `send/receive`；消息传递的全部细节都藏在两个库过程里，正如真正发起系统调用的细节藏在传统库中一样。总结起来，RPC 分以下步骤：(1) 客户端过程以正常方式调用 client stub；(2) client stub 构造消息、调用本地操作系统；(3) 客户端 OS 把消息发给远程 OS；(4) 远程 OS 把消息交给 server stub；(5) server stub 解包参数、调用服务器；(6) 服务器完成工作、把结果返回 stub；(7) server stub 把结果打包、调用其本地 OS；(8) 服务器 OS 把消息发给客户端 OS；(9) 客户端 OS 把消息交给 client stub；(10) stub 解包结果、返回给客户端。这些步骤的净效果，是把“客户端过程到 client stub 的本地调用”转成“对服务器过程的本地调用”，而客户端与服务器都不知中间步骤或网络的存在（图 4.7）。

Note 4.2: 常规过程调用与参数传递

以 `newlist = append(data, dbList)` 为例：其目的是把全局列表对象 `dbList` 追加一个数据元素 `data`。要点是在 C 等语言中 `dbList` 实现为指向列表对象的引用（指针），而 `data` 可能直接以其值表示。调用时两者都压栈供 `append` 实现访问：`data` 遵循 **call-by-value**，`dbList` 则遵循 **call-by-reference**（图 4.5）。值参数只是个已初始化的局部变量，被调过程可改它但不影响调用方原值；而指针参数压栈的是列表对象在主存中的地址，故追加会真正修改列表对象——这一差别对 RPC 相当重要。还有一种机制 **call-by-copy/restore**：像传值那样先把变量复制到栈、调用后再复制回去覆盖调用方原值，多数情况下与传引用效果相同，但若同一参数在参数表出现多次等情况语义会不同。用哪种机制通常由语言设计者定、是语言的固定属性：C 中标量总传值、数组总传引用；有些 Ada 编译器对 `inout` 参数用 `copy/restore`、另一些用传引用（语言定义容许二选一，语义因此略模糊）；Python 中所有变量都传引用，但有些实际被复制到局部变量、从而模仿了传值行为。

原文精读

The idea behind RPC is to make a remote procedure call look as much as possible as a local one.

A server stub is the server-side equivalent of a client stub: it is a piece of code that transforms requests coming in over the network into local procedure calls.

This blissful ignorance for the client is the beauty of the whole scheme.

RPC 背后的想法，是让一次远程过程调用尽可能地看起来像本地调用。

server stub 是 client stub 在服务器端的对应物：它是一段把经网络到来的请求转换为本地过程调用的代码。

客户端的这种“幸福的无知”，正是整个方案的妙处所在。

原文定位：§4.2.1，原书 p.192–197（图 4.5 本地调用参数传递、图 4.6 RPC 原理、图 4.7 `doit(a,b)` 的调用步骤、图 4.8 Python RPC 示例）

4.2.2 参数传递（Parameter passing）

client stub 的职责是取参数、打包成消息发给 server stub——听起来直截了当，却并不那么简单。把参数打包进消息称为 **parameter marshaling**。回到 `append`：要确保两个参数（`data` 与 `dbList`）经网络传送并被服务器正确解释。须意识到服务器看到的只是一串传入字节，消息通常不附带“这些字节是什么意思”的信息；而且还会再次遇到同一问题：这些元信息又该如何被服务器识别为元信息？除解释问题外，还须处理字节在内存中的摆放可能因机器架构而异：有些机器（如 Intel 处理器）从右到左给字节编号，许多其他机器（如较老的 ARM）反之——Intel 格式称 **小端（little endian）**、旧 ARM 格式称 **大端（big endian）**。字节序对网络同样重要，机器传输（进而接收）比特与字节的次序也可能不同，而网络上通常用大端。解法是把待发数据转成机器与网络无关的格式，并确保通信双方期待相同的消息数据类型：后者通常可在编程语言层面解决，前者靠依机器而异的例程在“机器/网络无关格式”之间来回转换。编组与解组（**marshaling and unmarshaling**）正是关于这种向中立格式的转换，构成 RPC 必不可少的组成部分。

接下来是个难题：指针（更一般地，引用）如何传递？答案是：只能极其困难地传，甚至根本传不了。指针只在其所在进程的地址空间内有意义。回到 `append`：第二个参数 `dbList` 实现为指向数据库中列表的引用；若该引用只是指向调用方主存中某本地数据结构的指针，就不能简单传给服务器——传过去的指针值多半指向完全不同的东西。一种解法是干脆禁止指针与引用参数，但它们太重要，此路极不可取，实际也常无必要。首先，引用参数常用于固定大小的数据类型（如静态数组）或能在运行

时轻易算出大小的动态类型（如字符串、动态数组）；此时可直接复制该参数所指的整个数据结构，用 copy-by-value/restore 取代传引用——虽语义不总完全一致，但常常足够好；一个明显优化是：当 client stub 知道所指数据只读，调用结束时就不必复制回去，传值便足够。更复杂的类型常也能支持，尤其当语言支持这些类型时——如 Python 或 Java 支持用户自定义类，语言系统可为这些类型提供全自动的编组与解组；但一旦涉及大型、嵌套或其他复杂的动态数据结构，自动（解）编组可能不可用、甚至未必可取。指针与引用的问题在于它们只在本地有意义；可用全局引用（**global references**）缓解：对调用与被调进程都有意义的引用。例如若客户端与服务器能访问同一文件系统，传文件句柄而非指针或许可行。一个重要观察是：两个进程都必须确切知道传全局引用时该做什么——若考虑带相关数据类型的全局引用，调用与被调进程应对“可执行哪些操作”有一致图景，且都须就“传文件句柄时究竟怎么做”达成一致；这些通常可由恰当的编程语言支持解决。

原文精读

Packing parameters into a message is called parameter marshaling.

把参数打包进消息，称为参数编组（parameter marshaling）。

We now come to a difficult problem: How are pointers, or in general, references passed? The answer is: only with the greatest of difficulty, if at all.

现在我们遇到一个难题：指针，或更一般地，引用，是如何传递的？答案是：只能极其困难地传递，甚至根本传不了。

Problems can be alleviated by using global references: references that are meaningful to the calling and the called process.

通过使用全局引用可以缓解这些问题：全局引用即对调用进程与被调进程都有意义的引用。

原文定位：§4.2.2，原书 p.197–201（图 4.9 带正确编组的 RPC 示例、图 4.10 对象按引用或按值传递）

4.2.3 基于 RPC 的应用支持（RPC-based application support）

要掩盖远程过程调用，调用者与被调者须就所交换消息的格式达成一致，并在（例如）传递复杂数据结构时遵循相同步骤；换言之，RPC 双方应遵循同一协议，否则 RPC 无法正确工作。支持基于 RPC 的应用开发至少有两种方式：其一，让开发者精确指定哪些需要远程调用，据此可生成完整的客户端与服务器端 stub；其二，把远程过程调用内嵌为编程语言环境的一部分。

Stub 生成：以图 4.11(a) 的函数 `someFunction` 为例，它有字符、浮点数、五个整数组成的数组三个参数。设字长为四字节，RPC 协议可规定：字符放在一个字的最右字节（其余三字节留空）、浮点占一个字、数组占与数组长度相等的若干字并在前面放一个给出长度的字（图 4.11(b)）。据此，`someFunction` 的 client stub 知道必须用图 4.11(b) 的格式，server stub 也知道该函数的进站消息将是此格式。定义消息格式只是 RPC 协议的一方面、还不够：还须客户端与服务器就简单数据结构的表示达成一致，如整数、字符、布尔等——例如协议可规定整数用 二补码、字符用 16 位 **Unicode**、浮点用 IEEE 754 格式、且全部小端存储；有了这些补充信息，消息才能被无歧义地解释。把编码规则钉到最后一个比特后，剩下的是双方就实际的消息交换达成一致：例如可决定用面向连接的传输服务（如 TCP/IP），也可用不可靠数据报服务、让客户端与服务器把差错控制作为 RPC 协议的一部分实现；实践中存在多种变体，由开发者指示偏好何种底层通信服务。RPC 协议完全定义后，就要实现客户端与服务器 stub。幸运的是，同一协议下不同过程的 stub 通常只在与应用对接的接口上不同。接口由一批可被客户端调用、由服务器实现的过程组成；接口通常以与客户端或服务器相同编程语言提供（严格说并非必须）。为简化，接口常通过接口定义语言（**Interface Definition Language, IDL**）规定，再编译成 client stub、server stub 及相应的编译期或运行期接口（图 4.12）。该图展示的是编译型语言的情形，不难想象解释型语言也类似；客户端与服务器代码也可用不同语言编写（如客户端 Python、服务

器 C)，此时公共 include 部分将分别为客户端与服务器各含文件。此方案中重要的是 运行库 (**runtime library**)：它是对实际运行时系统的接口，构成中间件。实践表明，用 IDL 能显著简化基于 RPC 的客户端-服务器应用；因为可轻松生成完整的客户端与服务器 stub，所有基于 RPC 的中间件系统都提供 IDL 支持应用开发，有时甚至是强制的。

原文精读

To simplify matters, interfaces are often specified through an Interface Definition Language (IDL).

为简化起见，接口常通过接口定义语言 (IDL) 来规定。

Practice shows that using an interface definition language considerably simplifies client-server applications based on RPCs.

实践表明，使用接口定义语言能显著简化基于 RPC 的客户端-服务器应用。

原文定位：§4.2.3，原书 p.201–205 (图 4.11 函数与其消息格式、图 4.12 由接口定义文件生成 stub、图 4.13 语言的 RPC 内嵌)

4.2.4 RPC 的变体 (Variations on RPC)

与常规过程调用一样，客户端调用远程过程后会阻塞直到应答返回。当没有结果要返回时，这种严格的请求-应答行为不必要；当需连续执行多个 RPC 时又可能妨碍效率。下面看两种变体。

异步 RPC：为支持根本没有结果返回的情形，RPC 系统可提供 **异步 RPC**。其中，服务器原则上在收到 RPC 请求的那一刻立即回送一个应答，然后才在本地调用所请求的过程；该应答作为“服务器将处理此 RPC”的确认，客户端收到确认后即继续、不再阻塞 (图 4.14(b) 对比普通请求-应答的 4.14(a))。当有结果要返回、但客户端不愿干等时，异步 RPC 也有用：典型如客户端需独立联系多台服务器，它可一个接一个发出调用请求、使各服务器大致并行工作；全部发完后客户端再开始等待各个结果返回。此类情形适合把通信组织成 **异步 RPC + 回调**，又称 **deferred synchronous RPC** (图 4.15)：客户端先调用服务器、等接受后继续；结果可用时服务器发来响应消息、触发客户端的回调 (**callback**)——回调是当特殊事件 (如到来消息) 发生时被调用的用户自定义函数，直白实现是另起一个线程，让它在事件发生时阻塞等待、主进程继续，事件发生则解除阻塞并调用该函数。还应注意存在 **one-way RPC**：客户端发出请求后立即继续，不等服务器确认接受。其问题是可靠性无保证时，客户端无法确知请求是否会被处理 (第 8 章再谈)。类似地，deferred synchronous RPC 中客户端也可轮询服务器看结果是否就绪，而非让服务器回调。

组播 RPC：异步与 deferred synchronous RPC 便于同时执行多个 RPC。采用 one-way RPC (服务器不告知客户端已接受其调用请求、而是立即开始处理，客户端发出后即继续)，组播 RPC 就归结为把 RPC 请求发给一组服务器 (图 4.16)：客户端把请求发给两台服务器，它们各自独立并行处理，完成后把结果返回客户端并在那里触发回调。有几点需考虑：其一，客户端应用可能不知道 RPC 实际被转发给了不止一个服务器——例如为提高容错，可决定所有操作都由备用服务器执行以便主服务器失败时接管；服务器被复制这一事实可由恰当的 stub 对客户端完全隐藏，甚至连 stub 也无需知道 (例如使用了传输层组播地址)。其二，需考虑如何处理响应：客户端是在全部响应到达后继续，还是只等一个？视情况而定。服务器为容错而复制时，可只等第一个响应、或等到多数服务器返回相同结果；而若服务器复制是为了在不同部分输入上做同样的工作，则其响应可能须先合并客户端才能继续。这些都能藏在客户端 stub 里，但应用开发者至少须指定该组播 RPC 的目的。

原文精读

With asynchronous RPCs, the server, in principle, immediately sends a reply back to the client the

使用异步 RPC 时，服务器原则上在收到 RPC 请求的那一刻立即向客户端回送一个应答，之后才

moment the RPC request is received, after which it locally calls the requested procedure.

在本地调用所请求的过程。

a multicast RPC boils down to sending an RPC request to a group of servers.

组播 RPC 归结为把一个 RPC 请求发送给一组服务器。

原文定位：§4.2.4，原书 p.205–208（图 4.14 传统与异步 RPC、图 4.15 deferred synchronous RPC、图 4.16 组播 RPC）

4.3 Message-oriented communication

面向消息的通信

RPC 与远程对象调用有助于掩盖分布式系统中的通信、增强访问透明，但二者并非总是合适：尤其当无法假定接收方在请求发出时正在执行时，就需要替代的通信服务；同样，RPC 固有的同步性（客户端阻塞到请求被处理完）也可能需要被替换。这个“别的东西”就是消息传递（**messaging**）。本节先细看同步行为究竟是什么、有何影响，再讨论假定通信时各方正在执行的消息系统，最后考察即使对方未在执行也能交换信息的信息队列系统。

4.3.1 基于 socket 的简单瞬态消息（Simple transient messaging with sockets）

许多分布式系统与应用直接建立在传输层提供的简单面向消息模型之上。为更好理解中间件方案中的面向消息系统，先讨论经传输层 **socket** 的消息传递。传输层接口的标准化受到特别关注：让程序员用一组简单操作使用整套（消息）协议，也使应用更易移植到其他机器。以 1970 年代 Berkeley Unix 引入、后被采纳为 POSIX 标准的 **socket** 接口为例（仅极少改动）。概念上，**socket** 是通信端点：应用可向它写要发到网络的数据、也可从中读入站数据；它是对本地操作系统用于特定传输协议的实际端口的抽象。下面聚焦 TCP 的 socket 操作（图 4.17）。服务器一般按序执行前四个操作：调用 **socket** 为特定传输协议创建新的通信端点（内部即为收发消息预留资源）；**bind** 把本地地址与新 socket 关联（如把机器 IP 与一个（可能知名的）端口号绑定到 socket，告诉 OS 服务器只想在该地址与端口接收消息；面向连接通信中该地址用于接收入站连接请求）；**listen** 仅用于面向连接通信，是非阻塞调用，让本地 OS 为调用者可接受的最大待处理连接请求数预留足够缓冲；**accept** 阻塞调用者直到连接请求到达，到达后本地 OS 创建一个属性相同的新 socket 并返回给调用者——这使服务器可（例如）**fork** 一个进程用新连接处理实际通信，而原 socket 可回去等待下一个连接请求。客户端同样须先 **socket** 创建，但不必显式绑定本地地址（OS 在建立连接时可动态分配端口）；**connect** 要求调用者指定要发连接请求的传输层地址，客户端阻塞直到连接成功建立，之后双方即可经 **send** 与 **receive** 交换信息。最后，用 socket 时关闭连接是对称的：客户端与服务器都调用 **close**。尽管规则有许多例外，客户端与服务器进行面向连接通信的一般模式如图 4.18 所示。

原文精读

Conceptually, a socket is a communication end point to which an application can write data that are to be sent out over the underlying network, and from which incoming data can be read.

概念上，socket 是一个通信端点：应用可以向它写入要经底层网络发送的数据，也可以从它读取到来的数据。

closing a connection is symmetric when using sockets, and is established by having both the client and server call the close operation.

使用 socket 时，关闭连接是对称的，做法是让客户端与服务器都调用 close 操作。

原文定位：§4.3.1，原书 p.208–213（图 4.17 TCP/IP 的 socket 操作、图 4.18 面向连接通信模式、图 4.19–4.20 socket 客户端-服务器示例）

4.3.2 高级瞬态消息 (Advanced transient messaging)

标准 socket 方式非常基础、因而也相当脆弱：容易出错；而且 socket 基本只支持 TCP 或 UDP，任何额外的消息设施都得由应用程序员另行实现。实践中我们常需更高级的面向消息通信方式，以便简化网络编程、超越现有网络协议的功能、更好利用本地资源等。

用消息模式编程：**ZeroMQ**。一种思路基于这样的观察：许多消息应用（或其组件）可有效按少数几个简单通信模式来组织；进而为每种模式提供 socket 增强，网络分布式应用便更好写。ZeroMQ 即此法 (Hintjens 2013; Akgul 2013)。与 Berkeley 一样，ZeroMQ 也提供 socket 承载全部通信，实际消息传输一般走 TCP 连接，且本质上都是面向连接的（传输前先建立连接）；但连接的建立与维护大体被藏在幕后，程序员无需操心。为更简化，一个 socket 可绑定到多个地址，使服务器能经单一接口处理来自很不同来源的消息（例如用一个阻塞 `receive` 监听多个端口）：ZeroMQ socket 因而能支持多对一通信，而不像标准 Berkeley socket 只有一对一；同样也支持一对多（即组播）。ZeroMQ 的要旨在于通信是异步的：发送方提交消息给底层通信子系统后通常即刻继续。异步与面向连接结合有个有趣副作用：进程可请求建立连接、并随即发送消息，即便接收方尚未启动、尚未准备好接受连接请求、更谈不上消息——此时连接请求与随后的消息会排队在发送方一侧，由 ZeroMQ 库中另一个线程负责最终建立连接并传输给接收方。简言之，ZeroMQ 通过 **成对匹配 socket** 把 socket 通信提升到更高抽象：用于发消息的某类 socket 与用于收消息的对应类配套，每对 socket 类型对应一种通信模式。ZeroMQ 支持的三种最重要模式是 **request-reply**、**publish-subscribe** 与 **pipeline**。`request-reply` 用于传统客户端-服务器通信（如 RPC 常用者）：客户端用 **REQ** 型 socket 发请求、期待服务器以恰当的响应回复，服务器假定用 **REP** 型 socket；该模式免去调用 `listen` 与 `accept`，简化开发；且服务器收到消息后再调用 `send` 会自动指向原发送者，客户端发过消息后调用 `recv` 时 ZeroMQ 也假定它在等原接收方的响应（这一思路其实已编码在图 4.20(b) 的本地 `sendrecv` 操作中）。`publish-subscribe` 模式中客户端订阅服务器发布的特定消息（第 2.1.3 节协调处见过）：只有客户端订阅了的消息才会被传输，若服务器发布无人订阅的消息则这些消息会丢失；最简单形式即服务器向多个客户端组播消息，服务器用 **PUB** 型 socket、每个客户端用 **SUB** 型 socket 并连到服务器的 socket；默认客户端不订阅任何具体消息，故未显式订阅前不会收到服务器发布的消息。`pipeline` 模式的特征是：一个进程想推出其结果，假定有其他进程想拉入这些结果；其本质是推出者并不真正关心由哪个其他进程拉走结果——第一个可用的就行；同理，从多个进程拉入结果者也是从第一个把结果变得可用的推出进程处拉取。`pipeline` 模式意在让尽可能多的进程保持工作、尽快把结果推出流水线。

原文精读

Essential to ZeroMQ is that communication is asynchronous: a sender will normally continue after having submitted a message to the underlying communication subsystem.

ZeroMQ 的关键在于其通信是异步的：发送方在把消息提交给底层通信子系统之后通常就会继续执行。

MPI is designed for parallel applications and as such is tailored to transient communication.

MPI 是为并行应用而设计的，因此它面向的是瞬态通信。

消息传递接口 (**Message-Passing Interface, MPI**)。随高性能多计算机的出现，开发者寻求既能方便编写高效应用（抽象层次要合适）、实现开销又极小的面向消息操作。socket 被认定不足，原因有二：其一，抽象层次不对，只支持简单的发送与接收；其二，socket 是为经通用协议栈（如 TCP/IP）的跨网通信设计的，不适合为高速互连网络（如高性能服务器集群所用者）开发的专有协议，后者需要能处理更高级特性（如不同形式的缓冲与同步）的接口。结果多数互连网络与高性能多计算机都随附专有通信库，提供丰富且总体高效的通信操作——但互不兼容，应用开发者于是面临可移植性问题。硬件与平台无关的需求最终带来一个消息传递标准，就叫 **MPI**。MPI 面向并行应用，因而面向瞬态通

信；它直接使用底层网络，并假定进程崩溃、网络分区等严重失败是致命的、不需自动恢复。MPI 假定通信发生在一个已知进程组内：每组有标识符，组内每个进程也有（局部）标识符，故 (groupID, processID) 对唯一标识消息的源或目的、取代传输层地址；一次计算中可有若干可能重叠、且同时执行的进程组。MPI 核心是支持瞬态通信的消息操作（图 4.24），要理解其语义可回想图 4.4、把中间件完全看作 MPI 实现；特别是中间件维护自己的缓冲，同步常与“所需数据何时从调用应用复制到中间件”相关：MPI_BSEND 支持瞬态异步通信（发送方提交消息，一般先复制到 MPI 运行时系统的本地缓冲，复制完即继续；本地运行时系统在有接收方调用接收时把消息从本地缓冲取出并负责传输）；MPI_SEND 是阻塞发送、语义依赖实现（或阻塞到消息被复制到发送方一侧的 MPI 运行时系统，或阻塞到接收方发起接收操作）；MPI_SSEND 提供同步通信（发送方阻塞到其请求被接受进一步处理）；MPI_SENDRECV 提供最强形式的同步（发送方发给接收方并阻塞到后者返回应答，基本对应普通 RPC）。MPI_SEND 与 MPI_SSEND 都有避免从用户缓冲复制到本地 MPI 运行时内部缓冲的变体（本质对应异步通信）：MPI_ISEND 中发送方传消息指针、随即继续，为防止通信完成前消息被覆盖，MPI 提供检查完成、必要时阻塞的操作；MPI_IRESEND 同样只传指针，当运行时系统表明已处理消息时，发送方即可保证接收方已接受该消息、正在处理。MPI_RECV 接收消息、阻塞调用者直到消息到达；异步变体 MPI_Irecv 让接收方表明已准备接受消息，可检查消息是否到达或阻塞直到到达。这些操作语义并不总是直白，有时不同操作可互换而不影响程序正确性；官方说法是支持如此多形式是为了让实现者有足够空间优化性能，愤世嫉俗者则说委员会达不成一致就全塞了进去。如今 MPI 已到第四版、有 650 多个操作；考虑到它面向高性能并行应用，这种多样性或许更易理解。

原文精读

In practice, we do often need more advanced approaches for message-oriented communication to make network programming easier, to expand beyond the functionality offered by existing networking protocols, to make better use of local resources

实践中，我们确实常需要更高级的面向消息通信方式，以简化网络编程、超越现有网络协议所提供的功能、更好地利用本地资源。

At the core of MPI are messaging operations to support transient communication

MPI 的核心，是支持瞬态通信的各种消息操作。

原文定位：§4.3.2，原书 p.213–220（图 4.21 ZeroMQ 客户端-服务器、图 4.22 多播 socket、图 4.23 生产者-工作者、图 4.24 MPI 消息操作）

4.3.3 面向消息的持久通信 (Message-oriented persistent communication)

现在看一类重要的面向消息中间件服务，通常称消息队列系统或面向消息的中间件 (**message-oriented middleware, MOM**)。消息队列系统为持久异步通信提供广泛支持，其本质是提供中期存储能力，无需发送方或接收方在消息传输期间处于活动状态。与 socket 和 MPI 的一个重要差别是：消息队列系统通常面向“允许耗时以分钟而非秒或毫秒计”的消息传输。

消息队列模型：基本思想是应用通过把消息插入特定队列来通信；这些消息经一系列通信服务器转发、最终投递到目的地，即便消息发出时目的地宕机也没关系。实践中多数通信服务器彼此直连，即消息一般直接传到目的服务器。原则上每个应用有自己私有的队列，其他应用可向其发消息；队列只能被其关联应用读取，但也可多个应用共享同一队列。消息队列系统的一个重要方面是：发送方一般只得到“消息最终会被插入接收方队列”的保证，对消息何时、甚至是否真会被读取不作保证——这完全取决于接收方的行为。这些语义允许通信在时间上解耦（如第 2.1.3 节所讨论）：接收方无需在消息发到其队列时正在执行，发送方也无需在其消息被接收方取走时正在执行，收发双方可完全独立执行。事

实上，消息一旦存入队列就会留到被移走为止，无论其发送方或接收方是否在执行——由此得到收发方执行模式的四种组合（图 4.25）：(a) 收发双方在整条消息传输期间都在执行；(b) 只有发送方执行、接收方被动（无法投递消息的状态），但发送方仍可发消息；(c) 被动的发送方与活动的接收方——接收方可读发给它的消息，但不要求各发送方也在执行；(d) 系统在收发双方都被动时仍在存储（并可能传输）消息——可以说只有支持这最后一种配置，消息队列系统才真正提供持久消息。消息原则上可含任意数据；从中间件视角唯一重要的是消息被恰当寻址，实践中通过给出系统范围内唯一的队列名来寻址。有时消息大小受限，但底层系统也可能以对应用完全透明的方式做分片与重组；其效果是提供给应用的基本接口可以很简单（图 4.26）：PUT（把消息追加到指定队列，非阻塞）、GET（阻塞直到指定队列非空，并移走第一条消息；仅当队列空时阻塞；变体可按优先级或匹配模式搜索特定消息）、POLL（非阻塞版：检查指定队列、移走第一条，若队列空或没找到指定消息则调用进程继续）、NOTIFY（安装一个 handler 作为回调函数，每当有消息被放入指定队列时自动调用）。回调还可用于在无进程执行时自动启动一个进程去取队列消息，这常由接收方的一个 daemon 实现，它持续监控队列并相应处理。

消息队列系统的一般架构。队列由 **queue manager** 管理，它要么是独立进程、要么是链接进应用的库。经验法则是：应用一般只能把消息放入本地队列，也只能从本地队列取出消息。因此若为应用 A 管理队列的 QMA 作为独立进程运行，QMA 与 A 通常放在同一台机器、最坏也在同一局域网（若所有 queue manager 都链接进各自应用，就谈不上持久异步消息系统了）。若应用只能往本地队列放消息，则每条消息都须携带目的信息，queue manager 负责确保消息到达目的地，于是引出若干问题。第一，目的队列如何寻址：为增强位置透明，队列最好有逻辑的、与位置无关的名字；若 queue manager 是独立进程，用逻辑名意味着每个名字须关联一个联系地址（如 (host,port) 对），且名字到地址的映射须对 queue manager 随时可用（图 4.27）；实践中联系地址还携带更多信息，尤其所用协议（如 TCP 或 UDP）。第二，名字到地址的映射如何提供给 queue manager：常见做法是做成查表、并复制到所有 manager，这显然带来维护问题（每新增或改名一个队列，许多（若非全部）表都需更新），第 6 章将讨论缓解办法。第三，与高效维护名字-地址映射相关：我们隐含假定若 queue manager QMA 知道 QMB 处的目的队列，QMA 就能直接联系 QMB 传消息——这实际意味着每个 queue manager 的（联系地址）都须为其他所有 manager 所知，规模一大就有可扩展性问题。实践中常有专门的 queue manager 充当路由器，把入站消息转发给其他 queue manager，于是消息队列系统可逐渐长成一个完整的应用级覆盖网络。若只需少数路由器了解网络拓扑，则源 queue manager 只需知道“给定目的队列该把消息转发给哪个相邻路由器 R”，R 又只需跟踪其相邻路由器以决定再往哪转发，如此等等；当然仍须有所有 queue manager（含路由器）的名字-地址映射，但不难想象这样的表可以小得多、也易于维护。

消息代理（**Message brokers**）。消息队列系统的一个重要应用领域是把既有与新的应用集成成一个连贯的分布式信息系统。若假定与应用的通信经消息进行，集成就要求各应用能理解所收到的消息，实践中这要求发送方的出站消息格式与接收方一致、且语义符合接收方预期——收发双方本质上要说同一种语言，即遵循同一消息协议。这样做的问题是：每当一个带自己消息协议的应用 A 加入系统，对每个要与 A 通信的其他应用 B 都需提供转换二者消息的手段；在 N 个应用的系统中就需 $N \times N$ 个协议转换器。替代方案是约定公共消息协议（如传统网络协议所为），但对消息队列系统一般行不通：问题在于这些系统所处的抽象层次——只有当使用该协议的那批进程确实有足够多共同点时，公共消息协议才有意义；而组成分布式信息系统的应用往往高度多样，发明“一刀切”方案根本不可行。若只聚焦消息的格式与含义，可通过提升抽象层次获得共性，如用 XML 消息：消息携带关于自身组织的信息，被标准化的是它们描述内容的方式，于是应用可提供能被自动处理的组织信息；当然这通常还不够，还须确保消息语义被充分理解。鉴于这些问题，一般做法是“学会与差异共处”，尽量让转换尽可能简单。在消息队列系统中，转换由排队网络中的特殊节点 **message broker** 处理：它充当应用级网关，主要目的是转换入站消息使目的应用能理解。注意对消息队列系统而言，broker 只是另一个应用（图 4.28），一般不算排队系统的内在部分。broker 可以简单到只是消息的重格式化器：如入站消息含一张数据库表（记录以特殊分隔符分隔、记录内字段定长），若目的应用期望不同的记录分隔符、且字段变

长, broker 即可把消息转成目的所需格式。更高级的设置中, broker 可充当应用级网关, 其中编码了多个应用的消息协议信息; 一般为每对应用各有一个能在两者间转换消息的子程序, 图 4.28 中该子程序画成 broker 的插件, 以强调这类部件可动态插入或移除。最后注意 broker 常用于高级企业应用集成 (EAI) (§1.3.2): 此时它不只 (或不只) 转换消息, 而是负责基于消息把应用匹配起来——在发布-订阅模型中, 应用以发布形式发消息 (如发布主题 X 的消息, 送达 broker), 声明对主题 X 感兴趣 (即订阅了这些消息) 的应用便从 broker 收到这些消息, 也可做更高级的中介。broker 的核心是一个“把一种消息变换成另一种”的规则库, 难点在于定义规则与开发插件; 多数 broker 产品带精巧的开发工具, 但归根结底规则库仍须由专家填充——商业产品常被误导性地称为提供“智能”, 其实智能只存在于那些专家头脑中; 或许更该一致地称其为“高级”系统 (可即便这样, 也常意味着复杂)。

原文精读

Message-queuing systems provide extensive support for persistent asynchronous communication.

消息队列系统为持久异步通信提供广泛支持。

The basic idea behind a message-queuing system is that applications communicate by inserting messages in specific queues.

消息队列系统的基本思想是: 应用通过把消息插入特定队列来进行通信。

A message broker acts as an application-level gateway in a message-queuing system.

消息代理 (message broker) 在消息队列系统中充当应用级网关。

原文定位: §4.3.3, 原书 p.220–227 (图 4.25 松耦合通信的四组合、图 4.26 队列基本接口、图 4.27 队列级命名与网络级寻址、图 4.28 消息队列系统中的 broker)

4.3.4 例: 高级消息队列协议 AMQP (Example: AMQP)

一个有意思的现象是: 消息队列系统本部分是为让遗留应用互操作而开发, 但在不同消息队列系统之间进行操作时却常撞墙; 结果一个组织一旦选了厂商 X 的消息队列系统, 就可能只能接受 X 才有的方案——消息队列方案很大程度是专有的, 谈何开放性。2006 年成立工作组以改变此局面, 促成了高级消息队列协议 (**Advanced Message-Queuing Protocol, AMQP**) 的规范。AMQP 有多个版本, 1.0 为最新; 也有多种实现, 尤其是 1.0 之前的版本——它们在 1.0 确立时已相当流行。由于 pre-1.0 与 1.0 差异很大却又各有稳定用户群, 我们可能见到 pre-1.0 服务器与 (无可否认不兼容的) 1.0 服务器并存。本节将有意混合 pre-1.0 与 1.0、抓住 AMQP 的要点与精神; 实现包括 RabbitMQ (Roy 2018) 与 Apache ActiveMQ。

基础: AMQP 围绕应用、queue manager 与队列。采用许多网络场景常见的做法, 区分作为消息服务的 AMQP、实际的消息协议、以及提供给应用的消息接口。为便于说明, 先考虑只有一个 queue manager、作为单个独立服务器运行, 构成 AMQP 服务的实现; 应用通过本地接口与该 manager 通信, 其间通信按 AMQP 协议进行 (图 4.29)。AMQP **stub** 把应用 (及 queue manager) 与消息传输和一般通信的细节隔开, 同时实现消息队列接口, 使应用能使用 AMQP 作为消息队列服务; 对 queue manager 而言 stub 与 manager 的区分是显式的, 但分离的严格程度交由实现决定——不过即便不严格, 概念上仍有“处理队列”与“处理相关通信”之别。

AMQP 通信: AMQP 允许应用与 queue manager 建立连接 (**connection**), 连接是若干单向信道 (**channel**) 的容器。信道的生命周期可以高度动态, 而连接被假定相对稳定; 连接与信道的这种区别便于高效实现, 尤其可用单条传输层 TCP 连接多路复用应用与 manager 间的许多不同信道。实践中 AMQP 假定用 TCP 建立 AMQP 连接。双向通信靠会话 (**session**) 建立: 会话是两条信道的逻辑分组; 一个连接可有多个会话, 但注意信道不必属于某个会话。最后, 真正传输消息需要 **link**: 概念上 link (更确切

说其端点)跟踪正在传输的消息状态,因而在应用与 queue manager 间提供细粒度流控,且可对不同消息在同一会话或连接上同时施加不同控制策略;流控靠信用 (**credit**) 建立——接收方可告诉发送方它被允许在特定 link 上发多少消息。传消息时应用把消息交给本地 AMQP stub,每次消息传输关联一个特定 link。消息传输通常分三步:(1)发送方给消息分配唯一标识符并在本地记为 **unsettled** 状态,stub 把消息传给服务器,服务器端的 AMQP stub 也记为 **unsettled**,随后交给 queue manager;(2)假定接收应用(此处即 queue manager)处理该消息并向其 stub 报告完成,stub 把此信息传给原发送方,原发送方的 AMQP stub 处消息进入 **settled** 状态;(3)原发送方的 AMQP stub 告知原接收方的 stub “传输已 settled”(即原发送方从此忘掉该消息),接收方 stub 也可丢弃有关该消息的一切、正式记为 **settled**。注意:由于接收应用可向底层 AMQP 通信层表明它已处理完消息,AMQP 因而能实现真正的端到端通信可靠性——应用(无论客户端应用还是真正的 queue manager)都可指示 AMQP 通信层继续持有消息(即消息保持 **unsettled** 状态)。

AMQP 消息 (messaging):逻辑上发生在处理通信的那一层之上,应用可在此表明需要对消息做什么、也可看到迄今发生了什么。消息在两个节点间传递,节点有三类:**producer**、**consumer**、**queue**;通常 producer 与 consumer 节点代表普通应用,queue 用于存储并转发消息,而 queue manager 通常由多个 queue 节点组成。要传输消息,两个节点须在它们之间建立 link。接收方可向发送方表明其消息是被接受(成功处理)还是拒绝,这意味着会向原发送方返回通知。AMQP 还支持大消息的分片与重组,为此会发送额外通知。当然,AMQP 的重要方面之一是对持久消息的支持,靠若干机制实现。第一,消息可标记为 **durable**,表示源方期望任何中间节点(如队列)在失败时能恢复;无法保证这种持久性的中间节点须拒绝该消息。第二,源或目标节点也可表明自身持久性:若持久,是只维护其状态,还是也维护 **durable** 消息的 (**unsettled**) 状态?把后者与 **durable** 消息结合,即可实现可靠的消息传输与持久消息。AMQP 确实是“消息协议”意义上的协议——它本身不支持例如发布-订阅原语,而是期望这类问题由更高级的专有 queue manager 处理(类似 §4.3.3 的消息代理)。最后,没理由不能让一个 queue manager 连到另一个 queue manager;事实上把 queue manager 组织成覆盖网络、让消息从 producer 路由到其 consumer 相当常见。AMQP 并不规定该覆盖网络应如何构建与管理,不同 AMQP 系统提供者给出不同方案;尤其重要的是规定消息应如何经网络路由,归根结底管理员需要手工完成大量此类规定,只有在覆盖网络结构规整(如环或树)时才较易给出必要路由细节。

实践中的 AMQP:较流行的消息中介实现之一是 **RabbitMQ** 服务器(Roy 2018 详述),它基于 AMQP 0.9 版、同时完全支持 1.0。两版的重要差别是 **exchange** 的使用:生产者不直接操作队列,而是联系 exchange,由它把消息放入一个或多个队列;本质上 exchange 让生产者能对 queue manager 做同步或异步 RPC,exchange 从不存储消息,只确保消息被放入合适的队列。图 4.30(a)的简单生产者即为例:先建立与服务器通信的手段(此处建立默认连接),随后在服务器范围内创建 exchange 与两个队列,再把两队列用同一个名为 **example-key** 的简单 key 绑定到同一 exchange;创建消息后,生产者告诉 exchange 按指定 key 把它发布到所有与之绑定的队列,于是 exchange 会把消息放入两个队列——图 4.30(b)的简单消费者从任一队列取消息并打印其内容,即可验证。exchange 与队列有许多选项;重要的是,用 RabbitMQ 等系统可一般性地搭建高级覆盖网络以实现应用级消息路由,但同时这些网络的维护很容易变成噩梦。

原文精读

AMQP allows an application to set up a connection to a queue manager; a connection is a container for a number of one-way channels.

because the receiving application can indicate to the underlying AMQP communication layer that it is done with a message, AMQP enables true

AMQP 允许应用与 queue manager 建立连接;连接是若干单向信道的容器。

由于接收应用可以向底层 AMQP 通信层表明它已处理完一条消息,AMQP 得以实现真正的端到端通信可靠性。

end-to-end communication reliability.

原文定位：§4.3.4，原书 p.227–232（图 4.29 单服务器 AMQP 实例、图 4.30 RabbitMQ 的生产者与消费者）

4.4 Multicast communication

组播通信

分布式系统通信中的一个重要主题是支持把数据发给多个接收者，即组播通信（**multicast communication**）。多年来该主题属于网络协议领域，涌现过许多网络层与传输层方案（Janic 2005；Obraczka 1998）。所有方案的一大问题是设置信息传播的通信路径，实践中涉及巨大的管理开销、常需人工干预；加上方案长期未收敛，ISP 对支持组播一直持保留态度（Diot 等 2000）。随着对等（**P2P**）技术尤其结构化覆盖网络管理的出现，建立通信路径变得更容易；由于 P2P 方案通常部署在应用层，便涌现出多种应用层组播技术。组播也可用设置显式通信路径之外的方式实现——本节还将探讨的基于 **gossip** 的信息传播就提供了简单（但常常效率较低）的组播方式。

4.4.1 应用层基于树的组播（Application-level tree-based multicasting）

应用层组播的基本思想是把节点组织成一个覆盖网络，用它把信息传播给其成员。一个重要观察是网络路由器不参与组成员管理：因此覆盖网络中节点间的连接可能跨越若干物理链路，在覆盖网络内路由消息相较网络层路由可能并非最优。一个关键设计问题是覆盖网络的构造，本质上两种思路（El-Sayed 等 2003；Hosseini 等 2007；Allani 等 2009）：其一，节点直接组织成树，即每对节点间有唯一的（覆盖层）路径；其二，节点组织成网格（**mesh**），每个节点有多个邻居、每对节点间一般存在多条路径。两者主要差别是后者通常健壮性更好：若某连接断开（如某节点失败），仍有机会传播信息、无需立即重组整个网络。

Note 4.13: 在 Chord 中构造组播树

一个较简单的方案是在 Chord（见 Note 2.6）中构造组播树，最初为 Scribe 提出（Castro 等 2002b），Scribe 是建于 Pastry（Rowstron & Druschel 2001）之上的应用层组播方案，后者也是基于 DHT 的 P2P 系统。设某节点想发起一个组播会话：它生成一个组播标识符（如随机选一个 160 位 key）**mid**，然后查找 **succ(mid)**——即负责该 key 的节点，并提升它为这棵组播树的根。要加入树，节点 P 只需执行 **lookup(mid)**，使一个“请求加入组播组 mid”的查找消息从 P 路由到 **succ(mid)**。沿途会经过若干节点：设先到达 Q，若 Q 此前从未见过针对 mid 的加入请求，它就成为该组的转发者（**forwarder**），此时 P 成为 Q 的子节点，Q 继续把加入请求向根转发；若下一个节点 R 也还不是转发者，它也成为转发者、记下 Q 为其子节点并继续向根发送加入请求。反之，若 Q（或 R）已经是 mid 的转发者，它也会把前一个发送者（分别是 P 或 Q）记为自己的子节点，但无需再把加入请求发往根，因为 Q（或 R）已经是组播树的成员。像 P 这样显式请求加入者按定义也是转发者。于是我们在覆盖网络上构造出一棵组播树，含两类节点：只做帮助者的纯转发者，以及同时也是转发者、但显式请求加入的节点。此时组播很简单：节点只需再次执行 **lookup(mid)** 把组播消息发往树根，消息随后即可沿树发送。

覆盖网络的性能问题。由上述高层描述可见：一旦把节点组织进覆盖网络，建树本身并不难，但建一棵高效的树可能另当别论——我们的描述中，参与树的节点选择 不考虑任何性能指标，纯基于消息经覆盖网络的（逻辑）路由。取图 4.31 的五节点小覆盖网络（节点 A 为组播树根，并标出各物理链路的遍历代价）为例：A 向其他节点组播时，覆盖层的逻辑路由是 $A \rightarrow B \rightarrow E \rightarrow D \rightarrow C$ ，实际路由按网络层链路为 $A \rightarrow R_a \rightarrow R_b \rightarrow B \rightarrow R_b \rightarrow R_a \rightarrow R_e \rightarrow E \rightarrow R_e \rightarrow R_c \rightarrow R_d \rightarrow D \rightarrow R_d \rightarrow R_c \rightarrow C$ 。

可见该消息会把链路 $\langle B, R_b \rangle$ 、 $\langle R_a, R_b \rangle$ 、 $\langle E, R_e \rangle$ 、 $\langle R_c, R_d \rangle$ 、 $\langle D, R_d \rangle$ 各遍历两次。若我们不构造覆盖链路 $\langle B, E \rangle$ 与 $\langle D, E \rangle$ ，而改构造 $\langle A, E \rangle$ 与 $\langle C, E \rangle$ ，覆盖网络会更高效，可省去对物理链路 $\langle R_a, R_b \rangle$ 与 $\langle R_c, R_d \rangle$ 的双重遍历。应用层组播树的质量一般用三个指标衡量：**link stress**、**stretch** 与 **tree cost**。**link stress** 按链路定义，统计一个包跨过同一条链路的次数：link stress 大于 1 源于“逻辑上一个包可能沿两条不同连接转发，而其中部分连接实际对应同一物理链路”（如图 4.31 所示）。**stretch**（或称相对延迟惩罚 RDP）度量“覆盖网络中两节点间的延迟”与“二者在底层网络中会经历的延迟”之比：例如 B 到 C 在覆盖网络走 $B \rightarrow R_b \rightarrow R_a \rightarrow R_e \rightarrow E \rightarrow R_e \rightarrow R_c \rightarrow R_d \rightarrow D \rightarrow R_d \rightarrow R_c \rightarrow C$ 、总代价 73 单位，而在底层网络本应走 $B \rightarrow R_b \rightarrow R_d \rightarrow R_c \rightarrow C$ 、总代价 47 单位，stretch 为 1.55；显然构造覆盖网络的目标是最小化聚合 stretch（或对全部节点对求平均 RDP）。最后，**tree cost** 是全局指标，一般关乎最小化聚合链路代价；若把链路代价取为两端节点间延迟，则优化 tree cost 即求最小生成树，使把信息传播到所有节点的总时间最小。为略作简化，假定组播组有一个知名节点跟踪已加入树的节点；新节点发加入请求时联系该 **rendezvous node**，获得（可能部分的）成员列表，目标是从中选出能作为其父节点的最佳成员。该选谁？备选很多，不同方案常常解法迥异。例如只有单一源的组播组，最佳节点显而易见应是源本身（此时可保证 stretch 为 1）；但这样做会形成以源为中心的星形拓扑，源很容易过载。换言之，节点选择一般会受约束：只能选邻居数不超过 k 的节点（ k 是设计参数），这一约束严重复杂化建树算法——合适的解可能需要重配置部分既有树（Tan 等 2003 给出广泛综述与评估）。

原文精读

The basic idea in application-level multicasting is that nodes are organized into an overlay network, which is then used to disseminate information to its members.

应用层组播的基本思想是：把节点组织成一个覆盖网络，再用它把信息传播给其成员。

Link stress is defined per link and counts how often a packet crosses the same link

link stress 按链路定义，统计一个包跨过同一条链路的次数。

原文定位：§4.4.1，原书 p.232–236（图 4.31 覆盖链路和网络级路由的关系）

4.4.2 基于泛洪的组播（Flooding-based multicasting）

此前我们假定组播消息要被覆盖网络中每个节点收到——严格说这对应广播（broadcasting）。一般地，组播指把消息发给全体节点中的一个子集，即某个特定节点组。组播的一个关键设计问题是尽量减少使用那些并非消息目标的中间节点：若覆盖网络组织成多级树，却只有叶节点才应收到组播消息，那显然会有相当多节点需要存储并转发并非发给它们的消息。避免这种低效的一个简单办法是为每个组播组各构造一个覆盖网络，于是把消息 m 组播给组 G 就等同于把 m 广播给 G ；缺点是属于多个组的节点原则上需为所属每个组各维护一份邻居列表。若假定覆盖网络对应一个组播组因而需要广播，最朴素的做法是泛洪（flooding）：每个节点把消息 m 转发给每个邻居（除了它收到 m 的那个邻居）；此外，若节点记录它收到并转发过的消息，就可直接忽略重复。要理解泛洪的性能，把覆盖网络建模为连通无向图 $G = (V, E)$ ，设 v_0 发起泛洪：原则上除 v_0 外每个节点都会向邻居发一次消息（除了它收到消息的那个邻居），于是总共会有 $\delta(v_0) + \sum_{v \in V - v_0} (\delta(v) - 1)$ 条消息，其中 $\delta(v)$ 是节点 v 的邻居数。对任何无向图 $\sum_{v \in V} \delta(v) = 2|E|$ ，故会发出 $2|E| - |V| + 1$ 条消息，使泛洪相当低效；只有当 G 是树时泛洪才最优（此时 $|E| = |V| - 1$ ）；最坏情形 G 完全连通、 $|E| = \binom{|V|}{2}$ ，导致 $|V|^2$ 量级的消息。

现在假定对覆盖网络结构一无所知、最多只能把它表示成随机图（简化为任意两顶点以概率 p_{edge} 相连，即 Erdős-Rényi 图，Erdős & Rényi 1959）。注意这等于把覆盖网络看作非结构化 P2P 网络、且对其构建方式无任何信息。以概率 p_{edge} 相连、共 $\binom{|V|}{2}$ 条边，不难看出可期望覆盖网络有 $|E| = \frac{1}{2} \cdot p_{\text{edge}} \cdot |V| \cdot (|V| - 1)$ 条边；图 4.32 给出不同 p_{edge} 下节点数与边数的关系——即便 p_{edge} 很小，边数也很容易变得很大。要

减少消息数，可用 Banaei-Kashani 与 Shahab (2003) 提出、Oikonomou 与 Stavrakakis (2007) 形式化分析的概率泛洪 (**probabilistic flooding**)：想法很简单——节点泛洪消息 m 需转发给某特定邻居时，以概率 p_{flood} 转发。效果可能很显著：发送的消息总数随 p_{flood} 线性下降。但也有风险： p_{flood} 越低，网络中并非所有节点都能被到达的可能性越大——原因很简单，某节点 Q 的所有邻居可能都决定不把 m 转发给 Q ；若 Q 有 n 个邻居，这大约以概率 $(1 - p_{\text{flood}})^n$ 发生。显然邻居数在决定是否转发消息时很重要，确实可把静态的转发概率换成考虑邻居度数的概率 (Serenio 与 Gaeta 2011 进一步发展与分析)。给个效率印象：在 10,000 节点、 $p_{\text{edge}} = 0.1$ 的随机网络中，只需令 $p_{\text{flood}} = 0.01$ 就能相对完全泛洪取得 50 倍以上的消息量削减。

面对结构化覆盖网络 (拓扑大体确定) 时，设计高效泛洪方案更简单。以 n 维超立方体为例 (图 4.33 为 $n = 4$ ；亦见 §2.4.1)。Schlosser 等 (2002) 设计了一个简单高效的广播方案，其依据是按维度跟踪邻居：可把 n 维超立方体中每个节点表示为长度 n 的比特串，每条边按其维度标记。 $n = 4$ 时，节点 0000 的邻居为 {0001, 0010, 0100, 1000}；边 {0000, 0001} 标为“4” (比较 0000 与 0001 时改变第 4 位)，边 {0000, 0100} 标为“2”，依此类推。节点最初把消息 m 广播给所有邻居，并给 m 打上“发送该消息所经边”的标签。若节点 1001 广播，它会发送：($m, 1$) 给 0001、($m, 2$) 给 1101、($m, 3$) 给 1011、($m, 4$) 给 1000。当节点收到广播消息时，只沿维度更高的边转发：如节点 1101 只把 m 转发给 1111 (与 1101 由标“3”的边相连) 与 1100 (由标“4”的边相连)。用此方案可证明每次广播恰需 $N - 1$ 条消息 ($N = 2^n$ 为 n 维超立方体的节点数)，因而在发送消息数意义下最优。

原文精读

In this case, each node simply forwards a message m to each of its neighbors, except to the one from which it received m .

在这种情况下，每个节点只是把消息 m 转发给它的每个邻居，除了它从中收到 m 的那个邻居。

The idea is simple: when a node is flooding a message m and needs to forward m to a specific neighbor, it will do so with a probability p_{flood} .

想法很简单：当一个节点在泛洪消息 m 、需要把 m 转发给某个特定邻居时，它会以概率 p_{flood} 转发。

原文定位：§4.4.2，原书 p.236–240 (图 4.32 随机覆盖网络的规模、图 4.33 四维超立方体、图 4.34 Chord 环中的广播)

4.4.3 基于 gossip 的数据传播 (Gossip-based data dissemination)

一种重要的信息传播技术是依赖流行病 (**epidemic**) 行为，也称 **gossiping**。研究者观察疾病如何在人群中传播，早就探讨能否用简单技术在超大规模分布式系统中传播信息。这些流行病协议的主要目标是仅用局部信息就在大量节点间迅速传播信息——换言之，没有中心组件来协调信息传播。为说明这些算法的一般原理，假定对某个数据项的所有更新都在单一节点发起，从而简单避开写-写冲突 (以下基于 Demers 等 1987 关于流行病算法的经典论文；综述见 Eugster 等 2004)。

信息传播模型：顾名思义，流行病算法基于研究传染病传播的流行病理论。在大规模分布式系统中，它们传播的是信息而非疾病；针对分布式系统的流行病研究还有完全不同的目标：卫生组织竭力阻止传染病在大群体中扩散，而分布式系统流行病算法的设计者则要尽快“感染”所有节点。借用流行病术语：持有数据并愿意传播的节点称感染 (**infected**)；尚未见过该数据的称易感 (**susceptible**)；已被更新但不愿或不能传播数据的称移除 (**removed**) (假定能区分新旧数据，如打了时间戳或版本号；此语境下节点也称传播更新 (**updates**))。一种流行的传播模型是 **anti-entropy**：节点 P 随机选另一节点 Q 并与 Q 交换更新，交换方式有三种：(1) P 只从 Q 拉取 (**pull**) 新更新；(2) P 只把自己的更新推 (**push**) 给 Q ；(3) P 与 Q 互相发送更新 (即 **push-pull**)。就快速传播而言，只推是糟糕选择：纯推方式下更新只能由已感染节点传播，而当很多节点已感染时，每个节点选中易感节点的概率很小，故某节点可能

因没有被感染节点选中而长期保持易感。相反，拉方式在多数节点已感染时表现好得多：此时传播本质上由易感节点触发，它很可能联系到一个已感染节点、拉取更新从而也被感染。若只有单个节点感染，两种 anti-entropy 都能让更新迅速扩散到所有节点，但 push-pull 仍是最佳策略（Jelasity 等 2007）。定义一轮（**round**）为一个时期、期间每个节点都会主动一次与随机选中的另一节点交换更新；则可证明把单个更新传给所有节点所需轮数为 $O(\log N)$ 量级（ N 为节点数）——这确实表明传播更新很快，更重要的是可扩展。

流行病协议的一个具体变体叫 **rumor spreading**（谣言传播）：若节点 P 刚被某数据项 x 更新，它就联系任意的另一节点 Q 、尝试把更新推给 Q ；但 Q 可能已被另一节点更新过，此时 P 可能就此失去继续传播的兴趣（比如以概率 p_{stop} 停止），换言之它变成了已移除。谣言传播就是我们日常生活中最常经历的 gossip：Bob 有劲爆消息，打电话告诉朋友 Alice；Alice 像 Bob 一样热衷把谣言传给她的朋友；可当她打给朋友 Chuck，却听说消息已传到他那里时，她很可能会就此不再给其他朋友打电话——反正他们知道了，还有什么用？谣言传播是迅速散布消息的绝佳方式，但不能保证所有节点都被更新（Demers 等 1987）：当参与流行的节点很多时，仍不知道某更新的节点比例（即仍易感者的比例） s 满足 $s = e^{-(1/p_{\text{stop}}+1)(1-s)}$ 。为有个直观印象（图 4.36）：即便 p_{stop} 很高，仍不知情的节点比例也相对低、总不超过约 0.2；当 $p_{\text{stop}} = 0.20$ 时可证明 $s = 0.0025$ 。但在 p_{stop} 较高时，仍须采取额外措施以确保所有节点都被更新。

流行病算法的一大优点是可扩展：与其他传播方法相比，进程间同步次数相对少。对广域系统，Lin 与 Marzullo（1999）表明考虑实际网络拓扑可获得更好结果：此时只与少数节点相连的节点以相对高的概率被联系，其潜在假设是这类节点构成通往网络其他远程部分的桥，因而应尽快联系它们；此法称定向 **gossip (directional gossiping)**，有不同变体。这触及多数流行病方案的一个重要假设：节点能随机选择任意其他节点 gossip，这意味着原则上每个成员都应知道全部节点集合。在大系统中这一假设永远不成立，须采取特殊措施来模拟这种性质（§5.5.2 讨论 peer-sampling service 时再谈）。

删除数据：流行病算法传播更新极佳，却有个奇怪的副作用——传播数据项的删除很难。问题本质在于删除数据项会摧毁关于该项的一切信息：当数据项被某节点简单移除后，该节点最终会收到该数据的旧副本，并把它们当作“自己此前没有的新东西”的更新来理解。技巧是把数据项的删除记录成又一次更新、并保留该删除记录；这样旧副本就不会被当成新东西，而只被当作“已被一次删除操作更新过的版本”。删除的记录通过传播 **death certificate**（死亡证书）完成。当然，death certificate 的问题是最终须被清理，否则每个节点会逐渐攒出一个巨大的历史信息本地库、其中是已删除数据项的记录、别无他用。Demers 等（1987）提出使用 **dormant death certificate**（休眠死亡证书）：每份 death certificate 创建时打时间戳；若可假定更新在已知有限时间内传播到所有节点，则超过该最大传播时间后即可移除 death certificate。但若要为“删除确实传播到了所有节点”提供硬保证，只有极少数节点保留永不丢弃的 dormant death certificate：设节点 P 持有数据项 x 的这样一份证书，若某个针对 x 的过时更新碰巧到达 P ， P 只需再次传播 x 的 death certificate 即可。

原文精读

The main goal of these epidemic protocols is to rapidly propagate information among a large collection of nodes using only local information.

这些流行病协议的主要目标，是仅利用局部信息，就在大量节点之间迅速传播信息。

The trick is to record the deletion of a data item as just another update, and keep a record of that deletion.

窍门在于把数据项的删除记录为又一次更新，并保留该删除的记录。

One specific variant of epidemic protocols is called rumor spreading.

流行病协议的一个具体变体叫做谣言传播（rumor spreading）。

原文定位：§4.4.3，原书 p.240–244（图 4.35 anti-entropy 未更新节点数随轮数变化、图 4.36 未更新比例 s 与 p_{stop} 的关系）

4.5 Summary

小结

- 进程间通信是分布式系统的核心；中间件的重要任务是在传输层接口之上提供更高的抽象层次，让进程间通信更易表达。
- **RPC** 是最广泛使用的抽象之一：服务由过程实现、过程体在服务器执行，客户端只拿到过程的签名；调用时 client stub 把参数值打包成消息发出，server stub 调用实际过程并回送结果。
- RPC 提供同步通信（客户端阻塞到收到应答），虽有各种放松严格同步的变体，但通用、高层的面向消息模型往往更方便。
- 面向消息模型的要点是通信是否持久、是否同步：持久通信中消息由通信系统存到送达为止，收发双方都不必在线；瞬态通信则不提供存储，接收方须在消息发出时准备好接收。
- 消息队列中间件一般提供持久异步通信，常用于把（广泛分布的）数据库集合集成成为大规模信息系统。
- 组播是把信息从单一发送者传播给多个接收者：既可通过建树（如今也可用 P2P 自组织去中心化动态建树）实现；泛洪极其健壮但要小心资源浪费（概率泛洪兼顾简单与高效）；以及流行病/gossip 协议，简单且极其健壮。

原文精读

RPCs offer synchronous communication facilities, by which a client is blocked until the server has sent a reply.

RPC 提供同步通信设施，客户端会被阻塞，直到服务器发回应答。

The essence of persistent communication is that a message that is submitted for transmission, is stored by the communication system as long as it takes to deliver it.

持久通信的本质是：已提交待传的消息会由通信系统一直存储，直到将其投递为止。

原文定位：§4.5，原书 p.245–246

实践清单

- 读原书第 4 章，重点看图 4.1 (OSI 分层)、图 4.4 (中间件作为中介服务)、图 4.6 (RPC 原理)、图 4.18 (socket 面向连接模式)、图 4.28 (message broker)、图 4.31 (覆盖与物理路由)、图 4.36 (rumor spreading 的 s)。
- 用 Python 手写一对 socket 客户端/服务器 (图 4.19)，再用 pickle 加编组 (图 4.9)，体会 RPC 两步与字节序问题。
- 用 ZeroMQ 分别实现 REP/REQ 与 PUB/SUB 两个小程序，验证“异步 + 面向连接”使客户端可先于服务器启动、且未订阅的消息收不到。
- 给一个小型消息队列选型 (RabbitMQ / Kafka / ActiveMQ)，画出 producer → exchange → queue → consumer 的路径，并说明哪些消息需要 durable。
- 对比 RPC 与 MOM：把你的一个真实业务场景分别按两者建模，指出何时必须用持久异步。
- 在 5–10 个节点的模拟覆盖网络上实现泛洪与概率泛洪，统计消息数随 p_{flood} 的变化；再实现 anti-entropy 的 push、pull、push-pull，对比传播到全体所需轮数。

Coordination

本章讲进程之间如何协调与同步。先处理“真实时间”：时钟同步（UTC、NTP、RBS）；再转向只需相对顺序的逻辑时钟（Lamport、向量时钟）；接着是分布式互斥的四类算法与选举算法（bully、ring、ZooKeeper、Raft、PoW/PoS）；然后是基于 *gossip* 的协调（聚合、peer sampling、覆盖网构建、安全）、发布-订阅中的分布式事件匹配，最后是用坐标刻画邻近性的定位系统。

5.1 Clock synchronization

时钟同步

集中式系统里时间是无歧义的：进程向 OS 要时间即可，后问的一定不小于先问的。分布式系统则没有全局共享时钟。作者用 Unix make 说明后果：若 `output.c` 的修改时间因机器时钟偏慢而被标成早于 `output.o`，make 就不会重新编译，可执行文件便混入新旧目标码。因此本章从最基础的问题问起：能否同步分布式系统中的所有时钟？

5.1.1 物理时钟（Physical clocks）

计算机里所谓的“clock”其实更像 **timer**：石英晶振按固定频率振荡，每振荡一次计数器减一，减到零产生中断（一次 **clock tick**）并从保持寄存器重装。单机时钟即使有偏差也无妨，因为所有进程共享同一时钟、彼此一致；但多台机器各有晶振时，频率无法完全一致，软件时钟会逐渐不同步，这种时间值之差叫 **clock skew**。实时系统需要外部物理时钟，于是产生两个问题：如何与真实世界时钟对齐、以及如何彼此对齐。

UTC 是全世界民用计时的基准：从太阳时（mean solar second）到 1948 年原子钟（铯 133 的 9,192,631,770 次跃迁定义 1 秒），再到 TAI；因地球自转变慢，BIH 在 TAI 与太阳时相差达 800 msec 时插入 **leap second**，得到 **Coordinated Universal Time (UTC)**。UTC 可由短波电台（±10 msec）、地球同步卫星（0.5 msec）乃至多星组合的地面 timeserver（50 ns）提供。

5.1.2 时钟同步算法（Clock synchronization algorithms）

算法目标用两个界刻画：**precision**（内部同步，任意两机软件时钟偏差 $|C_p(t) - C_q(t)| \leq \pi$ ）与 **accuracy**（外部同步，相对 UTC 偏差 $|C_p(t) - t| \leq \alpha$ ）。一组 accuracy 为 α 的时钟必然 precision 为 $\pi = 2\alpha$ ，反之不成立。硬件时钟有 **clock drift rate**（典型石英约 10^{-6} 秒/秒，即每年约 31.5 秒）。若两钟反向漂移，同步后经 Δt 可相差 $2\rho \cdot \Delta t$ ，故要保证 precision π ，至少每 $\pi/(2\rho)$ 秒重新同步。

Network Time Protocol (NTP) 源自 Cristian 的方法：客户端联系 timeserver，用四个时间戳 T_1 – T_4 估算 offset $\theta = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$ 与 delay $\delta = \frac{(T_4 - T_1) - (T_3 - T_2)}{2}$ ，缓存八对 (θ, δ) 取 δ 最小者。时间不能倒流，调整须渐进（如每次中断只加 9 msec 或 11 msec）。NTP 按 **stratum** 分级：stratum-1 服务器接 UTC 接收机或原子钟，只向更低层级同步。

无线传感网不适用上述假设（资源受限、多跳昂贵、要省电），于是有 **Reference Broadcast Synchronization (RBS)**：只让接收方同步、把发送方排除在外，从而消除“构造消息”和“访问信道”两项不确定延迟，只留传播时间；用线性回归把 offset 拟合成函数 $\text{Offset}[p, q](t) = \alpha t + \beta$ ，既更准又不必调整本地时钟（省电）。

原文精读

In a centralized system, time is unambiguous.

在集中式系统中，时间是无歧义的。

This difference in time values is called clock skew.

这种时间值上的差异被称为时钟偏移（clock skew）。

The whole idea of clock synchronization is that we keep clocks precise, referred to as internal synchronization or accurate, known as external synchronization.

时钟同步的核心思想是：让时钟保持精确（precise），称为内部同步；或保持准确（accurate），称为外部同步。

原文精读

The problem, of course, is that when contacting the server, message delays will have outdated the reported time.

问题当然在于：联系服务器的过程中，消息延迟会使报告的时间变得过时。

Of course, time is not allowed to run backward.

当然，时间不允许倒流。

Reference broadcast synchronization (RBS) is a clock synchronization protocol that is quite different from other proposals.

参考广播同步（RBS）是一种与其他方案相当不同的时钟同步协议。

Note 5.2: 精确计时的价值与 TrueTime

若能给事件打上无限精度的时间戳，生活会简单得多。Google 的 **Spanner** 为此实现 **TrueTime**，提供 `TT.now()`（返回区间 $[T_{lwb}, T_{upb}]$ ）、`TT.after(t)`、`TT.before(t)`，并保证区间是真界； $\epsilon = T_{upb} - T_{lwb}$ 仅约 6 ms。它靠每个数据中心的 time-master（部分配 GPS 接收机、部分配原子钟，彼此高度独立以容错）与 time-slave 查询多个 master 实现，并用 Marzullo-Owicki 算法剔除离群值。有了 6 ms 保证，跨服务器给事务打时间戳成为可能，代价是读取结果可能需等 ϵ 时间。

原文定位：§5.1，原书 p.249–260（图 5.1 时钟偏差、图 5.5 从 timeserver 取时、图 5.6 RBS 关键路径）

5.2 Logical clocks

逻辑时钟

精确的真实时间往往并非必需：只要能对事件顺序达成一致即可。对 **make** 而言，只需两节点都同意 `input.o` 已被新 `input.c` 取代。Lamport (1978) 指出：若两进程不交互，它们的时钟无须同步；真正重要的是事件发生的次序。这类时钟统称 **logical clocks**。

5.2.1 Lamport 逻辑时钟 (Lamport's logical clocks)

Lamport 定义 **happens-before** 关系 $a \rightarrow b$ (所有进程都同意 a 先于 b)。它可直接观察到两种情形：同进程内 a 先于 b ；以及 a 是发消息、 b 是收该消息。**happens-before** 是传递的；若两事件所在进程从不交换消息，则 $x \rightarrow y$ 与 $y \rightarrow x$ 都不成立，称二者 **concurrent**。所需的是一个时间函数 C ，满足：若 $a \rightarrow b$ 则 $C(a) < C(b)$ ，且 C 只增不减（修正只能加正值）。

实现只需每进程 P_i 维护本地计数器 C_i ：(1) 执行事件前 $C_i \leftarrow C_i + 1$ ；(2) 发消息时令时间戳 $ts(m) = C_i$ ；(3) 收到消息时先 $C_j \leftarrow \max\{C_j, ts(m)\}$ ，再执行第 1 步并投递。若要不出出现相同时间，可用 $\langle \text{时间}, \text{进程号} \rangle$ 打破平局，如 $\langle 40, i \rangle < \langle 40, j \rangle$ (当 $i < j$)。

应用：全序多播 (**totally ordered multicasting**)。以跨纽约、旧金山复制的银行账户为例：一笔“存 100 美元”与一笔“加 1% 利息”若在两副本上以不同顺序执行，会得到 \$1,111 与 \$1,110 的不一致结果。做法是每条消息带发送者逻辑时间戳、放入本地队列、并向其他进程多播 ACK；仅当消息位于队首且已被所有进程确认时才投递。由于各进程队列副本相同，所有消息处处以同一顺序投递，即 **state machine replication**。

原文精读

In a seminal paper, Lamport [1978] showed that although clock synchronization is possible, it need not be absolute.

If two events, x and y , happen in different processes that do not exchange messages (not even indirectly via third parties), then $x \rightarrow y$ is not true, but neither is $y \rightarrow x$.

These time values must have the property that if $a \rightarrow b$, then $C(a) < C(b)$.

在一篇开创性论文中，Lamport [1978] 指出：尽管时钟同步是可能的，但它不必是绝对的。

如果两个事件 x 和 y 发生在不交换消息（甚至不通过第三方间接交换）的不同进程中，那么 $x \rightarrow y$ 不成立， $y \rightarrow x$ 也不成立。

这些时间值必须具有这样的性质：若 $a \rightarrow b$ ，则 $C(a) < C(b)$ 。

5.2.2 向量时钟 (Vector clocks)

Lamport 时钟的局限是不捕获因果：由 $C(a) < C(b)$ 不能推出 a 确实先于 b 。**vector clocks** 弥补这一点：每进程 P_i 维护向量 VC_i ， $VC_i[i]$ 是 P_i 本地已发生的事件数， $VC_i[j]$ 是 P_i 所知的 P_j 的本地时间。更新规则与 Lamport 类似，但收消息时对每个 k 取 $VC_j[k] \leftarrow \max\{VC_j[k], ts(m)[k]\}$ （相当于合并因果历史）。定义 $ts(a) < ts(b)$ 当且仅当对所有 k 有 $ts(a)[k] \leq ts(b)[k]$ 且至少一个分量严格更小；据此可判断 m_4 是否可能因果依赖 m_2 ，或二者可能冲突。

原文精读

The problem is that Lamport clocks do not capture causality.

In practice, causality is captured by means of vector clocks.

问题在于 Lamport 时钟并不捕获因果关系。

在实践中，因果关系是通过向量时钟来捕获的。

解读 L

Lamport 时钟与向量时钟的分工要记牢：前者给全序，用于全序多播、互斥；后者给偏序，用于判断“是否可能因果相关”。Note 5.4 进一步用向量时钟实现因果有序多播：仅当 $ts(m)[i] = VC_j[i] + 1$ 且对所

有 $k \neq i$ 有 $ts(m)[k] \leq VC_j[k]$ 时才投递消息 m ，从而保证“可能因果先于它的消息都已收到”。Note 5.4 也讨论了经典的争议：把排序交给中间件还是应用，涉及 **end-to-end principle**。

原文定位：§5.2，原书 p.260–272（图 5.7 Lamport 算法修正时钟、图 5.12 向量时钟捕获因果、图 5.13 强制因果通信）

5.3 Mutual exclusion

互斥

多进程并发访问共享资源时必须保证互斥，否则资源可能被破坏或不一致。分布式互斥算法分两大类：**token-based**（传递唯一 token，持有者可访问；优点是易保证无饥饿、无死锁，缺点是 token 一旦丢失（如持有者崩溃）需复杂过程重建且必须保证唯一）与 **permission-based**（先取得其他进程许可）。

5.3.1 集中式算法 (A centralized algorithm)

选一个协调者 (coordinator)：进程请求资源，无冲突则立即授予；有冲突则挂起请求排队。每次使用资源仅需 3 条消息 (request、grant、release)，简单、公平、无饥饿。缺点是协调者单点失败（崩溃则整个系统可能瘫痪，且进程无法区分“协调者死了”与“被拒绝”），大规模下还可能是性能瓶颈。但作者指出其简单性的收益常胜过缺点，且分布式方案未必更好。

5.3.2 分布式算法 (A distributed algorithm)

Ricart–Agrawala (1981)：基于 Lamport 全序。请求者向所有进程（含自己）发 (资源名, 进程号, 逻辑时间)；收方三种处理：不感兴趣则回 OK；已持有则不回、排队；也想要则比较时间戳，小者胜。共需 $2(N-1)$ 条消息 ($N-1$ 请求 + $N-1$ OK)，无死锁无饥饿，但有 N 个失败点（任一进程崩溃都会阻塞全体），且最好用于成员固定的小组；可改为只需多数许可。

5.3.3 token-ring 算法 (A token-ring algorithm)

构造逻辑环，token 沿环点对点传递，持有者需要则访问、不需要则传下去。正确性直观：任一时刻仅一人持有 token，环序明确故无饥饿。问题是 token 丢失难检测（token 再现间隔无上界，一小时没见到不代表丢了），但进程崩溃恢复较容易（收到 token 需确认，邻居可发现死进程并跳过它）。

5.3.4 去中心化算法 (A decentralized algorithm)

Lin 等 (2004) 投票法：资源复制 N 份，每份有协调者；访问需取得 $m > N/2$ 个协调者的多数票。假设协调者崩溃后快速恢复但忘记此前投过的票（相当于任意时刻重置），这可能错误地再次授权。设重置概率 p ， k 个协调者重置的概率 $P[k] = \binom{m}{k} p^k (1-p)^{m-k}$ ；当 $f \geq 2m - N$ 时正确性被破坏，其概率 $\sum_{k=2m-N}^m P[k]$ 。图 5.17 表明该概率可低到可忽略。缺点：许多节点争抢同一资源时利用率骤降，可能谁都凑不够票。

Note 5.5: 四种互斥算法对比

算法	每次进出的消息数	进入前延迟 (MTTU)
集中式	3	2
分布式	$2(N - 1)$	$2(N - 1)$
token ring	$1, \dots, \infty$	$0, \dots, N - 1$
去中心化	$2kN + (k - 1)N/2 + N$	$2kN + (k - 1)N/2$

几乎所有这些算法在崩溃面前都很脆弱，需要额外复杂度来避免“一崩俱崩”。颇具讽刺的是：分布式算法通常比集中式更易受崩溃影响。因此集中式互斥被广泛使用并不奇怪——行为易理解，且提高集中服务器容错相对容易。

5.3.5 例：用 ZooKeeper 实现简单锁

ZooKeeper 为分布式协调提供支持（锁、选举、监控等），设计原则是没有阻塞原语：客户端发消息后立即得到响应。它维护一棵树状 namespace，节点分 **persistent**（显式创建删除）与 **ephemeral**（连接关闭或过期即自动删除）。不支持阻塞，就需要通知机制：客户端订阅某节点/分支的变化，变化时收到消息。为避免基于过期信息更新，用版本号： $W(n, k)a$ 表示“假设节点 n 当前版本为 k 时写入 a ”，版本不符则写失败，客户端需重读后再决定。实现锁很直接：创建 `/lock` 成功即获得锁，删除即释放；创建失败者订阅 `/lock` 的删除通知、本地阻塞，收到通知后重试。需处理细节：允许多次尝试后放弃；持有者崩溃时确保节点被删除；以及“订阅前锁已被释放”的竞态。

原文精读

Fundamental to distributed systems is the concurrency and collaboration among multiple processes.

分布式系统的根本在于多进程之间的并发与协作。

In token-based solutions mutual exclusion is achieved by passing a special message between the processes, known as a token.

在基于令牌的方案中，互斥是通过在进程间传递一条称为令牌（token）的特殊消息来实现的。

It is easy to see that the algorithm guarantees mutual exclusion: the coordinator lets only one process at a time access the resource.

很容易看出该算法保证了互斥：协调者一次只允许一个进程访问资源。

原文定位：§5.3，原书 p.272–283（图 5.14 集中式、图 5.15 分布式、图 5.16 token ring、图 5.18 算法对比、图 5.19 ZooKeeper 版本控制）

5.4 Election algorithms**选举算法**

许多分布式算法需要某进程充当 coordinator/initiator。若所有进程完全相同便无法选出，故假设每进程有唯一标识 $id(P)$ ，选举算法通常找出标识最大者并指定为协调者；还假设每进程知道所有其他进程的标识，但不知道谁在线。目标是选举开始后，所有进程就“新协调者是谁”达成一致。

5.4.1 bully 算法 (The bully algorithm)

Garcia-Molina (1982): 进程 P_k 发现协调者无响应即发起选举, 向所有标识更高的进程发 ELECTION; 无人应答则自己当选; 有人应答则由对方接手。收到 ELECTION 者回 OK 并 (若未在选举中) 自行发起选举。最终只剩一个胜者并广播 COORDINATOR。此前宕机者恢复后若标识最高便重新当选, 故“城里块头最大的总是赢”, 得名 bully。

5.4.2 环算法 (A ring algorithm)

进程知道自己的后继。发现协调者失效者构造含自身标识的 ELECTION 沿环传递, 每步加入自己的标识 (使自己成为候选); 消息绕回发起者时改为 COORDINATOR 再绕一圈, 通告新协调者 (标识最高者) 与新环成员。多个选举可并行, 各自绕环, 最多多耗些带宽, 无害。

5.4.3 例: ZooKeeper 中的领导者选举

ZooKeeper 用一小组服务器组成 **ensemble**, 对客户端看似单服务器, 内部由 **leader** 协调, 其余为 **follower** (最新状态的备份)。默认选举是简化的 bully: 每台服务器有标识 $id(s)$ 与最新事务计数器 $tx(s)$; 怀疑 leader 出问题广播 $(voteID, voteTX)$, 并维护 $leader(s)$ 与 $lastTX(s)$ 。规则: 若 $lastTX(s^*) < voteTX$ 则采纳对方的 $voteID$ 与 $voteTX$; 若相等而 $leader(s^*) < voteID$ 则只更新 leader。若某服务器认为自己应成为 leader 且尚未声明, 就广播 $(id(s^*), tx(s^*))$; 收到足够多 follower (多数) 即可自任 leader。

5.4.4 例: Raft 中的领导者选举

Raft 让少数 (通常五个) 已知的复制服务器执行相同操作、且顺序一致, 故需选出一个 leader 规定顺序。协议允许消息丢失与服务器崩溃。每台服务器处于 follower、candidate、leader 三态之一, 时间划分为 **term** (每 term 至多一个 leader, term 编号从 0 起)。leader 定期发心跳。follower 超时后进入 candidate 并发起选举 (term 加一、广播): 若收到同 term 的“自称 leader”则退回 follower; 若获得多数票则升为 leader; 否则等 candidate 超时后重新选举。为避免全体同时成为 candidate 而选不出, 各服务器随机化超时, 通常只剩单一 candidate。

5.4.5 大规模系统中的选举

无许可区块链中候选者可能极多, 问题变得棘手。**Proof of work**: 候选者进行计算竞赛——对已验交易块的哈希 (digest) 再找 **nonce**, 使两者合并后的哈希具有预定数量的前导零。前导零越多越难 (64 个前导零时平均需检查约 180 亿亿个 nonce, 超级计算机约 10 分钟、普通笔记本约 100 年)。难度被刻意调整以维持预期时长 (Bitcoin 约 10 分钟), 全体验证者用同一公开方法调整, 完全去中心化。缺点: 时长过长则吞吐低 (10 分钟、每块约 2500 笔, 约合每秒 4 笔); 时长过短则易出现多个胜者、产生分叉, 回滚会作废许多交易。**Proof of stake**: 以区块链上的 token 为权重, 用公开函数在 $1..N$ 中随机取 k , 第 k 个 token 的持有者成为 leader, token 越多越可能当选。它省去算力浪费, 但 leader 无须预付成本, 故通常更易受安全攻击。

Note 5.7: 选取 superpeer

有时需要选出多个节点（如 P2P 中的 super peer），要求：普通节点低延迟可达、super peer 均匀分布、占比预设、每个服务节点数有上限。在 DHT 系统中可把标识空间的一段（如前 $\lceil \log_2 N \rceil$ 位）留给 super peer；在几何空间方法中，把 N 个互斥的 token 随机散布到节点上，token 间施加斥力并通过 gossip 传播，使它们均匀铺开，持有 token 达一定时间者晋升为 superpeer。

5.4.6 无线环境中的选举

传统选举假设消息可靠、拓扑不变，这在移动环境不成立。Vasudevan 等（2004）的方案可处理节点失败与网络分区，且能选出最优而非随机的 leader：源节点广播 ELECTION，首次收到者认发送者为父并向邻居转发（父除外）；叶子节点回传自身容量（如电池寿命），逐层向上比较，最终源节点得知最佳候选并广播。多个选举并发时，各源用唯一标识标记，节点只参与标识最高的那个。

原文精读

Many distributed algorithms require one process to act as coordinator, initiator, or otherwise perform some special role.

许多分布式算法需要某个进程充当协调者、发起者，或扮演某种特殊角色。

In general, election algorithms attempt to locate the process with the highest identifier and designate it as coordinator.

一般来说，选举算法试图找到标识最大的进程，并将其指定为协调者。

Thus the biggest guy in town always wins, hence the name “bully algorithm.”

因此，城里块头最大的那个总是获胜——bully 算法之名由此而来。

原文定位：§5.4，原书 p.283–297（图 5.20 bully、图 5.21 环选举、图 5.22 ZooKeeper 选举、图 5.24 无线选举）

5.5 Gossip-based coordination**基于 gossip 的协调**

gossip (epidemic) 协议以指数速度传播信息。本节看三个应用：聚合、大规模 peer sampling、覆盖网构建，以及安全 gossip。

5.5.1 聚合 (Aggregation)

最广为人知的应用是传播更新；gossip 也可用于聚合信息。每个节点 P_i 初始取任意值 v_i ，与 P_j 交换后双方都更新为 $v_i, v_j \leftarrow (v_i + v_j)/2$ ，最终所有节点收敛到初值的平均。若令 P_1 为 1、其余为 0，则每节点算出的平均为 $1/N$ ，于是可用 $1/v_i$ 估计系统规模。节点频繁进出时可用 epoch 重置计算。也可用 $v_i, v_j \leftarrow \max\{v_i, v_j\}$ 随机选出发起者（保留初值 m_i ， $m_i < v_i$ 者出局，乐观地“在被证明不是赢家前假设自己是赢家”）。

5.5.2 peer-sampling 服务 (A peer-sampling service)

epidemic 协议需要节点能从全网随机选到另一节点，但大网络中节点没有全局视图。解决办法是用 epidemic 协议本身构建完全去中心化的 **peer-sampling service (PSS)**：每节点维护 c 个邻居的 **partial view**，条目带 age。两个线程协作：active 线程选 peer、构造 $c/2 + 1$ 条目的列表并等待回应；passive 线程对称处理。关键操作：**selectPeer**（从本地 partial view 随机选邻居）、**selectToSend**（挑选要发送的条目）、**selectToKeep**（合并、去重、裁剪到 c 条）。只要定期交换，从动态变化的 partial view 中随机选取，在统计上与从全网随机选取不可区分。常用实现是 Cyclon。

5.5.3 基于 gossip 的覆盖网构建 (Gossip-based overlay construction)

结构化与非结构化 P2P 并非泾渭分明：通过精心交换和挑选 partial view 条目，可构造并维护特定拓扑。采用两层：底层是非结构化 P2P，提供 peer-sampling 服务；上层在底层传来的 partial view 上再做选择（如按到某节点的距离排序的 **ranking function**），得到符合目标拓扑的邻居表。例如对 $N \times N$ 的逻辑网格维护 c 个最近邻，最终会演化出二维 **torus**。若只用上层、只交换“最优”链接，节点可能形成封闭集合（类似爬山陷入局部最优），因此引入随机性至关重要。

5.5.4 安全 gossip (Secure gossiping)

gossip 的同步速度也是其脆弱性：传播正确信息的时间同样在传播虚假信息。一个例子是 **hub attack**： c 个串通攻击者只返回彼此引用，逐渐污染良性节点的 partial view。在 10 万节点、partial view 大小 30、仅 30 个攻击者的例子中，不到 300 轮所有良性节点的 partial view 就只含攻击者。可用 **indegree** 分布检测：正常 Cyclon 的入度近似正态，受攻击后分布变宽、左移，最终几乎所有节点入度为 0（无人被引用）。Jesi 等（2010）的策略迫使攻击者守规矩：良性节点用返回的链接更新视图或收集统计，并可同时发起多个交换（最终只处理一个回复），使攻击者陷入两难——总返回同伙就会暴露，于是只能偶尔按规则行事，攻击因而失效。此外，允许推送的系统中攻击者可用高频推送快速污染，故有研究主张只允许拉取；聚合场景中攻击者可能故意返回错误值（如恒返回 v_A 使平均收敛到 v_A ），只能靠检测“未按协议行事”来缓解。

原文精读

An important aspect in epidemic protocols is the ability of a node P to choose another node Q at random from all available nodes in the network.

epidemic 协议的一个重要方面是：节点 P 能从网络中所有可用节点里随机选出另一节点 Q。

The crucial point is the construction of a new partial view.

关键在于新 partial view 的构造。

Gossiping is an attractive means for coordination. However, the speed by which a large collection of nodes manages to synchronize, is also an inherent vulnerability.

gossip 是一种有吸引力的协调手段。然而，大量节点得以同步的速度，本身也是一种固有脆弱性。

原文定位：§5.5，原书 p.297–306（图 5.25 PSS 两线程、图 5.27 torus 覆盖网、图 5.29 hub 攻击速度、图 5.30 入度分布）

5.6 Distributed event matching

分布式事件匹配

事件匹配（更准确说是通知过滤）是发布-订阅系统的核心：订阅者用订阅 S 说明感兴趣的事件，发布者发布通知 N ，系统判断 S 是否匹配 N ，匹配则把 N 送达订阅者。至少要做两件事：把订阅与事件匹配、以及在匹配时通知订阅者。可设函数 $\text{match}(S, N)$ 。当订阅表达力强（如基于内容的发布-订阅）时，匹配函数难以分布到多台服务器，会带来严重的可扩展性问题。

5.6.1 集中式实现 (Centralized implementations)

最朴素的做法是一台集中服务器保存所有订阅，发布通知时逐一比对。它可扩展性差，但在匹配高效且服务器算力足够时对许多场景可行（如 Linda tuple space）。扩展办法是确定性地把工作分到多台服务器：用 $\text{sub2node}(S)$ 与 $\text{not2node}(N)$ 把订阅/通知映射到非空服务器子集（rendezvous nodes），约束是二者交集非空；topic-based 系统常用对 topic 名做哈希来满足。进一步可把多台 broker 组织成覆盖网，用 flooding 路由通知：要么订阅存于每个 broker、通知只在单个 broker 发布；要么订阅只存一个 broker、通知广播到所有 broker（匹配被分散，负载更均衡）。TIB/Rendezvous 用主题关键词（如 `news.comp.*.books`）与 rendezvous daemon 实现。若订阅更富表达力（范围、包含、前后缀等，甚至类 SQL 谓词），可扩展性易成问题，可能需要退回到 topic-based。

5.6.2 安全发布-订阅方案 (Secure publish-subscribe solutions)

发布-订阅的特点是发布者与订阅者引用解耦（可能还有时间解耦），安全系统需在保证互相匿名的同时传递消息。用可信 broker 简单，但云端第三方是否可信存疑，且消息可能含依法不得泄露的数据（如病历）。于是产生冲突：既要匿名，又不能信任 broker 看内容，还要保证可用性与只送达合法订阅者。这本质是可搜索加密问题。**PEKS** (Public Key Encryption with Keyword Search) 是重要技术：消息 m 与关键词 $KW_1, \dots, KW_n$ 以公钥 PK 存为 $m^* = [PK(m) | PEKS(PK, KW_1) | \dots | PEKS(PK, KW_n)]$ ；为每个关键词生成 **trapdoor**，broker 用它判断标签 W 是否属于该发布，匹配则返回 $PK(m)$ 由订阅者用私钥 SK 解密。该方案通常假设 broker 是 **honest-but-curious** 且不与双方串通；Cui 等（2021）的系统则可处理串通 broker。

原文精读

Event matching, or more precisely, notification filtering, is at the heart of publish-subscribe systems.

Obviously, this is not a very scalable solution.

In practice, expressive subscriptions take the form of (attribute,value) pairs in which a value returns as an expression on the possible values for the specified attribute.

事件匹配，更准确地说通知过滤，是发布-订阅系统的核心。

显然，这不是一个可扩展性很好的方案。

在实践中，表达力强的订阅采取 (属性, 值) 对的形式，其中值是对该属性可能取值的一个表达式。

原文定位：§5.6，原书 p.306–315（图 5.31 TIB/Rendezvous、图 5.32–5.33 选择性路由与过滤器、图 5.34 范围查询分组、图 5.35 隐私保护发布-订阅）

5.7 Location systems

定位系统

跨广域网的大系统中常需考虑邻近性：覆盖网里相邻的两个进程在底层网络中可能相距很远，若它们频繁通信，最好在物理上也靠近。本节看用位置技术协调进程放置与通信。

5.7.1 GPS (Global Positioning System)

GPS 是 1978 年发射的卫星分布式系统，用至多 72 颗卫星（约 2 万公里高），每颗带至多四个原子钟，持续广播位置并给消息打本地时间戳。二维情形下三颗卫星的距离圆交于一点；三维则需四颗卫星以解经度、纬度、高度与接收机时钟偏差 Δ_r 四个未知数（测得距离 $\tilde{d}_i = d_i + c \cdot \Delta_r$ ）。误差来源众多（原子钟不完全同步、卫星位置不精确、接收机时钟有限精度、信号在电离层变慢等），平均约 5–10 米；差分 GPS 可进一步改善。

5.7.2 当 GPS 不可用时 (When GPS is not an option)

GPS 通常无法在室内使用，可用大量 WiFi 接入点：若已知接入点坐标且能估计距离，检测到三个接入点即可定位。难点是确定接入点坐标，常用 **war driving**：用带 GPS 的设备边移动边记录观测到的接入点（由 SSID 或 MAC 地址标识），在多个位置检测到后才能估算，简单方法是取质心 $\vec{x}_{AP} = \frac{\sum_i \vec{x}_i}{N}$ ，也可按信号强度加权。精度受各 GPS 检测点精度、接入点发射范围不均匀、采样数 N 影响，估计可能偏差数十米，且接入点频繁变动；Wigle 数据库靠众包维护。

5.7.3 节点的逻辑定位 (Logical positioning of nodes)

与其求绝对位置，不如用基于邻近性的逻辑位置：在 m 维几何空间中给每个节点一个坐标，使空间距离反映真实性能指标（最简单是节点间延迟）。这类系统称 **Network Coordinates System (NCS)**。应用包括 CDN 把请求重定向到最近的副本服务器、最优副本放置、以及基于位置的 **position-based routing**（只用位置信息转发，无须把链路信息传播到全网）。

集中式定位需 $m + 1$ 个已知位置的距离度量（二维需三个节点解三个二次方程）。但测得距离常不自洽，且 Internet 延迟可能违反三角不等式 $d(P, R) \leq d(P, Q) + d(Q, R)$ ，故通常无法完全消除不一致。Ng & Zhang (2002) 用 N 个 **landmark**，由中心节点最小化聚合误差为各 landmark 求坐标；实践表明 m 可小到 6 或 7，任意两节点用坐标算出的距离与真实延迟相差不超过 2 倍。去中心化的 **Vivaldi** 把节点看成用弹簧相连：误差 $|\tilde{d} - \hat{d}|$ 表示位移程度，力 $\vec{F}_{ij} = (\tilde{d}(P_i, P_j) - \hat{d}(P_i, P_j)) \times u(\vec{x}_i - \vec{x}_j)$ ，节点沿力方向微调位置，系统最终收敛到聚合误差最小的组织；关键是步长 δ 要自适应（误差大则大、误差小则小），否则会振荡或收敛过慢。

原文精读

When looking at large distributed systems that are dispersed across a wide-area network, it is often necessary to take proximity into account.

GPS uses up to 72 satellites, each circulating in an orbit at a height of approximately 20,000 km.

In geometric overlay networks each node is given a position in an m -dimensional geometric space, such

在考察横跨广域网的大规模分布式系统时，往往需要把邻近性纳入考虑。

GPS 使用至多 72 颗卫星，每颗在约 20,000 公里高的轨道上运行。

在几何覆盖网中，每个节点被赋予 m 维几何空间中的一个位置，使该空间中两节点间的距离能

that the distance between two nodes in that space reflects a real-world performance metric.

反映真实世界的性能指标。

原文定位: §5.7, 原书 p.315–322 (图 5.36 二维定位、图 5.37 一维不一致距离、GPS/landmark/Vivaldi 原理)

5.8 Summary

小结

- 同步的本质是“在正确的时间做正确的事”。分布式系统没有全局共享时钟，各机器有各自的时间观念。
- 时钟同步都基于交换时钟值并考虑消息往返时间；通信延迟的波动及处理方式在很大程度上决定了同步算法的精度。NTP 用四时间戳估计 offset 与 delay，并按 stratum 分级、渐进调整。
- 绝对时间常非必需，重要的是相关事件的顺序。Lamport 逻辑时钟给事件全局唯一的逻辑时间戳，保证 $a \rightarrow b \Rightarrow C(a) < C(b)$ ；向量时钟进一步使 $C(a) < C(b)$ 能推出 a 因果先于 b 。
- 分布式互斥保证任一时刻至多一个进程访问共享资源。用协调者最易实现；完全分布式算法虽存在，但通常更易受通信与进程失败影响。
- 需要某进程充当协调者而该角色又不固定时，由选举算法决定 (bully、ring、ZooKeeper、Raft)；无许可区块链中的选举则用 PoW/PoS。
- gossip 协调的关键是能从整个覆盖网中随机选 peer，可用 gossip 实现 peer-sampling 服务；结合对 partial view 条目的选择性替换即可高效构造结构化覆盖网。
- 分布式事件匹配是发布-订阅的核心。集中式实现简单；一旦要分散负载，就必须预先决定哪个节点负责哪部分订阅，而这对基于内容的匹配尤为困难，需要高级过滤技术。
- 定位把节点放入几何覆盖网，使几何距离可近似节点间延迟，方法与 GPS 求位置、时间高度相似。

实践清单

- 读原书第 5 章，重点看图 5.1 (时钟偏差导致乱序)、图 5.7 (Lamport 修正时钟)、图 5.12 (向量时钟捕获因果)、图 5.18 (四种互斥算法对比)、图 5.20 (bully 选举)、图 5.27 (gossip 生成 torus)、图 5.30 (入度分布暴露攻击)。
- 用一个三进程时序图，手动给事件标 Lamport 时间戳与向量时钟，验证“ $C(a) < C(b)$ 不蕴含 a 因果先于 b ”，而向量时钟可以判断。
- 对比集中式、分布式、token ring、去中心化四种互斥：在“消息数、失败点、是否可能饥饿”三个维度上各打分，判断你的场景更适合哪一种。
- 推演一次 Raft 选举：五台服务器、随机超时，说明为什么“通常只剩一个 candidate”。
- 用 gossip 平均法估算系统规模：若某节点收敛值为 v_i ，系统规模约为 $1/v_i$ ；再思考节点频繁进出时 epoch 如何避免估计失真。
- 对你熟悉的发布-订阅系统（如消息队列），判断它属于 topic-based 还是 content-based，并估计若改为基于内容匹配，瓶颈会出现在哪里。

Naming

名字用于引用实体、唯一标识、指向位置等，而名字解析让进程能访问被命名的实体。分布式命名系统的实现本身往往跨多机分布，分布方式决定其效率与可扩展性。本章按三类名字展开：*flat naming*（无结构标识符，靠 *DHT*、分层定位服务等追踪位置）、*structured naming*（人类可读的结构化名字，如 *DNS*，系统性地逐级解析）、*attribute-based naming*（用属性描述实体，本质是搜索，尤其困难）；最后讨论 *named-data networking*——直接用名字路由取数据，挑战“名字先解析成地址”的必要性。

6.1 Names, identifiers, and addresses

名字、标识符与地址

分布式系统中的名字是用于引用实体的位串或字符串，实体可以是主机、打印机、磁盘、文件，也可以是进程、用户、邮箱、网页、消息、网络连接等。要操作实体需先访问它，这需要一个访问点（**access point**）；访问点的名字叫地址（**address**）。一个实体可有多个访问点（如一个人有多部电话），且访问点会随时间变化（如移动主机换了 IP）。因此地址只是特殊类型的名字：它指向实体的某个访问点。

用地址直接引用实体很不灵活：服务器换主机、访问点被重新分配给别的实体时引用会立刻失效；实体有多个访问点也难选。于是更好的是与位置无关（**location independent**）的名字，独立于访问点地址。另一类需要特殊处理的是标识符（**identifier**），真标识符有三条性质：（1）一个标识符至多指向一个实体；（2）每个实体至多被一个标识符引用；（3）标识符始终指向同一实体（永不复用）。有了标识符，比较两个标识符是否相等即可判断是否指向同一实体（“John Smith”就不行）。电话号会重分配，故也不能当标识符。此外还有面向人的 **human-friendly name**（通常为字符串，如 Unix 文件名、DNS 名）。

本章核心问题：如何把名字/标识符解析成地址？有两条思路：一是维护（通常分布式的）(name, address) 表，如 *DNS*；二是通过路由把请求逐步送到名字关联的地址甚至直接到访问点，如结构化 P2P 与信息中心网络（*ICN*）。后一情形下，名字解析与消息路由的边界开始模糊。

原文精读

A name in a distributed system is a string of bits or characters that is used to refer to an entity.

分布式系统中的名字是用于引用某个实体的位串或字符串。

The name of an access point is called an address.

访问点的名字被称为地址。

An identifier always refers to the same entity (i.e., it is never reused).

一个标识符始终指向同一个实体（即永不复用）。

Such a name is called location independent.

这样的名字被称为与位置无关的。

原文定位：§6.1，原书 p.326–329

6.2 Flat naming

扁平命名

flat name（无结构名字）通常是随机位串，其重要性质是不包含任何关于如何定位其关联实体访问点的信息。问题便是：仅给定标识符，如何定位实体？

6.2.1 简单方案（Simple solutions）

广播：向每台机器广播含标识符的消息，能提供访问点者回送其地址。ARP 即用此法由 IP 地址找数据链路地址。网络变大后广播低效（浪费带宽、打扰太多主机），可改用多播只让受限的一组主机收到请求，也可用多播地址做通用定位服务或定位最近副本。转发指针：实体从 A 移到 B 时在 A 留下指向 B 的引用；优点是简单，缺点是高度移动实体的指针链可能过长、所有中间位置都须维护链、且任一指针丢失即失联，故须保持链短且指针健壮。

6.2.2 基于家的方案（Home-based approaches）

为移动实体引入 **home location** 跟踪其当前位置（常选实体的创建地）。**Mobile IP** 即用此法：移动主机用固定 IP，所有通信先送到其 **home agent**；主机移动到新网络时申请临时地址（**care-of address**）并注册到 home agent。home agent 收到分组后查当前位置：在本网则直接转发，否则隧道到当前位置（包在 IP 包里作为数据发送），同时告知发送方新位置；原 IP 地址实际被当作标识符使用。该机制对应用基本透明，实现了高度的位置透明。缺点是客户端须先联系可能相距很远的 home，增加延迟；且 home 固定，必须始终存在，长期移动实体的 home 最好能随之移动（可把 home 注册到传统命名服务中，因 home 相对稳定而可有效缓存）。

6.2.3 分布式哈希表（Distributed hash tables）

以 Chord 为代表：用 m 位标识符空间（通常 128 或 160 位）给节点和键随机赋标识符，键 k 归标识符 $\geq k$ 的最小节点管辖，记作 $\text{succ}(k)$ 。核心问题是高效地把 k 解析到 $\text{succ}(k)$ 的地址。朴素做法（只记录后继）平均要走半个环，不可扩展。Chord 每个节点维护 **finger table** $FT_p[i] = \text{succ}(p + 2^{i-1})$ ，即第 i 项指向至少比 p 大 2^{i-1} 的第一个节点；查找时把请求转发到满足 $q = FT_p[j] \leq k < FT_p[j+1]$ 的节点 q ，故一般只需 $O(\log N)$ 步。加入系统只需联系任一节点查出 $\text{succ}(p+1)$ 再插入环；难点在维护 finger table，尤其 $FT_q[1]$ （后继）必须正确，各节点定期联系 $\text{succ}(q+1)$ 并请求其前驱来校正，finger table 的其余项也定期在后台刷新；前驱失效则标记为“unknown”。

利用网络邻近性：Chord 的请求可能绕远路（如逻辑相邻的节点实际分处阿姆斯特丹与圣地亚哥）。Castro 等区分三种做法：**topology-based assignment**（让标识符邻近对应节点邻近，但在 Chord 这类一维空间中很难，且易暴露相关失效）、**proximity routing**（每个表项维护 r 个候选，转发时在不“越过”目标的前提下选离自己最近的）、**proximity neighbor selection**（优化路由表使最近节点被选为邻居，需有多个候选可选，如 Pastry）。

原文精读

An important property of such a name is that it does not contain any information whatsoever on how to locate the access point of its associated entity.

这类名字的一个重要性质是：它完全不含任何关于如何定位其关联实体访问点的信息。

The main issue in DHT-based systems is to

DHT 类系统的核心问题是：如何高效地把键 k 解

efficiently resolve a key k to the address of $\text{succ}(k)$.

析为 $\text{succ}(k)$ 的地址。

It should come as no surprise that a lookup will generally require $O(\log(N))$ steps, with N being the number of nodes in the system.

不出所料，一次查找通常需要 $O(\log N)$ 步，其中 N 是系统中节点的数量。

6.2.4 分层方案 (Hierarchical approaches)

以 Globe location service 为代表：网络划分为若干域 (**domain**)，顶层域覆盖全网，域可再分为子域，最底层是 **leaf domain**（通常对应一个局域网或移动蜂窝）。每个域 D 有目录节点 $\text{dir}(D)$ 记录该域中的实体，形成目录节点树；顶层域的 **root** 节点知道所有实体。实体在 leaf 域中的位置记录含其当前地址，而上层域的记录只含指向下级目录节点的指针；实体有多个地址（如被复制）时，包含这些地址的最小域会有多个指针。查找时从客户端所在 leaf 的目录节点开始，无记录则逐级上送，直到某节点有记录，再沿指针逐级向下到 leaf，取回地址——本质上是在以客户端为中心、逐渐扩大的“环”中搜索，利用局部性。插入（新增地址）同理自下而上推进，直到遇到已有记录的节点，再自上而下建指针链；删除则反向清理空记录。Note 6.2 说明逻辑设计（根需知道所有标识符）与物理实现的区别：可把逻辑目录节点分散到多台物理主机上，兼顾地理可扩展性与规模可扩展性。

6.2.5 安全的扁平命名 (Secure flat naming)

flat name 不含解析信息，故完全依赖解析过程。若解析过程不可信，响应也不可信。两条路：保障解析过程，或保障标识符与实体的绑定。后者即 **self-certifying name**：由实体数据算标识符 $\text{id}(\text{entity}) = \text{hash}$ （与实体关联的数据），客户端自行校验收到的数据；对可修改实体（如移动对象，实体即当前地址）可用 $\text{id}(\text{entity}) = \text{public key}(\text{entity})$ ，返回数据附签名摘要等供验证。这样一来，解析过程返回假数据或无数据是最坏结果，但返回什么客户端都能验证，信任被降级为“提供体面的服务”。保障解析过程更复杂，对 Mobile IP、分层定位服务需保证返回的（中间）答案合理；DHT 系统的安全较麻烦，主要面对 **Sybil** 与 **eclipse** 攻击，根本在于保证节点标识符是真实的（属于唯一拥有者且可验证），实践中往往需要集中式机构发放标识符。

原文精读

Flat names do not contain any information on how to resolve a name to the entity it is referring to.

扁平名字不含任何关于如何把名字解析到其所指实体的信息。

By securing the identifier-to-entity binding, it becomes less important to trust the name-resolution process.

通过保障标识符与实体之间的绑定，信任名字解析过程就变得不那么重要了。

原文定位：§6.2，原书 p.329–344（图 6.1 Mobile IP、图 6.2 Chord 查找、图 6.3 查找路径长度、图 6.5–6.9 分层定位服务）

6.3 Structured naming

结构化命名

flat name 适合机器，却不宜人用。命名系统通常支持由简单、人类可读的名字组成的 **structured name**（文件命名与 Internet 主机命名皆然）。

6.3.1 名字空间 (Name spaces)

结构化名字组织成 **name space**，可表示为带两类节点的有向图：**leaf node** 代表被命名实体、无出边，通常存实体的信息（如地址）或状态；**directory node** 有若干带标签的出边，存（节点标识符，边标签）的 **directory table**。只有一个节点只有出边、无入边者称 **root**。路径上的标签序列 $N : [label_1, \dots, label_n]$ 叫 **path name**，起点为根则称绝对路径名，否则为相对路径名。文件系统中常写成用“/”分隔的字符串（如 `/home/steen/mbox`），Plan 9 更把所有资源都用文件方式命名。名字空间多为单根、严格层次（树，每节点恰一入边），也有有向无环图（可多入边、无环）。需注意名字总是相对某个目录节点定义，故“绝对名字”一词略有误导；**global name** 处处指向同一实体，**local name** 的解释依赖使用位置。

6.3.2 名字解析 (Name resolution)

解析形如 $N : [label_1, \dots, label_n]$ 的路径名，从节点 N 开始，在 **directory table** 中查 $label_1$ 得到下一节点标识符，再在其中查 $label_2$ ，依此类推，直到返回末节点的内容。解析需要知道从何、如何开始，这称为 **closure mechanism**（如“00312059837784”须先知道是电话号码才能开始；环境变量 **HOME** 通过用户特定表解析）。别名有两种实现：多个绝对路径名指向同一节点（Unix 的 **hard link**），或 leaf 节点存一个绝对路径名（**symbolic link**）。挂载（**mounting**）用于透明合并不同名字空间：存有外部名字空间目录节点标识符的目录节点是 **mount point**，外部名字空间中的对应节点是 **mounting point**（通常是其根）。跨网络挂载至少需要三项信息：访问协议名、服务器名、外部名字空间中的挂载点名，且三者都要解析；例如 NFS URL `nfs://flits.cs.vu.nl/home/steen` 中，`nfs` 是众所周知的名字。

6.3.3 名字空间的实现 (The implementation of a name space)

命名服务由 **name server** 实现，大规模时须把名字空间分布到多台服务器。Cheriton & Mann 将其分为三层：**global layer**（根及其近旁节点，很少变动，代表组织；高可用性尤为关键）、**administrational layer**（同一组织内管理的目录节点，相对稳定）、**managerial layer**（常变动的节点，如本地主机、共享文件、用户目录，由终端用户维护）。DNS 中把不重叠的部分称为 **zone**，由独立 **name server** 实现。各层对可用性与性能要求不同：**global** 层靠复制 + 客户端缓存，更新可延迟生效；**administrational** 层须在几毫秒内响应且更新较快；**managerial** 层可用单机但须即时响应、缓存效果差。图 6.15 给出对比。

名字解析的实现有迭代式与递归式两种。迭代式：客户端 **resolver** 把完整名字交给根服务器，根能解析多少就返回多少及下一台服务器地址，客户端再逐级联系（如解析 `root:[nl,vu,cs,ftp]`）。递归式：服务器把结果传给下一台服务器，逐层向下直到取回结果再回传；缺点是每个 **name server** 负担更重（故 **global** 层通常只支持迭代式），优点是缓存更有效且通信开销更小（递归式下客户端只需与其本地服务器交互，其余在服务器间进行；迭代式则客户端要分别联系多台服务器，成本可能约为三倍）。

6.3.4 例：域名系统 (The Domain Name System)

DNS 是当今最大的分布式命名服务之一，主要用于查主机与邮件服务器的 IP 地址。其名字空间是有根树，**label** 是不区分大小写的字母数字串（最长 63 字符，完整路径名最长 255 字符），字符串表示从最右 **label** 开始、用“.”分隔，根用“.”表示（如 `flits.cs.vu.nl.`）。子树称 **domain**，到其根节点的路径名称 **domain name**。节点内容由一组 **resource record** 组成，主要类型：**SOA**、**A**（主机 IP）、**MX**（邮件服务器）、**SRV**（特定服务的服务器）、**NS**（区域的 **name server**）、**CNAME**（符号链接）、**PTR**（反向映射）、**HINFO**、**TXT**。DNS 区分别名与 **canonical name**；反向映射用 `in-addr.arpa` 域（如 `20.20.37.130.in-addr.arpa`）。DNS 名字空间可看作 **global** 与 **administrational** 两层，**managerial** 层（本地文件系统）形式上不属于 DNS。每个 **zone** 由一台 **name server** 实现、几乎总是复制；更新由

primary 处理，**secondary** 通过 **zone transfer** 获取内容。DNS 数据库是一小组文件，最重要的文件含某 zone 全部节点的 resource record。

现代 DNS 有三点变化：许多组织使用外部 **resolver**（可能导致 CDN 依据 resolver 而非客户端地址做择优选择）、客户端/浏览器常绕过本地配置直接联系自选 resolver（更难观测，且若客户端到 resolver 的通信加密则更难）、越来越多组织不再自建 DNS 服务器而把解析外包给第三方（DNS 日益集中化）。安全方面：**DNSSEC** 由 zone 拥有者签名 resource record，同类型记录先分组再签名，公钥（zone-signing key）由 key-signing key 签名、其哈希由父域签名，形成直到根的信任链；因记录变大，需扩展机制支持超过 512 字节的 UDP 报文。隐私方面：**DNS over TLS**（端口 853）与 **DNS over HTTPS** 防止第三方得知应用访问了哪些网站；还可只让服务器解析路径中相关部分（如只请求 `.nl`）以提升机密性。

原文精读

Flat names are good for machines, but are generally not very convenient for humans to use.

扁平名字对机器友好，但通常不太方便人类使用。

The process of looking up a name is called name resolution.

查找名字的过程被称为名字解析。

Knowing how and where to start name resolution is generally referred to as a closure mechanism.

知道从何处、如何开始名字解析，通常被称为闭包机制（closure mechanism）。

原文精读

One of the largest distributed naming services in use today is the Internet Domain Name System (DNS).

当今使用中最大的分布式命名服务之一，是 Internet 域名系统（DNS）。

The DNS name space is hierarchically organized as a rooted tree.

DNS 名字空间以有根树的形式层次化组织。

Note 6.8: 去中心化还是分层实现?

DNS 采用层级服务器（13 个众所周知的根节点，末端有数百万台服务器），高层节点收到远多于低层的请求，只有靠缓存高层名字到地址的绑定才能避免被淹没。原则上可用全去中心化方案（对 DNS 名做哈希、在 DHT 或全分区根的分层定位服务中查找），但会丢失名字的结构（如无法高效找出某域的所有子节点）。Pappas 等（2006）指出当前分层设计其实不差，原因有二：其一，分层中各节点并不平等，高层节点被特别工程化（如 RIPE NCC 的根节点在约 25 个站点实现、用同一 IP，均为高健壮复制集群），而 DHT 把所有名字一视同仁、难以把顶级域分离出来；其二，DNS 缓存极有效且几乎完全由查询的本地分布驱动，而 DHT 的缓存与局部性关系很弱。结论：用去中心化实现取代 DNS 未必是好主意。

6.3.5 例：网络文件系统（The Network File System）

NFS 命名模型的根本思想是让客户端完全透明地访问服务器维护的远程文件系统，做法是把远程文件系统挂载到本地文件系统中。服务器导出（**export**）一个目录即让其条目对客户端可用；客户端可只挂载文件系统的一部分。该设计的一个重要后果是用户不共享名字空间：同一文件在客户端 A 叫 `/remote/vu/mbox`、在 B 叫 `/work/me/mbox`，名字取决于本地组织与挂载点，故共享文件变难（Alice 无法用自己给文件起的名字告诉 Bob）。常见补救是给每个客户端提供部分标准化的名字空间（如 `/usr/bin`、`/local`）。NFS 服务器可挂载其他服务器导出的目录，但不允许再导出给自己的客户端，否则需返回含服务器标识的 file handle，而 NFS 不支持。

file handle 是文件系统内对文件的引用，独立于文件名、由服务器创建、对其导出的所有文件系统唯一，且对客户端完全不透明（NFSv2 为 32 字节，v3 可变至 64，v4 至 128）。理想情况下它是文件相对于文件系统的真标识符：只要文件存在，handle 就不变（持久性），于是客户端可缓存 handle、避免每次操作都重新查名字，也能在文件改名后仍访问它；服务器删除文件后不得复用 handle，否则客户端可能误访问。早期 NFS 的解析严格迭代（一次查一个名字），客户端负责整个路径解析；且查找不能跨越挂载点，会返回原目录的 handle 而非挂载文件系统根目录的内容。NFSv4 支持递归查找并允许查找跨越服务器上的挂载点（返回被挂载目录的 handle，客户端可通过文件系统标识符检测到并本地挂载）。初始 handle 问题：NFSv3 用单独的 mount 协议，NFSv4 用 **putrootfh** 操作让服务器相对其文件系统根解析名字，因而无需单独 mount 协议。Note 6.9 介绍 **automounter**：按需挂载（用户登录时才挂载其 home），为提升性能，可把目录挂到特殊子目录并安装符号链接，使后续操作绕过 automounter。

原文精读

The fundamental idea underlying the NFS naming model is to provide clients complete transparent access to a remote file system as maintained by a server.

NFS 命名模型的根本思想是：让客户端对服务器所维护的远程文件系统拥有完全透明的访问。

A file handle is a reference to a file within a file system. It is independent of the name of the file it refers to.

file handle 是文件系统内对某个文件的引用，它独立于其所指文件的文件名。

原文定位：§6.3，原书 p.344–375（图 6.10 命名图、图 6.14 DNS 三层分区、图 6.16–6.17 迭代/递归解析、图 6.24 DNSSEC 信任链、图 6.25–6.27 NFS 挂载与 automounter）

6.4 Attribute-based naming

基于属性的命名

flat name 与 structured name 提供唯一、与位置无关的引用，且常假定名字只指一个实体。但信息增多后，按描述搜索实体变得重要，用户只需给出“在找什么”的描述。分布式系统中流行用 (**attribute, value**) 对描述实体，即 **attribute-based naming**：实体有一组属性，用户指定某些属性的取值即约束感兴趣集合，命名系统返回满足描述的实体。

6.4.1 目录服务 (Directory services)

基于属性的命名系统又称 **directory service**，支持结构化命名者一般叫 naming system。设计合适的属性集并不简单（多为手工），即使属性集达成共识，不同人给值的一致性也是难题。为统一资源描述方式，出现了 **Resource Description Framework (RDF)**：资源用 (subject, predicate, object) 三元组描述（如 (**Person, name, Alice**）），三者本身都可为资源，引用本质是 URL。属性描述的存储若与资源不同处，会有严重性能问题：与结构化命名不同，基于属性的查找本质上需要穷举搜索所有描述符（可用索引等技术缓解），在单一非分布式数据仓库中尚可，但把搜索查询发给成百上千台服务器则通常不明智。

6.4.2 分层实现: LDAP (Hierarchical implementations: LDAP)

常见做法是把结构化命名与基于属性的命名结合, 如 Microsoft Active Directory, 多依赖 LDAP (源自 OSI 的 X.500)。概念上 LDAP 目录服务由若干 **directory entry** 组成, 每条是 (attribute, value) 对集合, 属性有类型, 分单值与多值。标准命名属性 (C、L、O、OU、CN) 可组成全局唯一名, 如 /C=NL/O=VU University/OU=Computer Science, 类比 DNS 的 nl.vu.cs。每个命名属性叫 **relative distinguished name (RDN)**, RDN 依序组合成层次化的 **directory information tree (DIT)** (即 LDAP 的命名图, 节点可同时充当目录与记录)。全部条目的集合称 **directory information base (DIB)**。两个查找操作: **read** 按 DIT 路径名读单条记录, **list** 列出某节点的所有子节点名 (不含记录)。实现方式类似 DNS: DIT 分区后分布到 **directory service agent (DSA)**, 客户端用 **directory user agent (DUA)** 按标准协议交互。与 DNS 不同的是搜索能力: 可按条件搜索条目, 如 `search("(C=NL)(O=VU University)(OU=*)(CN=Main server)")`。搜索通常昂贵: 往往要访问多个 leaf 节点乃至多个 DSA, 而命名服务的查找常只需一个 leaf。Active Directory 更进一步允许若干树并存互链 (**forest**), 常假设有一个可先搜索的全局索引服务器 (**global catalog**) 以缓解可扩展性问题; LDAP 也常与 DNS 结合 (树根常以 DNS 名表示, 并通过 SRV 记录找到)。

6.4.3 去中心化实现 (Decentralized implementations)

P2P 兴起后, 研究者也探索去中心化的基于属性的命名。用分布式索引: 若有 d 个属性, 就为每个属性设一台服务器, 保存该属性的 (E, val) 对; 查询发给相关属性的服务器, 客户端再求交集。缺点是: 涉及 k 个属性的查询要联系 k 台索引服务器 (通信开销大); 客户端要处理返回的集合 (如查“Pheriby Smith”, `lastName = Smith` 的键可能有上百万); 且不易支持范围查询 (如价格区间)。另一种常见方法是 **space-filling curves**: 把 N 个属性张成的 N 维空间映射到一维, 再用哈希分布到索引服务器, 关键是让“接近”的 (attribute, value) 对落到同一服务器。以 **Hilbert** 曲线为例: 递归地把正方形分成四个子方块并用一条线连接, 经旋转与反射使其与相邻大方块中的线良好衔接; order k 的曲线连接 2^{2k} 个子方块、有 2^{2k} 个索引。space-filling curve 的重要性质是保持局部性: 曲线上相近的索引对应多维空间中相近的点 (反之不必然)。搜索时给定各属性的区间, 即划定一个矩形区域, 曲线与该区域相交的段所索引的文件即匹配; 还需一个“给定区域返回一串曲线索引”的操作 (依赖所用曲线, 但不依赖具体实体)。Squid 系统用 Chord 环存放索引 (索引空间与环相同), 负责索引 i 的 Chord 节点保存被 i 索引的实体。Note 6.10 介绍 SWORD: 把 (attribute, value) 对转成 DHT 的 key (属性名与值分占不同位, 另加随机位保证唯一), 按属性名分区、按值区间再分; 优点是范围查询易支持, 缺点是更新要发往多台服务器、负载均衡不明。

原文精读

There are many ways in which descriptions can be provided, but a popular one in distributed systems is to describe an entity in terms of (attribute, value) pairs, generally referred to as attribute-based naming.

Unlike structured naming systems, looking up values in an attribute-based naming system essentially requires an exhaustive search through all descriptors.

A common approach to implementing decentralized attribute-based naming systems is to use what are known as space-filling curves.

描述的方式有很多, 但在分布式系统中流行的一种, 是用 (属性, 值) 对来描述实体, 通常称为基于属性的命名。

与结构化命名系统不同, 在基于属性的命名系统中查找值, 本质上需要对所有描述符进行穷举搜索。

实现去中心化基于属性命名系统的一种常见方法, 是使用所谓的空间填充曲线。

于属性的命名是本章最“难啃”的部分，因为它把命名问题转成了分布式搜索问题。记住两条主线：一是分布式索引（每个属性一台服务器、客户端求交集，简单但不支持范围查询且通信开销大），二是空间填充曲线（把多维降到一维并保持局部性，从而支持范围查询，Squid 把它与 Chord 环结合）。共同的核心矛盾始终是：如何把属性/子空间分配给服务器，使客户端知道该问谁，同时保持负载均衡。

原文定位：§6.4，原书 p.375–385（图 6.29 LDAP 条目、图 6.30 DIT、图 6.31 Hilbert 曲线）

6.5 Named-data networking

命名数据网络

前文区分了名字、标识符与地址，并主张名字须先解析成地址才能访问实体。唯一一次例外是用 Chord 等 DHT 的标识符直接路由查找请求。本节挑战“名字到地址解析”的必要性，聚焦 **Information-centric networking (ICN)** 中最流行的形式 **named-data networking (NDN)**。

6.5.1 基础 (Basics)

NDN 的原则是：应用其实不关心实体存在哪里，只求需要时能拿到一份副本在本地访问。自约 2007 年起，研究致力于设计替代当今 Internet 主机寻址的方案：应用用实体的名字从网络取回实体，网络以该名字为输入，把请求路由到存放该实体的合适位置并返回副本。效果上 NDN 在未来的 Internet 架构中取代 IP 的角色。重要后果是：不再先解析名字成地址、再向该地址路由，而是直接用名字取数据——这与 DHT 中基于键直接路由查找请求同理。NDN 假设名字是结构化的，如 `/distributed-systems.net/books/Distributed Systems/4/01/Naming`。它假定数据不经改名不得修改，否则路由器的缓存效果会大打折扣；用户若知道可能有更新，可请求最新版本，NDN 还提供单独的同步协议让数据所有者向感兴趣方通告更新。

6.5.2 路由 (Routing)

看似任意的结构化名字未必比 IP 更难路由：要按 IPv4 地址 `145.100.190.243` 找访问点，同样需要某种全局路由机制。Zhang 等 (2019) 认为两者本质无异，关键在于决定名字或地址的哪一部分（前缀）要在全局路由设施中通告，正如 IPv4 用 BGP 路由器所做。例如可把 `/distributed-systems.net` 作为全局前缀，在 BGP 层把请求路由到负责该前缀的组织网络，网络内再用名字的其余部分查找内容。NDN router 由三要素构成：(1) **content store**——缓存已查名字的数据，命中即直接返回；(2) **pending interest table**——记录 (name, interface) 对，以便数据到达时按接口回送，若无（已无）兴趣则可丢弃数据；(3) **forwarding information base**——无法服务请求时的处置（泛洪到邻居、随机游走或丢弃）。这些决策本质上与 IPv4/IPv6 路由无异。请求返回时，路由器查是否有 pending 请求、按接口回送并清除条目，也可能把内容存入 content store 以备将来。该设计为缓存内容、淘汰策略、转发对象等带来许多优化空间。

6.5.3 NDN 中的安全 (Security in named-data networking)

除安全命名外，name server 与路由器本身也需保护（威胁多是试图中断服务）。安全命名的核心是确保提供的名字与所指内容不可伪造地绑定。最简单做法是把内容的签名摘要包含进名字，类似 self-certifying name：只要接收方另有信息（所用哈希函数、公钥及其证书等），就能验证返回内容确实与名字关联。

原文精读

Named-data networking revolves around the principle that applications are not really interested to know where an entity is stored, but rather that they can get a copy to access it locally when needed.

An important consequence of this organization is that instead of first resolving a name to an address, and then routing a request toward that address, the name of an entity is directly used to fetch the associated data.

Secure naming is all about ensuring that the provided name is unforgeably linked to the content it refers to.

命名数据网络围绕这样一个原则：应用并不真的关心实体存储在哪里，而只关心在需要时能拿到一份副本供本地访问。

这种组织方式的一个重要后果是：不再先把名字解析成地址、再向该地址路由请求，而是直接用实体的名字来取回关联数据。

安全命名的核心，是确保所提供的名字与其所指的内容之间具有不可伪造的绑定。

原文定位：§6.5，原书 p.385–389（图 6.32 NDN 与 IP 的“沙漏”对比、图 6.33 NDN 路由器三要素）

6.6 Summary

小结

- 名字用于引用实体，本质上分三类：**address**（实体某访问点的名字）、**identifier**（每实体恰一个、只指一个实体、永不复用）、**human-friendly name**（面向人、字符串）。据此区分 flat、structured、attribute-based 三类命名。
- **flat naming** 要把标识符解析成实体地址，定位方式有：广播/多播；转发指针（须定期缩短指针链）；home（实体移动时通知 home，先问 home 再定位）；结构化 P2P（按标识符分配节点并设计路由算法）；分层搜索树（网络划分为互不重叠的域，逐级目录节点存指针，顶层知道所有实体）。
- **structured name** 便于组织成 name space（命名图：节点代表实体、边标签是名字），大规模命名图常为有根有向无环图。实体用 path name 引用，**name resolution** 逐个查找路径分量、遍历命名图；大规模图把节点分布到多台 name server，解析到由某服务器实现的节点时即转到该服务器继续。
- **attribute-based naming** 用 (attribute, value) 对描述实体，查询也如此表述，本质上要穷举搜索所有描述符，仅在单一数据库中可行；把对映射到 DHT 可分散描述符，再用空间填充曲线让不同节点负责属性不同取值，有助于在搜索时分摊负载。
- 与其区分名字与地址，研究者探索直接按名字路由请求（类似 P2P 中用标识符查信息）。在 **named-data network** 中，应用直接以名字取（只读）数据，网络最终返回关联数据而非访问点地址。

实践清单

- 读原书第 6 章，重点看图 6.1 (Mobile IP)、图 6.2 (Chord 查找)、图 6.5–6.8 (分层定位服务的查找与更新)、图 6.14 (DNS 三层分区)、图 6.16–6.17 (迭代与递归解析)、图 6.24 (DNSSEC 信任链)、图 6.31 (Hilbert 曲线)、图 6.33 (NDN 路由器)。
- 手算一次 Chord 查找：在 $m = 5$ 的环上从节点 1 解析键 26，写出 finger table 并标出每一跳，验证为 $O(\log N)$ 。
- 对比迭代式与递归式名字解析：从“服务器负担、缓存效果、通信开销”三点各写一句话。
- 用 RDF 三元组描述你身边的三个资源（如（书，作者，X）），再写出对应的一个属性查询，思考为什么它需要穷举搜索。
- 判断一个你常用的系统用的是 flat、structured 还是 attribute-based 命名，并说明它把“名字解析”与“路由”的边界画在哪里。
- 就 NDN 的“数据不得改名修改”假设展开一次小辩论：它带来什么好处，又给需要频繁更新的应用（如聊天、协同编辑）制造了什么麻烦。

Consistency and Replication

复制数据有两大动机——可靠性 (*reliability*) 与性能 (*performance*)，但副本一多就带来一致性 (*consistency*) 问题。本章先讲清“一致”究竟指什么：从以数据为中心 (*data-centric*) 的模型，到以客户端为中心 (*client-centric*) 的模型；再讲副本如何管理与更新如何传播，以及实现这些模型的一致性协议 (*consistency protocol*)；最后以 *Web* 缓存与 *CDN* 收尾。核心张力是：一致性越强，越难扩展、代价越大，所以现实系统往往选择“放松”。

7.1 Introduction

引论

复制的动机不外乎两条：一是可靠性——一份副本崩溃可切到另一份，多份还能对抗数据损坏（三份中取至少两份返回的值）；二是性能——把数据放到离使用它的进程更近的地方，从而降低访问时间，应对规模或地理范围的扩展。但复制有代价：只要一份副本被修改，它就和其余副本不同了，于是必须把修改传播到所有副本。何时、如何传播，决定了代价的大小。以浏览器缓存网页为例：本地副本让访问极快，可一旦网页在别处被更新，缓存就变旧了。应对办法要么是禁止本地副本（但离用户远时访问时间仍差），要么让服务器去失效/更新每个缓存——而后者要求服务器跟踪所有缓存并发消息，可能反过来拖垮自己的性能。

7.1.1 复制的理由 (Reasons for replication)

可靠性复制：文件系统有副本时，一份崩溃可切换到另一份继续工作；维护多份还能防损坏，例如三份副本、每次读写都作用于三份，则以至少两份返回的值作为正确值。性能复制：当大量进程访问同一台服务器上的数据（规模扩展），或需要缩短地理距离（地理扩展）时，复制服务器、分散负载即可。代价在于一致性：一旦某副本被修改，就必须把修改传播到其余副本。

原文精读

There are two primary reasons for replicating data.

复制数据有两个主要理由。

The problem with replication is that having multiple copies may lead to consistency problems.

复制的问题在于：拥有多份副本可能引发一致性问题。

Whenever a copy is modified, that copy becomes different from the rest.

每当一份副本被修改，该副本就与其余副本不同了。

原文定位：§7.1.1，原书 p.393–394（图 7.1 逻辑数据存储的一般组织）

7.1.2 作为扩展技术的复制 (Replication as scaling technique)

设进程 P 每秒访问本地副本 N 次，而副本每秒被更新 M 次。若 $N \ll M$ ，大量更新版本永远读不到，为它们付出的网络通信就是浪费——此时不如不装本地副本。更严重的是：保持多副本一致本身可能带来扩展问题。所谓 **tight consistency**（由 **synchronous replication** 提供）要求更新在所有副本上作为单一原子操作（事务）；难点在于需要全局同步——副本先要对“更新在何时本地执行”达成一致，例如用 Lamport 时间戳决定全局顺序，或由协调者指派顺序，这在广域网上很耗时。于是形成两难：复制/缓存改善性能，全局同步又拖累性能。出路是放松一致性约束，代价是副本不总相同；能放松到什么程度，取决于访问/更新模式与数据用途。

原文精读

Often, the only real solution is to relax the consistency constraints.

通常，唯一真正的解法是放松一致性约束。

The cure may be worse than the disease.

这种“治疗”可能比疾病本身更糟。

原文定位：§7.1.2，原书 p.394–395

7.2 Data-centric consistency models

以数据为中心的一致性模型

这里把共享数据抽象为 **data store**（可物理分布、每进程有本地副本），写操作传播到其他副本。一致性模型本质上是进程与数据存储之间的契约：若进程遵守某些规则，存储就承诺正确工作。理想情况下读应返回“最后一次写”的结果，但没有全局时钟，“最后一次”很难精确定义，于是产生一系列模型，各自限制读能返回的值。规律是：限制越强的模型越好用，性能却越差；限制越弱的越难用，性能越好。

7.2.1 操作的一致性排序 (Consistent ordering of operations)

顺序一致性 (sequential consistency) (Lamport 1979)：结果等价于把所有进程的读写操作按某个顺序执行，且每个进程的操作保持其程序序。它不涉及时间，只要求所有进程看到同一交错。原文用 $W_i(x)a$ 、 $R_i(x)b$ 记号与 720 种潜在序列（其中 90 种合法）说明其微妙。顺序一致性不可组合：两个各自顺序一致的变量，合起来未必一致；补上 **linearizability**（线性一致性）可解决——要求每个操作“在其起始与完成之间的某一瞬间原子生效”。

因果一致性 (causal consistency) 把操作分为潜在因果相关与并发（并发即无因果相关）：因果相关的写必须被所有进程以同一顺序看到，并发写可在不同机器以不同顺序看到。实现需维护依赖图。更弱的模型通过对同步变量（锁）把一组读写“括起来”（**ENTER_CS/LEAVE_CS**，临界区 **critical region**），提高粒度，即 **entry consistency**：锁随对象分别管理，未加锁的普通读可能读到本地旧值。最后要区分 **consistency**（针对一组数据项）与 **coherence**（针对单个数据项）。

原文精读

The result of any execution is the same as if the (read and write) operations by all processes on the data store were executed in some sequential order and the operations of each individual process appear in this

任何一次执行的结果，都等同于所有进程对数据存储的（读和写）操作按某个顺序执行，且每个进程各自的操作在此序列中按其程序规定的顺序出现。

sequence in the order specified by its program.

Each operation should appear to take effect instantaneously at some moment between its start and completion.

Writes that are potentially causally related must be seen by all processes in the same order. Concurrent writes may be seen in a different order on different machines.

Where data consistency is concerned with a set of data items, coherence models describe what can be expected to hold for only a single data item.

每个操作都应看起来是在其起始与完成之间的某一瞬间即时生效的。

潜在因果相关的写必须被所有进程以相同顺序看到；并发写则可在不同机器上以不同顺序被看到。

数据一致性关注的是一组数据项，而一致性（coherence）模型描述的是仅对单个数据项可以期望成立的性质。

原文定位：§7.2.1，原书 p.396–406（图 7.2–7.13，含顺序一致性、线性一致性、因果一致性、entry consistency）

7.2.2 最终一致性（Eventual consistency）

许多系统里并发其实很受限：数据库多为读、很少写；Web 页面通常只有单一权威（webmaster 或页主）在更新，不存在 write-write 冲突；DNS 按域划分，每个域只由一个命名权威更新。这些场景可以懒惰地（lazy）传播更新，容忍暂时不一致。它们的共同点是：只要长期没有更新，所有副本会逐渐变得相同——这就是 **eventual consistency**。它只要求更新最终传播到所有副本；无 write-write 冲突时副本会收敛，实现往往很便宜。为了让它在冲突时也“收敛到同一状态”，出现了 **strong eventual consistency** 与 **CRDT**（无冲突复制数据类型），允许并发更新而无需额外协调。

原文精读

This form of consistency is called eventual consistency.

这种形式的一致性称为最终一致性。

Eventual consistency essentially requires only that updates are guaranteed to propagate to all replicas.

最终一致性本质上只要求保证更新会传播到所有副本。

原文定位：§7.2.2，原书 p.406–410（Note 7.3：如何把最终一致性变强）

7.2.3 连续一致性（Continuous consistency）

不存在“复制数据的最佳方案”。Yu 与 Vahdat 用三条彼此独立的轴来描述可容忍的不一致：数值偏差（numerical deviation，如股价副本间不超过 \$0.02）、陈旧度偏差（staleness deviation，如天气报告几小时内可接受）、排序偏差（ordering deviation，各副本更新顺序可不同但有界）；它们构成 **continuous consistency ranges**。度量单元叫 **conit**：例中把 (g, p, d) 三个变量视为一个 conit，副本用二维向量时钟记录；A 有 3 个暂未提交的更新即排序偏差为 3，数值偏差用“未看到的操作数 + 缺失值之和”表示。conit 的粒度存在权衡：太大易令副本过早进入不一致（false sharing），太小则管理开销大。

原文精读

There is no such thing as the best solution to replicating data.

复制数据并不存在所谓的最佳方案。

They refer to these deviations as forming continuous

他们把这些偏差统称为连续一致性区间。

consistency ranges.

原文定位：§7.2.3，原书 p.410–414（图 7.15 偏差示例、图 7.16 conit 粒度）

7.3 Client-centric consistency models

以客户端为中心的一致性模型

以数据为中心的模型追求系统级一致视图，前提是并发进程会同时更新数据。但有一类数据存储少有并发更新（更新容易解决、多为读），只提供弱一致（如最终一致）。问题在于：客户端若短时间内访问不同副本（典型是移动用户 Alice 换位置或换设备），先前更新尚未传播，她就会觉得“什么都没发生”。**client-centric consistency** 只对单个客户端的访问序列提供保证，不保证不同客户端的并发访问。它源自 Bayou 等移动数据系统。记号： x_i 是数据项 x 的版本， $WS(x_i)$ 是其写集， $W(x_i; x_j)$ 表示 x_j 由 x_i 派生。

7.3.1 单调读（Monotonic reads）

保证：进程读到 x 的某个值后，后续对 x 的读永远返回该值或更新的值——即绝不会看到更旧的版本。例：分布式邮件库，在旧金山读过的邮件，飞到纽约再打开时仍然在（消息不会被删除或标记）。

原文精读

If a process reads the value of a data item x , any successive read operation on x by that process will always return that same value or a more recent value.

若某进程读到了数据项 x 的值，该进程此后对 x 的任何读取操作都将返回同一值或更新的值。

原文定位：§7.3.1，原书 p.417–418（图 7.18）

7.3.2 单调写（Monotonic writes）

保证：同一进程对 x 的写操作按发起顺序完成。换言之，某副本上执行新写之前，必须先让该副本反映出同一进程先前的所有写（这些写可能发生在别的副本上）。它类似于以数据为中心的 **FIFO consistency**，只是仅针对单个进程。例：软件库更新——若在某副本上更新，则先前所有更新必先执行，最终副本含全部更新。

原文精读

A write operation by a process on a data item x is completed before any successive write operation on x by the same process.

同一进程对数据项 x 的某次写操作，先于该进程此后对 x 的任何写操作完成。

原文定位：§7.3.2，原书 p.418–420（图 7.19）

7.3.3 读你所写（Read your writes）

保证：某进程对 x 的写，必被同一进程后续对 x 的读看到。典型反例是更新网页后刷新却看到旧版（浏览器或服务端返回了缓存副本）；改密码后短时间内登录不了也属此类。把编辑器与浏览器集成，

更新时使缓存失效即可满足。

原文精读

The effect of a write operation by a process on data item x will always be seen by a successive read operation on x by the same process.

某进程对数据项 x 的写操作的效果，总会被同一进程此后对 x 的读操作看到。

原文定位：§7.3.3，原书 p.420–421（图 7.20）

7.3.4 写跟随读（Writes follow reads）

保证：某进程在读过 x 之后的写，作用在所读到的那个值或更新的值之上。例：网络新闻组中，对某文章的回复只有在原文章已被写入同一副本后才会被写入，保证读者先看到原文再看到回复。

原文精读

A write operation by a process on a data item x following a previous read operation on x by the same process is guaranteed to take place on the same or a more recent value of x that was read.

同一进程在先前读过数据项 x 之后对 x 的写操作，保证作用在其所读到的 x 的同一值或更新值之上。

原文定位：§7.3.4，原书 p.421–422（图 7.21）

7.3.5 例：ZooKeeper 中的客户端一致性（Example: client-centric consistency in ZooKeeper）

ZooKeeper 结合了数据为中心与客户端为中心的元素：它保证更新可序列化且保持优先级（monotonic writes：可按某个全序解释所有更新，且每个客户端提交的操作顺序被保留），还保证单调读；但不保证读你所写，也不保证写跟随读。结构上有一个固定的主服务器（primary）接收全部写并按提交顺序处理，读由客户端所连的服务器执行；主服务器何时把其他服务器更新，客户端并不知情。

原文精读

ZooKeeper guarantees that update operations are serializable and keep precedence.

ZooKeeper 保证更新操作是可序列化的，并保持优先级。

It does not, however, guarantee read-your-writes or writes-follow-read consistency.

然而，它不保证读你所写、也不保证写跟随读的一致性。

原文定位：§7.3.5，原书 p.422–423（图 7.22）

7.4 Replica management

副本管理

一旦决定复制，就要回答“放哪里、放什么、如何保持同步”。放置问题应拆成两个子问题：**replica-server placement**（在哪里放能托管数据存储的服务器）与 **content placement**（把内容放在哪些服务器上）；后者常只需为单个数据项找最优位置，但必须先有副本服务器。

7.4.1 寻找最佳服务器位置 (Finding the best server location)

本质是“从 N 个位置里选最优的 K 个”的优化问题，计算复杂，只能用启发式。判优标准分成本相关与网络相关（延迟、带宽、逻辑距离、跳数）。算法分几类：**QoS aware**（优化或保证某 QoS 参数）、**consistency aware**（考虑保持副本最新的代价，含周期性/非周期/基于过期/基于缓存的更新）、**energy**（能耗）、以及其他（如跨自治系统的费用）。难点之一是输入本身难测——准确测量客户端到副本的延迟或带宽并不容易，因此优化结果要“打折扣地”看待。

原文精读

The placement problem itself should be split into two subproblems: that of placing replica servers, and that of placing content.

放置问题本身应拆成两个子问题：放置副本服务器，以及放置内容。

原文定位：§7.4.1，原书 p.424–426（图 7.23 放置算法分类）

7.4.2 内容复制与放置 (Content replication and placement)

副本分三类（三个同心环）：**permanent replicas**（初始副本集，数量少，如集群内的站点复制、跨 Internet 的 mirroring、分布式数据库的 shared-nothing / federated 架构）；**server-initiated replicas**（由数据拥有者发起的临时副本，如纽约的 Web 服务器遭遇某远方突发请求时在该区域装临时副本；CDN 虽由客户端请求触发，但决策权在 CDN，仍算服务器发起）；**client-initiated replicas**（即客户端缓存，管理完全交给客户端）。缓存命中（cache hit）可共享，但传统文件系统里文件很少被共享，共享缓存往往无用；Web 缓存的重要性也在下降，服务器发起复制反而更有效。

原文精读

Permanent replicas can be considered as the initial set of replicas that constitute a distributed data store.

永久副本可视为构成一个分布式数据存储的最初那批副本。

Client-initiated replicas are more commonly known as client caches.

由客户端发起的副本更常被称为客户端缓存。

原文定位：§7.4.2，原书 p.426–430（图 7.24 三类副本、Note 7.6 动态内容放置）

7.4.3 内容分发 (Content distribution)

要传播什么？三种选择：只传播通知（失效协议 invalidation，带宽省，适合写多读少）、传数据（适合读多写少，可打包日志）、传操作（active replication，各副本自己执行操作，参数小则带宽极省，但各副本更耗算力）。推送还是拉取？**push**（服务器主动推，适合强一致、多读者共享的永久/服务器发起副本）与 **pull**（客户端轮询，适合客户端缓存，代价是 cache miss 时响应变慢）各有取舍：push 要求服务器跟踪所有客户端缓存（有状态、可能很重），pull 响应时间更长。折中方案是 **lease**：服务器承诺在一段时间内主动推送，到期后客户端改为轮询；租期可按年龄、续租频率、服务器状态开销动态调整。此外还有 unicast（逐条发 N 条）与 multicast（网络代为高效分发）之别。

原文精读

An important design issue concerns what is actually to be propagated.

一个重要的设计问题在于：究竟要传播什么。

In a push-based approach, also referred to as server-based protocols, updates are propagated to other replicas without those replicas even asking for the updates.

在基于推送（也称基于服务器）的协议中，更新被传播到其他副本，而无需那些副本主动索取。

a lease is a promise by the server that it will push updates to the client for a specified time

租约是服务器的一项承诺：它将在指定时间内把更新推送给客户端。

原文定位：§7.4.3，原书 p.430–434（图 7.26 push/pull 对比）

7.4.4 管理被复制对象 (Managing replicated objects)

对分布式对象而言，以数据为中心的一致性自然表现为 entry consistency：把操作按锁分组。但复制对象要解决两个问题：一是防止同一对象的多个调用并发执行（本地加锁即可串行化）；二是保证所有副本上的状态变更相同——即需要把调用排序，让每个副本以相同顺序看到所有调用。多线程对象服务器还得让线程调度确定性，否则各副本上的执行顺序可能被打乱。另一个头疼问题是 **replicated invocations**：若对象 B 被复制，B 的每个副本都去调用 C，C 就被调用多次（若该调用转账 \$100,000，迟早有人抗议）；解决办法之一是由 B 的副本协调者只转发一份请求给 C 的各副本。

解读 注意：这里的确定性线程调度与因果/全序多播是同一件事的两面——一致性协议保证了消息的全序，操作系统层的调度不能把它打乱。

原文精读

we need to ensure that all changes to the replicated state of the object are the same.

我们需要确保对该对象被复制状态的所有变更都相同。

原文定位：§7.4.4，原书 p.434–437（图 7.27 确定性线程调度、图 7.28–7.29 复制调用）

7.5 Consistency protocols

一致性协议

一致性协议是一致性模型的具体实现。数据为中心的模型之后，是客户端为中心的模型。

7.5.1 顺序一致性：主副本协议 (Primary-based protocols)

每个数据项 x 有一个主副本 (**primary**) 协调写。**remote-write**：写转发到固定的单一服务器，读可本地；即 **primary-backup** 协议——主副本更新本地并转发给备份，收齐确认后应答。它是阻塞操作（等待较久），也可做成非阻塞（主副本更新后立即应答，再通知备份），但非阻塞更难应对故障。**local-write**：主副本迁移到发起写的进程处，适合连续多次本地写；也可用于可断连的移动计算机（断连前成为主副本、本地更新，重连后再传播）。

原文精读

Primary-backup protocols provide a straightforward implementation of sequential consistency, as the primary can order all incoming writes in a globally

主备协议为顺序一致性提供了一种直接的实现：主副本能以一个全局唯一的顺序排列所有到达的写。

unique order.

原文定位：§7.5.1，原书 p.438–440（图 7.30–7.31）

7.5.2 顺序一致性：复制写协议（Replicated-write protocols）

写可发生在多个副本上。**active replication**：操作转发给所有副本执行，需全序多播，常用中央 **sequencer** 分配序号（可扩展性差，可用多个 sequencer + Lamport 时间戳）。**quorum-based** (Thomas 1979; Gifford 1979)：读写都要取得多个服务器许可。读配额 N_R 、写配额 N_W 须满足两条约束： $N_R + N_W > N$ （防读写冲突，读集与写集必相交）、 $N_W > N/2$ （防写写冲突）。若取 $N_R = 1$ 、 $N_W = N$ 即 **ROWA** (read one, write all)，读极快但写要写全部。

原文精读

The first constraint is used to prevent read-write conflicts, whereas the second prevents write-write conflicts.

第一条约束用于防止读写冲突，第二条则防止写写冲突。

原文定位：§7.5.2，原书 p.440–443（图 7.32 三种投票选择、ROWA）

7.5.3 缓存一致性协议（Cache-coherence protocols）

缓存是复制的特例——通常由客户端而非服务器控制。可按两个标准分类：一致性检测时机（静态：编译器分析插指令；动态：运行时检查，进一步分为访问时校验、乐观地先执行后校验、提交时才校验）；一致性强制策略（禁止缓存共享数据 / 服务器发送失效 / 传播更新；写透 write-through vs 写回 write-back）。NFS 是经典例子：客户端缓存文件数据、属性、文件句柄与目录，用租约 (lease) 自动失效；NFSv4 还引入 **open delegation** 把部分权限下放给客户端，并需服务器能通过 **callback** 召回委托。

原文精读

Caches form a special case of replication, in the sense that they are generally controlled by clients instead of servers.

缓存是复制的一个特例：它们通常由客户端而非服务器控制。

原文定位：§7.5.3，原书 p.443–446（图 7.33 NFS 客户端缓存、图 7.34 callback, Note 7.9）

7.5.4 实现连续一致性（Implementing continuous consistency）

限定数值偏差：每个写 W 关联数值 $val(W)$ ，服务器用日志 L_i ，用 $T_W[i, j]$ 表示“由 S_j 发起、被 S_i 执行的写之和”；给每个 S_i 设上界 δ_i ，使 $v - v_i \leq \delta_i$ ，靠流行病 (epidemic) 协议快速传播写。限定陈旧度：用实时向量时钟 RVC_k ，当 $t_k - RVC_k[i]$ 逼近上限时主动拉取来自 S_i 的较新写（拉取优于推送，因为不知道最大传播时间）。限定排序偏差：把暂定写放入本地队列，偏差即队列最大长度；超限时不再接受新写，转而与其他服务器协商这些暂定写的全局顺序。

原文定位：§7.5.4，原书 p.446–448

7.5.5 实现客户端一致性 (Implementing client-centric consistency)

朴素实现：每个写分配全局唯一标识，为每个客户端维护读集（与读操作相关的写）与写集（客户端执行的写）。单调读时服务器核对读集并补齐；单调写时按写集补齐再执行新写；读你所写要求服务器已见过写集中的全部写；写跟随读则先让服务器追上读集。问题在于集合可能极大，可用会话 (session) 分组，并用向量时间戳压缩表示 ($SVC_A[j]$ 表示会话 A 中来自 S_j 的最新写时间戳)，这正是向量时间戳在分布式系统中表示历史的优雅之处。

原文精读

In a naive implementation of client-centric consistency, each write operation W is assigned a globally unique identifier.

在客户端为中心一致性的朴素实现中，每个写操作 W 被赋予一个全局唯一的标识符。

原文定位：§7.5.5，原书 p.448–450 (Note 7.10 用向量时间戳提高效率)

7.6 Example: Caching and replication in the Web

实例：Web 中的缓存与复制

Web 可以说是史上最大的分布式系统。客户端缓存有两处：浏览器缓存，以及站点常配的 **Web proxy** (可缓存结果供其他客户端复用，即共享缓存)。ISP 也常放缓存以降低流量、改善性能，但请求路径上缓存过多，未命中时反而增加延迟。一致性协议方面：条件式 HTTP GET 加 **If-Modified-Since** 头 (pull 式)；Squid 按 $T_{\text{expire}} = \alpha(T_{\text{cached}} - T_{\text{last_modified}}) + T_{\text{cached}}$ ($\alpha = 0.2$) 设过期时间，过期前不回服务器；服务器主动发失效 (invalidation) 则需跟踪大量代理，有扩展性问题，故 Web 代理缓存很少用失效协议。至于动态内容，边缘服务器有三种方案：全复制 (源服务器数据库，适合更新少、查询复杂)、内容感知缓存 (content-aware，按查询模板组织本地库，需记录记录与模板的对应关系)、内容盲缓存 (content-blind，对查询求哈希、命中即返回，计算省但存储冗余大)。CDN 则像一个反馈控制系统：指标估计 (延迟、带宽、跳数、网络用量、一致性、财务)、触发适应、采取放置/一致性/路由措施。

原文精读

The Web is arguably the largest distributed system ever built.

Web 可以说是迄今建造过的最大的分布式系统。

the server may need to keep track of many proxies, inevitably leading to a scalability problem.

服务器可能需要跟踪大量代理，这不可避免地导致可扩展性问题。

原文定位：§7.6，原书 p.451–458 (图 7.35 协作缓存、图 7.36 CDN 反馈控制、图 7.37 边缘服务器方案，Note 7.11)

7.7 Summary

小结

- 复制数据的两大理由：提高可靠性与提高性能；代价是一致性问题——一份副本被更新就与其余不同，必须传播更新，且传播方式可能严重拖累性能。
- 唯一出路是放松一致性。连续一致性用数值偏差、陈旧度偏差、排序偏差三个维度来界定可容忍的不一致。

- 操作的一致性排序是许多模型的基础：顺序一致性给程序员最自然的语义（所有写以同一顺序被所有人看到）；因果一致性较弱但仍有用。
- 更弱的模型把读写序列用锁“括起来”（如 entry consistency），实现更高效。
- 以客户端为中心的模型（单调读、单调写、读你所写、写跟随读）针对移动客户端在不同副本间切换的场景，保证副本对该客户端“补齐”。
- 更新传播要决定：传播什么（通知/操作/状态）、传给谁、由谁发起（push/pull）、以及用单播还是多播。
- 一致性协议：顺序一致性有主副本协议与复制写协议（active replication 与 quorum）。Web 与 CDN 是这些技术的集大成应用。

实践清单

- 读原书第 7 章，重点看图 7.3（顺序一致性反例）、7.5（有效执行序列）、7.13（2PC 式调度）、7.24（三类副本）、7.32（投票选择）、7.36（CDN 反馈控制）。
- 用 ENTER_CS/LEAVE_CS 的思想，判断你熟悉的数据库事务属于哪种粒度的一致性（单操作 $\approx$ 顺序一致性，多操作 $\approx$ entry consistency）。
- 对比“最终一致性”与“强最终一致性（CRDT）”：在一个购物车场景里，删除操作会破坏单调性，你会如何拆分 add/delete 两个可增长集合？
- 给你所在系统画一张副本放置图，标出 permanent / server-initiated / client-initiated 三类副本各有哪些。
- 解释为什么 push 协议要求服务器有状态，以及 lease 如何在 push 与 pull 之间动态切换（年龄、续租频率、服务器负载三种租期策略）。

Fault Tolerance

分布式系统区别于单机的根本一点是部分失效 (*partial failure*): 一部分出故障, 其余仍 (看似) 正常。容错的目标是让系统在故障发生时仍以可接受的方式继续运行并自动恢复。本章从基本概念与失效模型 (*crash / omission / timing / response / arbitrary*) 出发, 讲进程组与复制、共识 (*flooding consensus*、*Raft*、*Paxos*、拜占庭协议 *PBFT*、区块链共识), 再讲可靠通信 (*RPC*/组通信/原子多播)、分布式提交 (*2PC/3PC*) 与恢复 (检查点与消息日志)。一条主线贯穿始终: 冗余是容错的关键, 但冗余会带来协调开销。

8.1 Introduction to fault tolerance

容错引论

容错与可依赖性 (**dependability**) 密切相关, 后者包含四个要求: 可用性 (**availability**) —— 系统随时就绪的概率 (某一瞬间); 可靠性 (**reliability**) —— 在一段时间区间内连续无故障 (与可用性不同: 每小时随机宕 1 毫秒可用性高达 99.9999%, 但仍“不可靠”; 每年 8 月固定停机两周则可靠但可用性仅 96%); 安全性 (**safety**) —— 暂时失效也不发生灾难; 可维护性 (**maintainability**) —— 失效后易于修复。要区分三个词: **failure** (系统未兑现承诺)、**error** (可能导致 **failure** 的那部分状态)、**fault** (**error** 的成因)。**fault** 分瞬态、间歇、永久。应对故障有预防/容忍/移除/预测, 本书重点是容错: 即使存在故障也能提供服务。

8.1.1 基本概念 (Basic concepts)

失效模型: **crash failure** (服务器突然停止, 此前一直正确)、**omission failure** (漏收 *receive-omission* / 漏发 *send-omission*)、**timing failure** (响应超出规定时间区间, 太晚即 *performance failure*)、**response failure** (*value failure* 值错 / *state-transition failure* 控制流偏离)、**arbitrary failure** (也称 **Byzantine failure**, 任意响应, 最难处理)。关于能否判断进程已崩溃, 要区分异步系统 (无法从“无响应”断定崩溃, 可能只是慢) 与同步系统 (执行速度与消息延迟有界, 可断定崩溃); 现实中更合理的是 **partially synchronous**: 多数时候像同步系统, 可用超时判定崩溃, 但偶尔判错。据此 **halting** 失效从轻到重分为 **fail-stop**、**fail-noisy**、**fail-silent**、**fail-safe**、**fail-arbitrary**。

原文精读

A system is said to fail when it does not meet its promises.

当一个系统未能兑现其承诺时, 就说它失效了。

The cause of an error is called a fault.

错误的成因称为故障 (**fault**)。

The most serious are arbitrary failures, also known as Byzantine failures.

最严重的失效是任意失效, 也称为拜占庭失效。

原文定位: §8.1.1, 原书 p.463–466 (图 8.2 失效类型、Note 8.1 传统度量 MTTF/MTTR/MTBF、Note 8.2)

8.1.2 失效模型 (Failure models)

上一个子节的失效类型即失效模型；它们针对“服务器、通信信道或两者”未履行职责。如何量化一个故障的严重程度，取决于模型假设。

解读 注意 **omission** (该做未做) 与 **commission** (不该做却做了) 的区分，可避免在“是否恶意”上做主观判断——恶劣的“故障”与恶意的“攻击”在技术上往往难以分辨，这也说明可依赖性与安全性有时交织在一起。

原文精读

A crash failure occurs when a server prematurely halts, but was working correctly until it stopped.

崩溃失效指服务器过早停机，但在此之前它一直正确运行。

原文定位：§8.1.2，原书 p.466–470

8.1.3 通过冗余掩盖故障 (Failure masking by redundancy)

容错能做的就是将故障对其他进程藏起来，关键手段是冗余，共三种：信息冗余（加校验位，如 Hamming 码，可从比特错中恢复）、时间冗余（重做，如事务中止后重试、请求超时重发，对瞬态/间歇故障特别有效）、物理冗余（加设备或进程，可硬件也可软件，如复制进程）。经典的硬件例子是三模冗余 (TMR)：每级电路复制三份，后接三输入一输出的投票器，只要多数一致就输出该值；即使某个器件甚至投票器坏掉，结果仍正确。

原文精读

The key technique for masking faults is to use redundancy.

掩盖故障的关键技术是使用冗余。

原文定位：§8.1.3，原书 p.470–471 (图 8.3 三模冗余，Note 8.3)

8.2 Process resilience

进程韧性

容错的第一条路径是把若干相同进程组织成进程组：消息发给组，所有成员收到，一个成员倒下另有成员顶上。组分扁平组（所有进程平等、无单点、靠表决决策但慢）与分层组（有协调者，决策快但协调者失效则全组停摆，需重新选举）。成员管理可由组服务器集中管理（简单但有单点故障），也可分布式进行；加入/离开必须与数据消息同步（加入后必收到此后所有消息，离开后不得再收）。若组小到无法运作，还需能重建组。

8.2.1 通过进程组实现韧性 (Resilience by process groups)

引入进程组是为了让进程把“一组别的进程”当作单一抽象：P 向组 $Q = \{Q_1, \dots, Q_N\}$ 发消息时无需知道成员是谁、有几个、在哪，且这些都可能变化。基于组的复制有两种：主备（分层，主协调所有写，主崩溃则由备份选举接替，即 primary-backup）与复制写（active replication / quorum，扁平组无单点，代价是分布式协调）。复制要多少份？若进程“静默失效”， $k+1$ 份即可容忍 k 个故障（用其余那份的答案）；若出现任意失效（故障者可能发出相同错误答复），则需 $2k+1$ 份——多数派票即可胜出。

原文精读

The key approach to tolerating a faulty process is to organize several identical processes into a group.

容忍故障进程的关键方法，是把若干相同的进程组织成一个进程组。

A system is said to be k -fault tolerant if it can survive faults in k components and still meet its specifications.

若一个系统能承受 k 个组件的故障并仍满足其规格，则称它是 k -容错的。

原文定位：§8.2.1–8.2.2，原书 p.472–475（图 8.4 扁平组与分层组）

8.2.3 崩溃失效下的共识（Consensus in faulty systems with crash failures）

要让复制的进程组像单一健壮进程那样工作，必须满足一个关键假设：每个无故障进程以相同顺序执行相同命令——即组内需就“执行哪条命令”达成共识。无故障时很简单（全序多播或中央 sequencer 即可）；有故障时则棘手。**flooding consensus**：按轮进行，每轮把“已知的候选命令列表”发给所有进程，合并后按相同选择函数选一条执行；当某进程检测到别的进程崩溃时，它无法确定他人是否已收到该崩溃者的列表，于是推迟决策到下一轮，直到收到所有无故障进程的消息。更实用的是 **Raft** (Ongaro & Ousterhout 2014)：约 5 个复制服务器，一个 **leader** 决定待定操作的提交顺序，本质是 primary-backup 协议。每个客户端请求以 $\langle o, t, k \rangle$ 形式追加到 leader 日志；leader 收到多数确认后执行 o 并把最近提交索引 c 设为 $n + 1$ 。leader 崩溃后选举新 leader，server 不会投票给日志更旧的候选者；新 leader 广播日志即可补齐并提交旧操作。

原文精读

In a fault-tolerant process group, each nonfaulty process executes the same commands, in the same order, as every other nonfaulty process.

在容错的进程组中，每个无故障进程都与其他无故障进程一样，以相同顺序执行相同的命令。

Consensus is expressed in terms of these logs: committed operations have the same position in each of the respective server's logs.

共识是用这些日志来表达的：已提交的操作在每个服务器各自的日志中处于相同位置。

原文定位：§8.2.3，原书 p.475–479（图 8.5 flooding consensus、图 8.6 leader 崩溃后的恢复）

8.2.4 例：Paxos（Example: Paxos）

Paxos (Lamport, 1989 报告、1998 正式发表) 假设很弱：部分同步甚至异步、消息可能丢失/重复/乱序、损坏可检测、操作确定、进程只会崩溃不会任意失效、也不串通。参与者分三类逻辑角色：**proposer**（代表客户端提案）、**acceptor**（接受提案）、**learner**（学习并执行被选定的提案）。若多数 **acceptor** 接受了同一提案，该提案即被选定（**chosen**）。核心安全不变式：若提案 p 被选定，则任何时间戳 $ts(p') > ts(p)$ 的提案 p' 必携带同一操作。协议两阶段：**Phase 1** (prepare / promise) 用于排除“多个自认为 leader 的 proposer”的干扰，acceptor 承诺不再接受更低时间戳的提案并回报已接受的最高时间戳提案，让 proposer 采用 (**adopt**) 前任的操作；**Phase 2** (accept / learn) 中 proposer 推广单一操作，acceptor 接受后发 **learn**，learner 从多数 acceptor 收到同一 **learn** 即执行。Paxos 保证安全 (**safety**)——不会发生坏事；但只在“足够多进程存活”时保证条件活性 (**conditional liveness**)，需额外的显式领导权接管 (propose/promise) 才能持续推进。

原文精读

If a majority of acceptors accepts the same proposal, the proposal is said to be chosen.

若多数接受者 (acceptor) 接受了同一提案, 则该提案被称为已选定。

a safety property asserts that nothing bad will happen.

安全性这一性质断言: 坏事不会发生。

原文定位: §8.2.4, 原书 p.479–491 (图 8.7 Paxos 逻辑进程、图 8.8–8.13、Note 8.4 推进 Paxos)

8.2.5 任意失效下的共识 (Consensus in faulty systems with arbitrary failures)

当组员可能任意失效 (值错、控制流偏离, 甚至向不同进程说不同的话), 共识更难。拜占庭协定 (Byzantine agreement) 两条要求: **BA1**——所有无故障备份存相同值; **BA2**——若 primary 无故障, 则备份存的值恰是 primary 发出的值。结论是: 要容忍 k 个任意失效, 至少需要 $3k + 1$ 个成员 ($3k$ 不够: $k = 1$ 、3 个进程时, 无论“分歧的 primary”还是“转发错值的备份”, 都让无故障者收到 $\{T, F\}$ 而无法判断)。经典的多轮转发协议 (BAP) 可满足 BA1/BA2。实用的 **PBFT** (Castro & Liskov 2002) 用 $3k + 1$ 个副本、主备模型、消息签名, 分 pre-prepare / prepare / commit 三阶段: 某无故障副本收到含自身在内的 $2k + 1$ 条匹配 prepare (构成 prepare certificate PC) 即就操作顺序达成共识; 收到 $2k$ 条 commit 后执行并应答; 客户端取至少 $k + 1$ 个副本返回的、相同的响应。主失效时用 view-change 换视图, **HotStuff** 改进了 view change, 使其可扩展到上百个副本。

原文精读

we will show that we need at least $3k + 1$ members to reach consensus under these failure assumptions.

我们将证明: 在这些失效假设下, 要达成共识至少需要 $3k + 1$ 个成员。

BA1: Every nonfaulty backup process stores the same value.

BA1: 每个无故障的备份进程存储相同的值。

原文定位: §8.2.5, 原书 p.491–502 (图 8.14–8.17、图 8.18 PBFT 各阶段, Note 8.5–8.6)

8.2.6 区块链系统中的共识 (Consensus in blockchain systems)

区块链共识有四条要求: **Agreement** (所有无故障节点就某交易块的接受及其在链中位置达成一致, 或一致同意丢弃)、**Integrity** (每个无故障节点看到的已接受块及其位置相同)、**Termination** (每个无故障节点最终要么接受要么丢弃该包含在块中的交易)、**Validity** (若每个节点收到相同且已验证的块, 就应接受其入链)。前两条是 安全的核心, 后两条关乎活性。无许可链靠领导者选举 (选出后由 leader 决定追加哪个块); 若两个 leader 同时产生, 会出现分叉, 常见解法是丢弃较短的分支、只取较长分支。许可链常用 PBFT (或 HotStuff)。

解读 区块链把共识 (谁有权追加块) 与完整性 (密码学哈希链) 分开看, 是理解它的关键。

原文精读

Agreement: All nonfaulty nodes agree on the acceptance of a block of transactions and its position in the blockchain, or agree that it should be discarded.

一致性: 所有无故障节点就某交易块的接受及其在区块链中的位置达成一致, 或一致同意将其丢弃。

原文定位: §8.2.6, 原书 p.502–503

8.2.7 实现容错的一些局限 (Some limitations on realizing fault tolerance)

共识需要满足三条：进程产生相同输出值、每个输出值必须有效、每个进程最终必须产生输出。能否达成取决于系统假设（同步/异步、延迟是否有界、消息是否保序、单播/多播），只有其中一部分组合存在解。**FLP** 结果 (Fischer et al. 1985) 指出：若消息交付时间无已知有限上界，则即使只有一个进程（静默）失效，共识也不可解——因为任意慢的进程与已崩溃的进程无法区分。相关地，**CAP** 定理 (Brewer 2000) 说：共享数据的网络系统在 **C**（一致性，副本表现为单一最新副本）、**A**（可用性，更新最终总会被执行）、**P**（容忍分区）三者中至多满足两个。理解方式是：网络故障下两进程无法通信，允许一方接受更新就失去 **C**（得 $\{A, P\}$ ），要维持一致的假象则一方须假装不可用（得 $\{C, P\}$ ），只有能通信才能同时高可用且一致（得 $\{C, A\}$ 但无 **P**）。实践含义：分区发生时只能选择“继续推进”，同时启动恢复流程去缓解潜在不一致，如何取舍高度依赖应用（重复的数据库记录易修，重复的大额转账则不能）。

原文精读

The problem with such systems is that arbitrarily slow processes are indistinguishable from crashed ones (i.e., you cannot tell the dead from the living).

这类系统的问题在于：任意慢的进程与已崩溃的进程无法区分（也就是说，你分不清死者和生者）。

Any networked system providing shared data can provide only two of the following three properties

任何提供共享数据的网络系统，至多只能同时满足以下三个性质中的两个

原文定位：§8.2.7，原书 p.503–506（图 8.19 可达成共识的情形、CAP 定理）

8.2.8 失效检测 (Failure detection)

要掩盖故障通常得先检测故障：一个组的无故障成员要能判定谁还在、谁已离开。机制只有两种：主动发“你还活着吗？”（等待回答），或被动等待消息到来（需保证有足够通信）。理论上失效检测器花样很多，归根到底靠超时：P 探测 Q，若 Q 超时未响应，则怀疑 Q 已崩溃。同步系统中“被怀疑崩溃”等于“已知崩溃”；部分同步系统中更实际的是最终完美失效检测器：超时后怀疑，但若 Q 之后又发来消息，就停止怀疑并增大超时值。现实问题：不可靠网络易产生误判 (**false positive**)，把健康进程踢出成员表；超时机制也很粗糙。设计要点还包括：用 gossip 传递存活信息、区分网络故障与节点故障（不靠单一节点判断，而是询问其他邻居）、以及如何把成员失效通知他人（如 FUSE 用生成树传播，一个节点失效会被递归提升为组失效通知）。

解读 失效检测是 Consensus、原子多播、2PC 等一切容错协议的共同地基：判错的代价可能是把正确进程排除，从而破坏安全性。

原文精读

Failure detection is one of the cornerstones of fault tolerance in distributed systems.

失效检测是分布式系统容错的基石之一。

原文定位：§8.2.8，原书 p.506–508（Note 8.7 完美失效检测器）

8.3 Reliable client-server communication

可靠的客户端-服务器通信

容错也涉及通信失效：信道同样会出现 crash、omission、timing、arbitrary 失效。实践中主要掩盖 crash 与 omission；arbitrary 失效常表现为重复消息（消息在网中被缓冲过久，重传后又被注入）。

8.3.1 点对点通信 (Point-to-point communication)

可靠传输协议（如 TCP）用确认与重传掩盖 omission 失效（丢包），对客户端完全隐藏；但连接崩溃不被掩盖——连接突然断开时客户端通常收到异常，只能自动重连（假设对方仍或再次可响应）。

原文精读

TCP masks omission failures, which occur in the form of lost messages, by using acknowledgments and retransmissions.

TCP 通过确认与重传来掩盖以丢包形式出现的遗漏失效。

原文定位：§8.3.1，原书 p.508–509

8.3.2 存在失效时的 RPC 语义 (RPC semantics in the presence of failures)

RPC 的目标是让远程调用看起来像本地调用，但出错时差异难掩盖。五类失效：客户端找不到服务器、请求消息丢失、服务器收到请求后崩溃、应答消息丢失、客户端发送后崩溃。（1）找不到服务器：抛异常或信号，但破坏透明性。（2）请求丢失：最简单，超时重发；问题是无法区分“请求丢了”与“应答丢了”，后者会导致重复执行。（3）服务器崩溃：正常是先执行后应答；若执行完崩溃则须上报失败，若执行前崩溃则可重发，但客户端操作系统无法区分。三种哲学：**at-least-once**（重试到有应答，至少执行一次）、**at-most-once**（立刻放弃，至多执行一次）、不保证（易实现）。**exactly-once** 通常无法实现：服务器两种策略（先发完成消息后处理 / 先处理后发）与客户端四种重发策略，共 8 种组合，没有一种在所有事件序列下都正确——要么请求永久丢失，要么被执行两次。（4）应答丢失：靠重发；幂等（**idempotent**）操作可安全重复，非幂等（如转账）则需给请求编号、由服务器跟踪最近序号以区分原请求与重传。（5）客户端崩溃：留下孤儿计算（**orphan**），浪费算力、可能锁住资源。四种应对：orphan extermination（先写日志、重启后杀掉）、reincarnation（按 epoch 广播，收到即杀所有远程计算）、gentle reincarnation（先尝试找回 owner 再杀）、expiration（每个 RPC 给固定时间 T ，超时须显式续期）。实践中最优往往是什么都不做，而是把客户端恢复到能处理任何应答的状态（见 §8.6 检查点与消息日志）。

原文精读

it can be shown that for any combination either the request is lost forever, or carried out twice.

可以证明：对任意一种组合，请求要么被永久丢失，要么被执行两次。

Such a computation is called an orphan (computation).

这样的计算称为孤儿计算（orphan）。

原文定位：§8.3.2，原书 p.509–514（图 8.20 服务器三种情形、图 8.21 客户端/服务器策略组合，Note 8.8）

8.4 Reliable group communication

可靠的组通信

复制进程让可靠多播 (**reliable multicast**) 变得重要：消息要交付给组内每个成员。关键区分 **receive** (消息处理组件收到) 与 **deliver** (交付给上层核心功能)。当进程可能失效时，可靠组通信要求消息被所有无故障成员收到并交付。

8.4.1 引言 (Introduction)

最简单方案：发送者给每条多播消息编号并在 history buffer 中保留，直到每个接收者确认；接收者若收到比缺失序号更靠后的消息，就发否定确认请求重传。设计权衡包括确认可捎带、重传可用点对点或多播。

原文精读

group communication is considered to be reliable when it can be guaranteed that a message is received by and subsequently delivered to all nonfaulty group members.

当可以保证一条消息被所有无故障组成员收到并随后交付时，组通信才被认为是可靠的。

原文定位：§8.4.1，原书 p.515–518 (图 8.22 接收与交付、图 8.24)

8.4.2 可靠多播的可扩展性 (Scalability in reliable multicasting)

给 N 个接收者返回确认会让发送者被反馈淹没，即反馈内爆 (**feedback implosion**)。解法之一是只回否定确认 (更易扩展，但无硬保证)。两个代表性方案：**feedback suppression** (SRM)：接收者只在发现缺失时多播否定确认、并加随机延迟，听到别人已请求就抑制自己，使送到发送者的请求趋于一条；分层方案：把接收者分成子组、组成树，每个子组选一个本地协调者，协调者之间用可靠连接转发消息并聚合反馈——一个 ack/nack 就聚合了大量反馈。还有基于 gossip 的 push-pull 反熵方案，天然健壮。

原文精读

With many receivers, the sender may be swamped with such feedback messages, which is also referred to as a feedback implosion.

当接收者众多时，发送者可能被这类反馈消息淹没，这也被称为反馈内爆。

原文定位：§8.4.2，原书 p.518–522 (图 8.25 反馈抑制、图 8.26 分层可靠多播)

8.4.3 原子多播 (Atomic multicast)

原子多播要求消息要么交付给所有无故障成员，要么谁都不交付 (**all-or-nothing**)。例：复制数据库把更新多播给各副本，若发送者 P 在多播完成前崩溃，有的副本会执行、有的不会，数据库就不一致了，必须避免。虚拟同步 (**virtual synchrony**) 把多播与组成员变化联系起来：每条多播消息关联一个交付列表 (即发送者当时的 group view)，视图变化 (**view change**) 像一道屏障，不允许任何多播穿过——所有在途多播要么在视图变化生效前被所有成员交付，要么全不交付。消息排序分四种：无序、FIFO 序、因果序、全序；原子多播 = 全序交付 + all-or-nothing。Isis 的实现用稳定消息 (被所有成员收到后才交付) 与 flush 消息来安装新视图。

原文精读

what is often needed in a distributed system is the guarantee that a message is delivered to either all (nonfaulty) group members or to none.

a view change acts as a barrier across which no multicast can pass.

分布式系统中常常需要的保证是：一条消息要么交付给所有（无故障的）组成员，要么谁都不交付。

视图变化就像一道屏障，任何多播都无法穿过它。

原文定位：§8.4.3，原书 p.522–528（图 8.27 虚拟同步多播、图 8.28–8.30 排序、图 8.31 视图变化，Note 8.9）

8.5 Distributed commit

分布式提交

原子多播是一类更一般问题——分布式提交的特例：让进程组每个成员都执行某操作，或都不执行。最简的单阶段提交由协调者直接通知参与者执行，缺点是无法反馈失败。常用的是两阶段提交（2PC）：第一阶段（voting）协调者发 `vote-request`，参与者回 `vote-commit` 或 `vote-abort`；第二阶段（decision）协调者收齐后广播 `global-commit` 或 `global-abort`，参与者据此本地提交/中止。2PC 的参与者与协调者都有阻塞等消息的状态，需用超时处理崩溃：参与者处于 READY 等不到全局决定时，可询问其他参与者（若对方已 COMMIT/ABORT/INIT 即可决定；若对方也 READY 则无法决定）。当所有参与者都 READY、协调者却崩溃时，协议必须阻塞等待协调者恢复，故 2PC 是阻塞式提交协议。为避免阻塞，可用多播让接收者立即转发，或改三阶段提交（3PC）：多一个 `pre-commit` 阶段，满足“不存在可直接到 COMMIT/ABORT 的单状态”等条件，在 fail-stop 假设下不阻塞（但实践中 2PC 阻塞很少发生，3PC 用得不多）。

原文精读

The distributed commit problem involves having an operation being performed by each member of a process group, or none at all.

For this reason, 2PC is also referred to as a blocking commit protocol.

分布式提交问题涉及让进程组的每个成员都执行某操作，或都不执行。

正因如此，2PC 也被称为阻塞式提交协议。

原文定位：§8.5，原书 p.528–535（图 8.32 2PC 状态机、图 8.33 参与者动作、图 8.34–8.35 Python 伪码，Note 8.10–8.11）

8.6 Recovery

恢复

故障发生后，出错进程须能恢复到正确状态。两种错误恢复：**backward recovery**（从当前错误状态回到先前正确状态，需检查点）与 **forward recovery**（把系统带到可继续执行的新正确状态，前提是事先知道可能发生哪些错误）。例：丢包后重传是 backward（回到重发前状态）；**erasure correction**（用 (n, k) 块纠删码从其他包重构丢失包）是 forward。backward 通用、可集成进中间件，但代价高、且有些状态无法回滚（取走的钱、`/bin/rm -fr *`），也不能保证同类故障不再发生。

8.6.1 引言 (Introduction)

由于检查点昂贵，容错系统常把它与消息日志结合：发消息前记录（sender-based logging）或收到消息先记录再交付（receiver-based logging）。只靠检查点时，重启后的行为可能与故障前不同（通信时间不确定导致消息顺序不同）；配合消息日志则能重放事件，更便于与外部世界交互——例如用户输入导致错误时，用较旧检查点 + 重放到输入点比频繁打检查点更高效。

原文精读

In backward recovery, the main issue is to bring the system from its present erroneous state back into a previously correct state.

在向后恢复中，主要问题是将系统从当前的错误状态带回先前的正确状态。

原文定位：§8.6.1，原书 p.536–538

8.6.2 检查点 (Checkpointing)

需要记录一致全局状态，即 **distributed snapshot**：若某进程 P 记录了收到某消息，则应有进程 Q 记录其发送——消息总有来处。尽量恢复到最近的分布式快照，即 **recovery line**。**coordinated checkpointing**：所有进程同步写本地存储，天然全局一致（用两阶段阻塞协议：协调者发 checkpoint-request，各进程打检查点、排队后续消息并确认，收到全部确认后再发 checkpoint-done 放行）；可只通知依赖协调者的进程，得到 incremental snapshot。**independent checkpointing**：各自随时打点，但恢复时可能级联回滚，甚至触发 **domino effect**（一直退回初始状态）；需记录区间依赖 $INT_i(m) \rightarrow INT_j(n)$ ，计算复杂。由于真正瓶颈是写稳定存储的开销而非协调本身，更简单的 coordinated checkpointing 反而更常用。

原文精读

In a distributed snapshot, if a process P has recorded the receipt of a message, then there should also be a process Q that has recorded the sending of that message.

在分布式快照中，若进程 P 记录了某消息的接收，那么就应存在进程 Q 记录了该消息的发送。

原文定位：§8.6.2，原书 p.538–540（图 8.37 恢复线、图 8.38 domino effect）

8.6.3 消息日志 (Message logging)

若消息传输可重放，就能从某个检查点出发、重放其后所有消息达到一致状态，从而减少检查点数量。前提是 **piecewise deterministic execution model**：每段执行从一次非确定性事件（如收到消息）开始，之后完全确定；只要记录所有非确定性事件即可完整重放。核心风险是孤儿进程（**orphan process**）：它在别的进程崩溃后存活，但恢复后状态不一致。用记号 $DEP(m)$ （依赖消息 m 交付的进程集）与 $COPY(m)$ （持有 m 尚未可靠存储的进程集）可精确定义： Q 是孤儿 $\iff \exists m : Q \in DEP(m)$ 且 $COPY(m) \subseteq FAIL$ 。两类日志协议：**pessimistic**（保证每条非稳定消息至多一个进程依赖它，即交付前先把 m 写入可靠存储，实现简单，实践中首选）与 **optimistic**（崩溃后再回滚孤儿进程，需跟踪依赖，实现更复杂）。

原文精读

The basic idea underlying message logging is that if

消息日志的基本思想是：如果消息的传输可以被

the transmission of messages can be replayed, we
can still reach a globally consistent state

重放，我们仍能到达一个全局一致的状态

原文定位：§8.6.3，原书 p.541–542（图 8.39 错误重放导致孤儿进程，Note 8.12）

8.7 Summary

小结

- 容错 = 在故障存在时仍能继续运行；冗余是关键，包括信息、时间、物理三类冗余。
- 失效模型：crash、omission、timing、response、arbitrary（拜占庭）。是否能从“无响应”断定崩溃，取决于系统是异步、同步还是部分同步。
- 进程组把多个进程伪装成一个健壮进程。无故障进程必须以相同顺序执行相同命令，即达成共识。
- **Paxos** 用 $2k+1$ 个服务器实现 k -容错（处理崩溃失效）；处理任意失效则需 $3k+1$ 个服务器（PBFT）。
- 可靠组通信：组小则易实现，组大则扩展性成问题，关键是减少反馈消息（feedback suppression、分层协调者）。原子多播再用 all-or-nothing + 全序交付，可用虚拟同步精确表述。
- 分布式提交：2PC 是阻塞协议，3PC 在 fail-stop 下不阻塞。
- 恢复靠检查点；为减少检查点开销，常与消息日志结合，通过重放达到一致状态（pessimistic vs optimistic 日志）。

实践清单

- 读原书第 8 章，重点看图 8.2（失效类型）、8.5（flooding consensus）、8.7–8.13（Paxos 演进）、8.18（PBFT 阶段）、8.19（可达成共识的情形）、8.32（2PC 状态机）、8.38（domino effect）。
- 手推一遍：为什么 3 个进程无法容忍 1 个任意失效，而 4 个可以？分别画出“primary 说谎”和“backup 说谎”两种情形。
- 用自己的话解释 CAP 定理，并给一个“宁可牺牲 C 保 A、P”与一个“宁可牺牲 A 保 C、P”的真实场景。
- 把 Raft 的 leader 日志与 Paxos 的 accept/promise 对应起来，找出二者的共同安全不变式。
- 检查你所在系统：是否使用了检查点 + 消息日志？若是纯检查点，找出一个“只回滚会出错”的场景。
- 写一段伪代码，实现 2PC 参与者在 READY 状态超时后的终止协议（询问其他参与者并处理 COMMIT/ABORT/INIT/READY 四种回答）。

Security

安全很难，因为它要满足的是一个否定的目标：防止一切未授权行为，而“证明做到了”几乎不可能。本章围绕两大机制展开：认证（*authentication*，先确认对方身份）与授权（*authorization*，再限制其能做的操作）；在此之上还涉及完整性与机密性，二者都高度依赖密码学（*cryptography*）。技术之外，设计原则、可信计算基（*TCB*）、隐私（*GDPR*），以及信任（*trust*）、区块链、监控与入侵检测都会涉及。贯穿全章的主线是：安全必须从设计之初就内建，而非事后补丁。

9.1 Introduction to security

安全引论

安全与可依赖性密切相关，但还需加入机密性（**confidentiality**）（信息只向授权方披露）与完整性（**integrity**）（资产只能被授权方式更改，且不准确可被检测与恢复）。威胁分三大类：未授权信息披露、未授权信息修改、未授权拒绝使用，分别对应机密性、完整性与可用性。构建安全系统不能只喊口号，先要有一份 **security policy**：精确规定哪些实体（用户、服务、数据、机器等）可以做哪些、禁止哪些动作；然后才谈 **security mechanism** 如何执行策略，共四类：加密（*encryption*，实现机密性，也支持完整性校验）、认证、授权、监控与审计（*monitoring and auditing*）。审计虽不直接提供保护，但对事故分析与威慑攻击者很有用。

9.1.1 安全威胁、策略与机制（**Security threats, policies, and mechanisms**）

一个安全策略精确描述实体被允许与禁止的动作；策略定下后，就可用四类机制去执行它。加密是基础：既提供机密性，也能用于检测数据是否被改动，从而支撑完整性。认证用于核实用户/客户端/服务器/设备等所声称的身份；认证之后才谈授权（如医疗数据库按角色决定读哪些记录、改哪些字段、增删哪些记录）。

原文精读

Confidentiality refers to the property that information is disclosed only to authorized parties.

机密性指信息只向授权方披露这一性质。

A security policy describes precisely which actions the entities in a system are allowed to take and which ones are prohibited.

安全策略精确描述了一个系统中的实体被允许采取哪些动作、被禁止采取哪些动作。

原文定位：§9.1.1，原书 p.547–548

9.1.2 设计问题（**Design issues**）

安全原则（Saltzer & Schroeder 1975，Smith 2012 总结）：**fail-safe defaults**（默认设置即应提供良好保护，如每台边缘设备出厂带唯一强密码）、**open design**（不搞“security by obscurity”，机制应可公开

审查)、**separation of privilege** (关键操作不能由单一实体完全控制, 如双人各持一把钥匙)、**least privilege** (进程只持有最少权限, 如 Unix 中用 `sudo` 显式提权)、**least common mechanism** (多个组件需要同一机制时提供同一实现)。另有三条: *economy of mechanism*、*complete mediation*、*psychological acceptability*。

安全机制的层次: 网络层有 **VPN** (隧道并加密), 传输层有 **TLS** (HTTPS 的基础), 操作系统服务层有 **SSH** (自带安全协议、不依赖传输层安全), 中间件层有 **Kerberos**、**secure RPC** 等, 应用层则有端到端安全 (WhatsApp、Signal、Telegram; 或先用客户端加密再上云, 如 BoxCryptor)。可信计算基 (**TCB**) 是“为执行某安全策略所必需且充分、因而必须被信任”的全部安全机制, 涵盖固件、硬件、软件乃至人; **TCB** 越小越好。难点在于划分“可信/不可信模块”本身就不简单 (ownCloud 的脚本集例子), 也涉及 **honest-but-curious** 这类假设是否被证明成立。隐私 (**privacy**) 与机密性相关但不同: 机密性是“不被未授权披露”, 隐私则是 Nissenbaum 所说的“个人信息恰当流动的权利”(谁在何时、以何种方式看到, 并可停止与撤回)。相关方向有 Solid (个人数据归个人控制)、GDPR, 以及用 **shard** 与数据流视角设计可撤回的系统。

原文精读

A TCB is the set of all security mechanisms in a (distributed) computer system that are necessary and sufficient to enforce a security policy, and that thus need to be trusted.

The smaller the TCB, the better.

TCB 是 (分布式) 计算机系统中为执行某项安全策略所必需且充分、因而必须被信任的全部安全机制之集合。

TCB 越小越好。

原文定位: §9.1.2, 原书 p.548–554 (图 9.1 分层安全机制、图 9.2 GDPR 对数据库的影响)

9.2 Cryptography

密码学

发送者 S 把消息 m 加密成不可读的 m' (ciphertext), 接收者 R 再解密回 m 。用 $C = E_K(P)$ 表示以密钥加密明文 P 得密文 C , $P = D_K(C)$ 表示解密。加密能对抗三类攻击: 拦截/窃听 (无密钥则看到不可读数据)、修改 (改密文需先解密再正确重加密)、插入伪造消息。

解读 一个常被忽视的元信息: 消息“是否在传输”本身有时就有价值。

9.2.1 基础 (Basics)

加密与解密由以密钥为参数的密码方法完成。明文记为 P 、密文记为 C , $C = E_K(P)$ 与 $P = D_K(C)$ 给出密文/明文/密钥的关系。

原文精读

Encryption and decryption are accomplished by using cryptographic methods parameterized by keys

加密与解密是通过以密钥为参数的密码学方法来完成的

原文定位: §9.2.1, 原书 p.555–556 (图 9.3 窃听者与入侵者)

9.2.2 对称与非对称密码系统 (Symmetric and asymmetric cryptosystems)

对称密码 (secret-key / shared-key) 加解密用同一密钥 ($K_{A,B}$)；非对称密码 (public-key) 用一对不同密钥，一个公开、一个保密 (PK_A / SK_A)。要让消息保密，用接收者公钥加密 ($C = PK_B(m)$)；要证明来源，用发送者私钥加密 ($C = SK_A(m)$)，即数字签名的基础。同态加密 (homomorphic encryption) 使“对明文的数学运算”等价于“对密文的运算”，让不可信服务器能在密文上做计算；全同态 (FHE) 性能太慢，部分同态 (PHE) 支持加法或乘法、有高效实现。Note 9.1 用 **Bloom filter** + 同态加密在城市人流统计中做到了“计数但不追踪个人”。

原文精读

One of the keys in an asymmetric cryptosystem is kept private; the other is made public.

非对称密码系统中的一把密钥被保密，另一把则被公开。

when using homomorphic encryption, mathematical operations on plaintext can also be performed on the corresponding ciphertext.

使用同态加密时，对明文所做的数学运算也可以在对应的密文上执行。

原文定位：§9.2.2，原书 p.557–560 (Note 9.1 Bloom filter 计数)

9.2.3 哈希函数 (Hash functions)

哈希函数 $h = H(m)$ 把任意长度消息映射为固定长度位串。密码学哈希 (trapdoor) 三性质：单向性 (由 h 求 m 计算上不可行)、弱抗碰撞 (给定 m 与其 h ，难找 $m' \neq m$ 使哈希相同)、强抗碰撞 (仅给定 H ，难找任意不同 m, m' 使哈希相同)。经典应用是存储口令的哈希而非明文。数字签名用消息摘要 (message digest) 降低成本：Alice 计算 $sig = SK_A(H(m))$ 随消息发出；Bob 用 Alice 公钥解密摘要并自行计算 $H(m)$ ，两者相符即确认来自 Alice，且 Alice 无法抵赖；代价是私钥泄漏或换钥会让签名失效，可能需要中心权威记录换钥时间。

原文精读

First, they are one-way functions, meaning that it is computationally infeasible to find the input m that corresponds to a known output h .

首先，它们是单向函数，意味着要找到与已知输出 h 相对应的输入 m 在计算上是不可行的。

原文定位：§9.2.3，原书 p.560–562 (图 9.4 记号表)

9.2.4 密钥管理 (Key management)

密钥建立：SSH 用 `ssh-keygen` 生成 (public, private) 密钥对、`ssh-copy-id` 分发公钥；会话密钥 (session key) 是单次会话的临时共享密钥，会话结束即丢弃 (长期密钥加解密越多越易被破解)。跨不安全信道建立共享密钥可用 **Diffie-Hellman** 密钥交换：双方各自保密 x, y ，交换 $g^x \bmod p$ 与 $g^y \bmod p$ ，各自算出 $g^{xy} \bmod p$ ，而无需公开私密数；实践中常用 ephemeral DH (每次会话换密钥)。密钥分发：对称密钥须经安全信道 (互相认证 + 机密性) 或带外 (out-of-band) 分发；公钥可明文传输但必须经认证信道，依靠 **public-key certificate** (由 CA 用其私钥 SK_{CA} 签名，证书 = 公钥 + 标识 + CA 标识 + 签名)，客户端用 CA 公钥验签。证书需撤销 (CRL 或限制有效期，如 Let's Encrypt 自动续期)。替代 CA 的是 **web of trust** (PGP)：人们当面交换并互相签名公钥，可给签名加权 (需 k 个信任者签名才接受)。

原文精读

establishing and distributing keys is not a trivial matter.

密钥的建立与分发绝非小事。

An elegant scheme that is widely applied for establishing a shared key across an insecure channel is the Diffie-Hellman key exchange.

一种被广泛应用、用于在不安全信道上建立共享密钥的优雅方案是 Diffie-Hellman 密钥交换。

原文定位：§9.2.4，原书 p.562–570（图 9.5 DH 交换、图 9.6 不经意传输、图 9.7 密钥分发，Note 9.2 MPC、Note 9.3 证书生命周期）

9.3 Authentication

认证

认证是核实某人/某软件/某设备所声称的身份。四种手段：知道什么（口令、PIN）、拥有什么（身份证、手机、软件令牌）、是什么（静态生物特征，如指纹、面部）、做什么（动态生物特征，如声音、打字节奏）。单因子认证只用其一；实践中越来越多用多因子认证。认证通常只在访问开始时做一次，但这不足以保证整个会话都是同一客户端（如 Alice 登录后 Bob 借用），因此需要持续认证（**continuous authentication**），例如用设备位置/邻居信息判断。

9.3.1 认证引论（Introduction to authentication）

四种认证手段（know / have / is / does）与多因子、持续认证的关系如上。

解读 认证与授权是两件事：认证回答“你是谁”，授权回答“你能做什么”。

原文精读

Authentication is used to verify the claimed identity of a user, client, server, host, device, and so on.

认证用于核实用户、客户端、服务器、主机、设备等所声称的身份。

原文定位：§9.3.1，原书 p.571–572

9.3.2 认证协议（Authentication protocols）

认证与消息完整性不可分割：只认证不保证完整性，或只保证完整性不认证，都无意义。基于共享密钥：**challenge-response**——Bob 发随机挑战 R_B ，Alice 回 $K_{A,B}(R_B)$ ，Bob 解密见 R_B 即知对方是 Alice；再反向挑战 R_A 实现双向认证。把 5 条消息“优化”成 3 条会引入 **reflection attack**（攻击者 Chuck 让 Bob 替自己加密挑战），教训是不要给未经认证者泄露加密响应、双方挑战应不同（发起者用奇数、响应者用偶数）。密钥分发中心（**KDC**）解决可扩展性： N 台主机只需 N 把密钥（而非 $N(N-1)/2$ ）。**Needham-Schroeder** 协议用 **nonce**（一次性随机数）把消息绑定，防重放；并用 **ticket**（如 $K_{B,KDC}(K_{A,B})$ ）让 Alice 自己联系 Bob。公钥认证则不需要 KDC，双方持彼此公钥，用挑战-响应建立会话密钥。会话密钥的好处：减少长期密钥的使用（“用久易被破”）、防重放、泄漏时只影响单个会话。**Kerberos** 基于 Needham-Schroeder，含 **AS**（认证服务器，处理登录）与 **TGS**（票据授予服务，颁发票据），提供单点登录（**single sign-on**）：登录一次即可访问多个服务。**TLS 1.3** 用 ephemeral Diffie-Hellman 建立共享密钥，再由服务器用 CA 签名的证书证明身份，派生应用层密钥。

原文精读

authentication and message integrity cannot do without each other.

认证与消息完整性二者缺一不可。

This scheme establishes what is known as single sign-on.

该方案建立了所谓的单点登录。

原文定位: §9.3.2, 原书 p.572-584 (图 9.8 共享密钥认证、图 9.10 reflection attack、图 9.13 Needham-Schroeder、图 9.16-9.18 Kerberos 与 TLS, Note 9.4-9.5)

9.4 Trust in distributed systems

分布式系统中的信任

认证解决“证明你是你”，但证明有多可靠、接收方是否接受，就进入信任的范畴。信任也因区块链/分布式账本与（半）自动化决策（尤其机器学习）而更受关注：在无法理解决策过程时仍要信任它，是最大的问题。

9.4.1 面对拜占庭失效时的信任 (Trust in the face of Byzantine failures)

当 P 依赖 Q 且 Q 可能不再符合预期时，信任才成为问题。**Trust** = “一个实体所持有的、关于另一实体会按特定预期执行特定动作的保证”。一旦预期被显式规定，也许就不必再谈信任。拜占庭失效模型的妙处正在于：可以放弃对个体进程的信任，只要求进程组满足规格—— n 个进程中至多 $k \leq (n - 1)/3$ 个作恶，其余仍能达成正确一致；超出则一切作废。

原文精读

Trust is the assurance that one entity holds that another will perform particular actions according to a specific expectation.

信任是一个实体所持有的保证：另一个实体将按照某个特定预期执行特定动作。

原文定位: §9.4.1, 原书 p.586

9.4.2 信任一个身份 (Trusting an identity)

在去中心化 P2P 系统中，若不加认证就接受请求，会遭遇 **Sybil attack**：攻击者创建多重身份分别加入系统。它破坏了三条假设：标识对应至多一个实体、每个实体至多一个标识、标识永远指向同一实体且不复用。后果可轻可重（不转发查询造成 DoS、合谋删掉自己负责的文件、在 web of trust 中批量伪造背书）。密切相关的 **eclipse attack** 用合谋节点不断返回恶意链接，使诚实节点的本地列表被污染、从而被“日食”隔离。最直接的解法是中心化 **CA** 或信任链（证书再签名），但链条末端仍需被信任。区块链方案让创建多身份无利可图：PoW 需算力竞赛，PoS 需持有代币（stake）。另有 **NetFlow** 记账机制：节点 P 为候选节点 Q 计算容量 $cap(Q) = \max\{MF(Q, P) - MF(P, Q), 0\}$ （最大流之差），Sybil 之间互相“刷工作”不会增加总容量，故该机制抗 **Sybil**。

原文精读

The essence of this attack is that in distributed systems, we generally rely on the fact that when presented with a logical identity, there is just a single

这种攻击的本质在于：分布式系统中我们通常依赖这样一个事实——当面对一个逻辑标识时，只有一个与之关联的物理实体。

associated physical entity.

原文定位：§9.4.2，原书 p.586–591（图 9.19 Sybil 攻击伪码）

9.4.3 信任一个系统（Trusting a system）

区块链“无需可信第三方”的说法要拆开看：区块链本身（只读账本）与 维护它的共识协议是两回事。保证数据完整性靠哈希链：每个块含若干交易与其数据的哈希，该哈希被复制到后继块并作为其后继哈希的输入。改动块 B_i 会使 h_i 失效，攻击者须重算 h_i^* 并传播到 B_{i+1} 、进而使 h_{i+1} 失效——只有把 B_i 之后的所有块都改掉，攻击才可能成功，而账本已被海量复制，故极难。但信任账本不等于信任整个系统：实现是否被正确加固、**smart contract**（满足条件自动执行的程序）的规格是否符合意图，以及面向终端用户的中介（区块链应用的入口服务）是否可信，都仍需信任。所以区块链与其他分布式系统在“信任一个系统”上其实有很多共性。

原文精读

an attack can be successful only if all blocks
succeeding B_i are modified as well.

只有当 B_i 之后的所有块都被一并修改时，攻击才可能成功。

原文定位：§9.4.3，原书 p.591–593（图 9.20 区块链原理）

9.5 Authorization

授权

认证之后，才谈“实体能否访问资源”。通用模型：**subject**（代表用户的进程）向 **object**（封装状态、通过接口暴露操作）发请求；保护通常由一个 **reference monitor** 执行，它记录“谁可以做什么”，每次对象被调用时都要被查问，因此它自身必须防篡改（**tamperproof**）。

9.5.1 访问控制的一般问题（General issues in access control）

保护有三种思路：直接保护数据（完整性约束）、按操作规定谁能做、按用户规定谁能访问。策略分四类：**MAC**（强制访问控制，由中央按保密级别分配，如 military 的 public 到 top secret）、**DAC**（自主访问控制，对象拥有者自行设定 read/write/execute，Unix 的 owner/group/world）、**RBAC**（基于角色，如 teacher/student/dean）、**ABAC**（基于属性，粒度最细）。访问控制矩阵 $M[s, o]$ 列出 subject s 可对 object o 执行的操作；真实系统不会存稠密矩阵，而是按列分布成 **ACL**（每个对象一张访问列表）或按行分布成 **capability**（每个主体一组能力，可看作票据，需防伪造/转手）。

解读 ACL 让服务器按“你是谁”查表；capability 让客户端带着“凭证”来，服务器不必认识你——这与后文的委托（delegation）直接相关。

原文精读

Controlling the access to an object is all about
protecting the object against invocations by subjects
that are not allowed to have specific (or even any) of
the methods carried out.

控制对对象的访问，归根结底是保护该对象不被
那些不允许执行某些（乃至任何）方法的调用者
所调用。

原文定位：§9.5.1，原书 p.593–597（图 9.21 访问控制模型、图 9.22 三种保护思路、图 9.23 ACL 与 capability）

9.5.2 基于属性的访问控制 (Attribute-based access control)

ABAC 可视为其他策略的超集，但落地仍面临挑战。属性可分：用户属性（姓名、角色、部门……）、对象属性（创建者、修改时间、版本、文件类型/大小、内容相关信息）、环境属性（时间、负载、维护状态、存储属性）、连接属性（IP、会话时长、带宽、所用安全强度）、管理属性（全局策略）。**Policy Machine** 用 **assignment**（如 $u \rightarrow ua$ ，产生集合）、**prohibition**（ $denied(u, ops, \neg os)$ 表达“除 os 外都禁止”）、**obligation**（事件触发时自动动作，如 `when u reads f in fs then denied(u, {write}, not fs)` 防止信息泄露）来表达规则。它也能模拟 MAC（按安全级别 OL_i 与“读高不能写低”的 obligation）、DAC（Unix 的 `chmod/chown` 语义）与 RBAC（ $Alice \rightarrow Teachers$ ）。

原文精读

Attribute-based access control (ABAC) is a powerful access control policy that can viewed as superseding other policies.

基于属性的访问控制 (ABAC) 是一种强大的访问控制策略，可被视为对其他策略的取代/超集。

原文定位：§9.5.2，原书 p.598–601

9.5.3 委托 (Delegation)

委托（把访问权从一个进程传给另一个）是保护技术中的重要手段，可避免大量开销（保护常可本地完成）。最差的做法是直接把用户凭据交给应用；更好的做法是使用 **proxy**——一种令牌，其持有者可拥有与授权者相同或更受限的权限与特权，派生 proxy 的权限只减不增。Neuman 的方案把 proxy 分两部分：证书 $C = \{R, PK_{proxy}\}$ （权利集 + 公钥部分，由 A 签名 $sig(A, C)$ 防篡改）与私密部分 SK_{proxy} （须保密传输）。持有凭证者须能回答基于 PK_{proxy} 的“难题”以证明自己有权。注意 Alice 无需被再次咨询，Bob 可把部分权利转授给 Dave。**OAuth 2.1** 是重要的实用委托框架，含四个角色：**resource owner**（用户）、**client**（代办应用）、**resource server**、**authorization server**；流程为客户注册得 `cid`，用临时密钥 S 的哈希请求，用户确认后得授权码 `AC`，再换取 **access token** (AT)。AT 分两种：只是标识符（资源服务器每次找授权服务器核验）或签名证书（自带权利、有有效期）。

原文精读

A proxy in the context of security in computer systems is a token that allows its owner to operate with the same or restricted rights and privileges as the subject that granted the token.

在计算机系统安全的语境中，代理 (proxy) 是一种令牌，允许其持有者以与授予该令牌的主体相同或受限的权利与特权进行操作。

原文定位：§9.5.3，原书 p.601–605（图 9.24 proxy 结构、图 9.25 委托与证明，Note 9.6 OAuth）

9.5.4 去中心化授权：一个例子 (Decentralized authorization: an example)

前面方案多有中心化组件（Kerberos 的 AS/TGS，委托需在授权服务器注册），跨行政域扩展时遇到 **administrative scalability** 问题。**WAVE** 是面向多组织的可扩展去中心化授权服务，核心是 **graph-based authorization**：全局有向图，顶点代表能授予/接收权限的实体，有向边代表委托 (attestation)。B 通过展示“从 A 到 B 的有向路径”来证明其对 A 所有资源的操作权；用 **RTree** (resource tree, 资源 URI 的首元素标识负责实体) 组织策略。两个要点：图须以可扩展且去中心化方式存储；委托信息本身也需保护（不是全世界都该知道 Alice 给了 Bob 什么权限）。存储须满足四点：**proof of monotonicity**（不可伪造的 append-only 日志）、**proof of inclusion**、**proof of nonexistence**、**proof of nonequivocation**（对所

有客户端呈现同一视图)——这与区块链账本能提供的性质非常接近。数据保护用类似 Kerberos 的链式结构, 让 C 在 A、B 都无需在线的情况下证明自己有权读文件。

原文精读

Together, these requirements ensure the integrity of the global authorization graph.

这些要求合在一起, 保证了全局授权图的完整性。

原文定位: §9.5.4, 原书 p.605–608

9.6 Monitoring

监控

光有策略还不够, 还需检查策略是否充分、正确实现并被强制执行; 因此要跟踪系统中发生了什么(记录登录、资源访问等)。被动日志有助于事后分析, 但无法在损害造成前阻止。本节重点是可检测未授权活动的入侵检测 (**intrusion detection**)。

9.6.1 防火墙 (Firewalls)

防火墙把系统某部分与外界隔离, 所有出(尤其入)包先经特殊机器检查再放行, 未授权流量被丢弃; 防火墙自身必须高度受保护、规则必须自治。两类(常结合): **packet-filtering gateway** (像路由器, 按包头源/目的地址决定放行, 可保护内网 Web 服务器、或让多 LAN 只接受来自其他 LAN 的流量从而构成 VPN)、**application-level gateway** (检查消息内容, 如邮件网关按大小过滤、过滤垃圾邮件、只给外部用户提供文摘的图书馆网关); 特殊形式是 **proxy gateway** (如 Web 代理, 过滤/改写含可执行代码的页面)。局限: 防火墙只看网络流量, 不擅长认证/授权具体资源; 一旦攻击者绕过规则或被接管账户, 防火墙形同虚设。

原文精读

Essentially, a firewall disconnects any part of a distributed system from the outside world

本质上, 防火墙把分布式系统的任一部分与外部世界隔离开来

原文定位: §9.6.1, 原书 p.609–611 (图 9.26 防火墙实现)

9.6.2 入侵检测基础 (Intrusion detection: basics)

两类: **SIDS** (签名/特征检测, 比对已知攻击模式, 但对零日攻击无能为力, 越来越不够用) 与 **AIDS** (异常检测, 先建模“典型/正常”行为, 再识别异常)。AIDS 依赖机器学习, 需训练阶段收集正常数据、构造检测模型(决策树、神经网络、分类器), 再进入测试阶段。核心难题是假阴性 (**false negative**, 把异常当正常、漏报) 与假阳性 (**false positive**, 把正常当异常、误报) 的权衡: 为避免漏报而偏保守, 会导致大量误报, 令安全运营人员对告警麻木。

原文精读

Most problematic with a SIDS is when no pattern is yet available, which typically happens with new, unknown attacks, such as zero-day attacks: attacks

SIDS 最成问题的情况是尚无可用模式时, 这通常发生在新的、未知的攻击上, 例如零日攻击: 基于尚未公开漏洞的攻击。

based on vulnerabilities that have not yet been made public.

原文定位: §9.6.2, 原书 p.611–612

9.6.3 协作式入侵检测 (Collaborative intrusion detection)

大规模分布式系统中没有单一监控点能掌握全貌, 于是有 **CIDS**: 一组入侵检测系统共享检测与分析结果。传感器 (sensor) 按社区 (**community**) 分组, 各有 **community head** 收集数据并分析; 社区可重叠 (一个传感器可报多个 head)。若各社区 head 互不通信且无共用传感器, 则只是孤立的 IDS; 常见配置是 **centralized CIDS** (所有 head 报给单一实体), 更有趣的是把 head 组织成 P2P, 即 **distributed CIDS**。性能用 $\text{accuracy} = (TP + TN) / (TP + TN + FP + FN)$ 与 $\text{precision} = TP / (TP + FP)$ 度量。实验结论: 大社区精度更高、社区数在可扩展前提下应尽量少、重叠社区 (发现被多个 head 共同考虑) 有利于精度——但集中式 (单一社区) 精度最优, 代价是扩展性问题。

原文精读

a CIDS is a collection of intrusion detection systems that share detections and analyses.

CIDS 是一组共享检测结果与分析的入侵检测系统。

原文定位: §9.6.3, 原书 p.612–613

9.7 Summary

小结

- 安全要实现的是否定目标 (防止一切未授权行为); 系统应能支持多种 安全策略, 并提供相应机制。
- 五条设计原则: **fail-safe defaults**、**open design**、**separation of privilege**、**least privilege**、**least common mechanism**; 机制应分布在从 OS 到应用的各层, 并明确 **TCB** (须满足全部安全要求的可信基)。
- 隐私日益重要, GDPR 等法规促使系统从设计之初就内建安全与隐私。
- 密码学是基石: 对称/非对称加密、哈希、数字签名、密钥交换 (Diffie-Hellman)、证书与 CA; 同态加密与多方计算 (MPC) 让不可信第三方也能参与计算。
- 安全大体归结为认证与授权: 认证核实身份, 授权保护资源, 访问控制总在认证之后发生; 但能验证身份并不足以建立信任。
- 面对跨组织扩展, 需要更少依赖“信任各方”的方案 (区块链是一例, 但未必是通用解, 更简单高效的应用专用方案可能更好)。
- 持续监控 (防火墙、SIDS/AIDS、CIDS) 不可或缺; 实证表明安全与功能并非必须取舍——只要从一开始就考虑安全。

实践清单

- 读原书第 9 章, 重点看图 9.3 (窃听与入侵)、9.5 (Diffie-Hellman)、9.10 (reflection attack)、9.13 (Needham-Schroeder)、9.16 (Kerberos)、9.18 (TLS 1.3)、9.19 (Sybil 攻击)、9.20 (区块链哈希链)、9.23 (ACL vs capability)。
- 亲手在本地用 `ssh-keygen` 生成密钥、`ssh-copy-id` 分发, 观察公钥/私钥文件与 `.ssh` 目录, 理解“密钥即文件、以文件方式分发”。
- 对比 MAC/DAC/RBAC/ABAC 四种策略, 为“大学课程成绩表”各写一条访问规则, 并指出 ABAC 相比 RBAC 细在哪里。

- 用 ACL 与 capability 两种视角描述“Alice 让邮件客户端代收邮件”，再说明 OAuth 的 access token 更接近哪一种。
- 给一个 Sybil 攻击场景，解释 PoW/PoS 为什么让“多开身份”无利可图；再说明 NetFlow 记账为何抗 Sybil。
- 在假阳性与假阴性之间做一次取舍练习：为你所在系统选定可接受的 accuracy 与 precision 目标，并说明理由（安全场景通常更不容忍假阴性）。